



ÁRVORE BINÁRIA DE BUSCA BALANCEADA



Prof. Me. Darcy H.F.R. Moreira



EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

```
typedef struct NoArv NoArv;
typedef struct Arvore Arvore;

Arvore* criarArvore();
//Cria um novo nó com um valor específico.
static NoArv* criarNo(int valor);
//Insere um valor na árvore;
//Retorna um ponteiro para o nó inserido;
NoArv* inserir(Arvore* arvore, int valor);
//Busca um valor na árvore;
//Retorna um ponteiro para o nó, ou NULL se não encontrado.
NoArv* buscar(Arvore* arvore, int valor);
//Remove um valor da árvore, se existir.
void remover(Arvore* arvore, int valor);
//Imprime os valores da árvore em ordem
//(esquerda, raiz, direita)
void imprimirEmOrdem(NoArv* raiz);
//Retorna a raiz de uma árvore
NoArv* obterRaiz(Arvore* arvore);
//Libera a memória alocada para a árvore e seus nós.
void liberarArvore(NoArv* raiz);
```

EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

Essa função deve:

- 1.Localizar o **valor** na árvore;
- a.caso o valor **não exista**, imprimir:

i.Valor X não encontrado na árvore.
- b.caso o valor **exista**, informar:

■ se o nó não possui filhos;

■ se o nó possui apenas um filho;

■ se o nó possui dois filhos.

```
typedef struct NoArv NoArv;
typedef struct Arvore Arvore;

Arvore* criarArvore();
//Cria um novo nó com um valor específico.
static NoArv* criarNo(int valor);
//Insere um valor na árvore;
//Retorna um ponteiro para o nó inserido;
NoArv* inserir(Arvore* arvore, int valor);
//Busca um valor na árvore;
//Retorna um ponteiro para o nó, ou NULL se não encontrado.
NoArv* buscar(Arvore* arvore, int valor);
//Remove um valor da árvore, se existir.
void remover(Arvore* arvore, int valor);
//Imprime os valores da árvore em ordem
//(esquerda, raiz, direita)
void imprimirEmOrdem(NoArv* raiz);
//Retorna a raiz de uma árvore
NoArv* obterRaiz(Arvore* arvore);
//Libera a memória alocada para a árvore e seus nós.
void liberarArvore(NoArv* raiz);
```

EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

Essa função deve:

- 1.Localizar o **valor** na árvore;
- a.caso o valor **não exista**, imprimir:

i.Valor X não encontrado na árvore.

b.caso o valor **exista**, informar:

■ se o nó não possui filhos;

■ se o nó possui apenas um filho;

■ se o nó possui dois filhos.

```

typedef struct NoArv NoArv;
typedef struct Arvore Arvore;

Arvore* criarArvore();
//Cria um novo nó com um valor específico.
static NoArv* criarNo(int valor);
//Insere um valor na árvore;
//Retorna um ponteiro para o nó inserido;
NoArv* inserir(Arvore* arvore, int valor);
//Busca um valor na árvore;
//Retorna um ponteiro para o nó, ou NULL se não encontrado.
NoArv* buscar(Arvore* arvore, int valor);
//Remove um valor da árvore, se existir.
void remover(Arvore* arvore, int valor);
//Imprime os valores da árvore em ordem
//(esquerda, raiz, direita)
void imprimirEmOrdem(NoArv* raiz);
//Retorna a raiz de uma árvore
NoArv* obterRaiz(Arvore* arvore);
//Libera a memória alocada para a árvore e seus nós.
void liberarArvore(NoArv* raiz);

```

EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

Essa função deve:

1. Localizar o **valor** na árvore;
 - a. caso o valor **não exista**, imprimir:
 - i. Valor X não encontrado na árvore.
 - b. caso o valor **exista**, informar:

- se o nó não possui filhos;
- se o nó possui apenas um filho;
- se o nó possui dois filhos.



EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

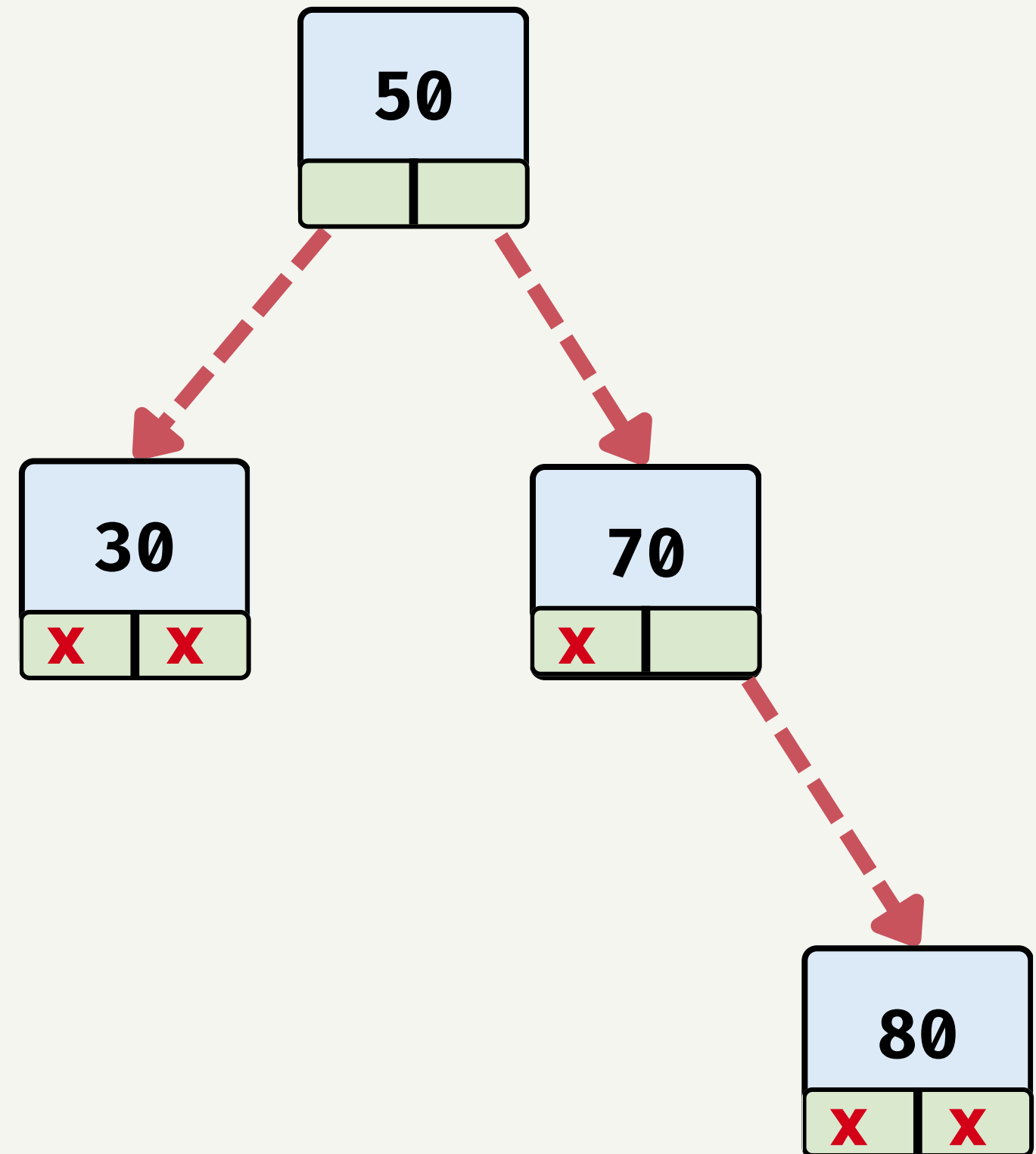
Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

Essa função deve:

1. Localizar o **valor** na árvore;
 - a. caso o valor **não exista**, imprimir:
 - i. Valor X não encontrado na árvore.
 - b. caso o valor **exista**, informar:

- se o nó não possui filhos;
- se o nó possui apenas um filho;
- se o nó possui dois filhos.



EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

```
void verificarFilhos(Arvore* arvore, int valor) {  
    NoArv* no = buscar(arvore, valor);
```

Essa função deve:

1. Localizar o **valor** na árvore;
 - a. caso o valor **não exista**, imprimir:
 - i. Valor X não encontrado na árvore.
 - b. caso o valor **exista**, informar:
 - se o nó não possui filhos;
 - se o nó possui apenas um filho;
 - se o nó possui dois filhos.

EXERCÍCIO

Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

```
void verificarFilhos(Arvore* arvore, int valor) {  
    NoArv* no = buscar(arvore, valor);  
  
    if (no == NULL) {  
        printf("Valor %d não encontrado na árvore.\n", valor);  
        return;  
    }  
}
```

Essa função deve:

1. Localizar o **valor** na árvore;

a. caso o valor **não exista**, imprimir:

i. **Valor X não encontrado na árvore.**

b. caso o valor **exista**, informar:

- se o nó não possui filhos;
- se o nó possui apenas um filho;
- se o nó possui dois filhos.

EXERCÍCIO

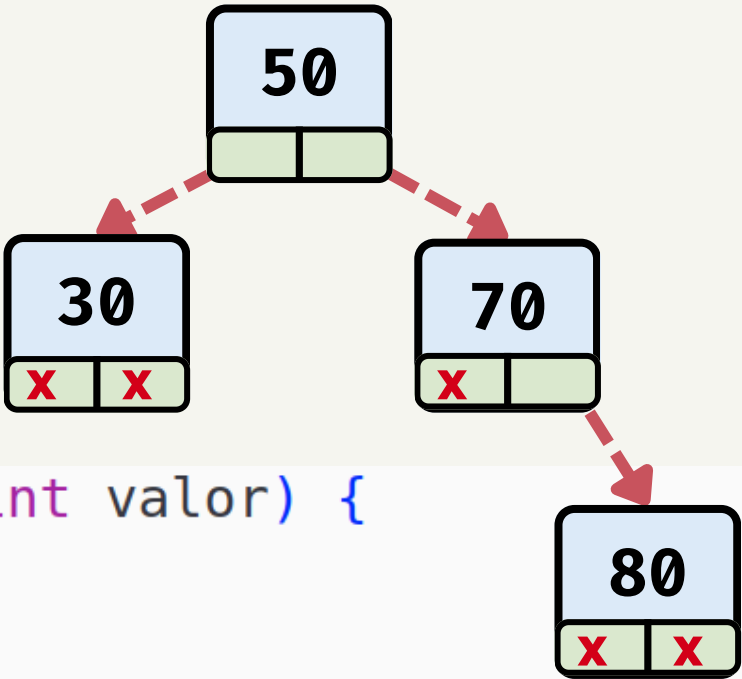
Considere a implementação de **árvore binária de busca** desenvolvida em sala, contendo as funções:

Implemente a seguinte **função**:

void verificarFilhos(Arvore* arvore, int valor);

Essa função deve:

- Localizar o **valor** na árvore;
 - caso o valor **não exista**, imprimir:
 - Valor X não encontrado na árvore.
 - caso o valor **exista**, informar:
 - se o nó não possui filhos;
 - se o nó possui apenas um filho;
 - se o nó possui dois filhos.



```

void verificarFilhos(Arvore* arvore, int valor) {
    NoArv* no = buscar(arvore, valor);

    if (no == NULL) {
        printf("Valor %d não encontrado na árvore.\n", valor);
        return;
    }

    if (no->esq == NULL && no->dir == NULL) {
        printf("O nó %d não possui filhos.\n", valor);
    } else if (no->esq != NULL && no->dir != NULL) {
        printf("O nó %d possui dois filhos.\n", valor);
    } else {
        printf("O nó %d possui apenas um filho.\n", valor);
    }
}
  
```

EXERCÍCIO

Dada uma **árvore binária de busca** inicialmente vazia, insira os seguintes elementos, nesta ordem, e depois:

Desenhe a árvore resultante e classifique:

Completa ou incompleta?

Balanceada ou Desbalanceada?

Elementos a serem inseridos:

8, 4, 12, 2, 6, 10, 14, 1, 3, 5

EXERCÍCIO

Dada uma **árvore binária de busca** inicialmente vazia, insira os seguintes elementos, nesta ordem, e depois:

Desenhe a árvore resultante e classifique:

Completa ou incompleta?

Elementos a serem inseridos:

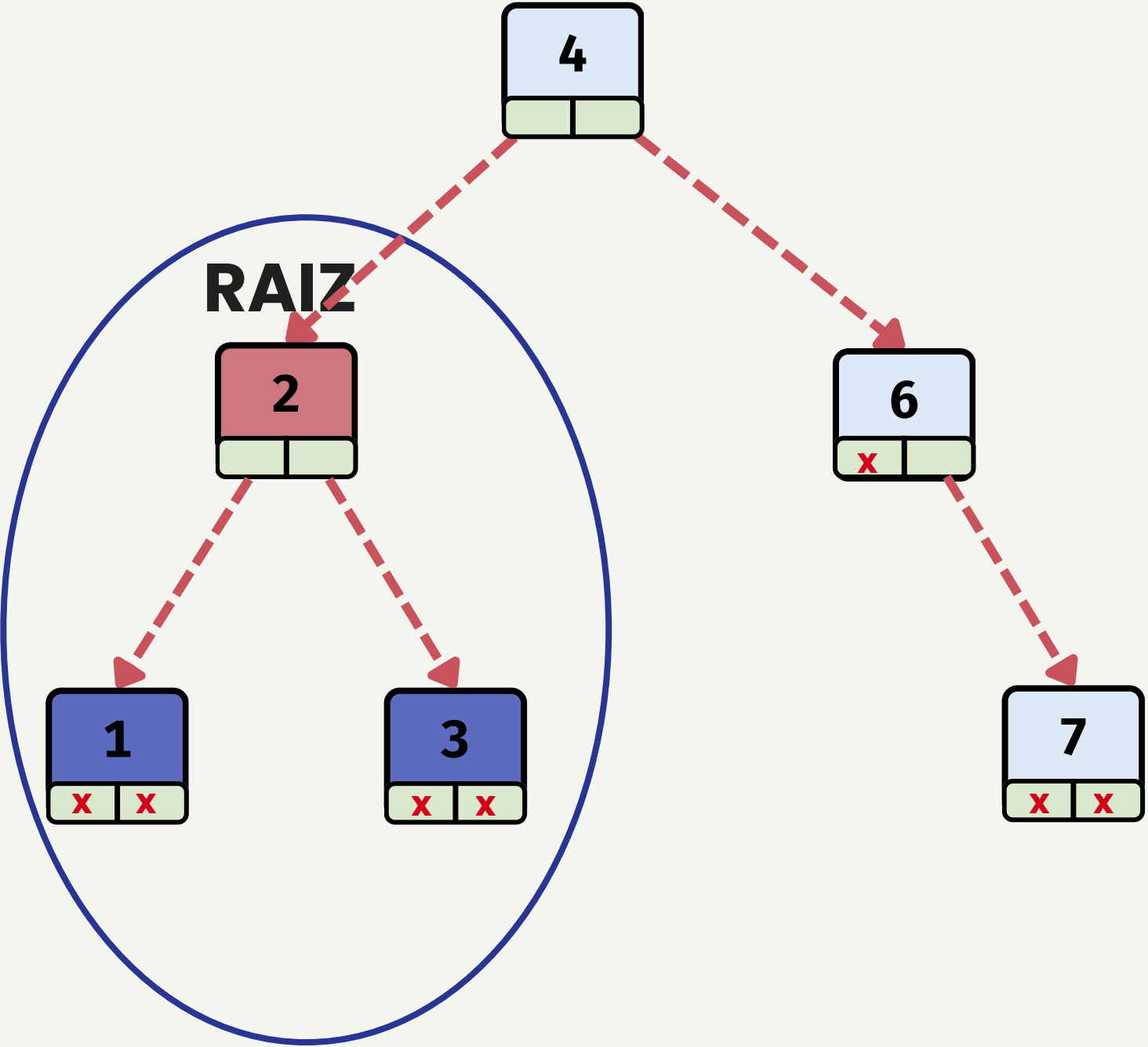
8, 4, 12, 2, 6, 10, 14, 1, 3, 5

Árvore completa:

Todos os níveis estão **completos**, exceto possivelmente o último, que é preenchido da **esquerda** para a **direita**.

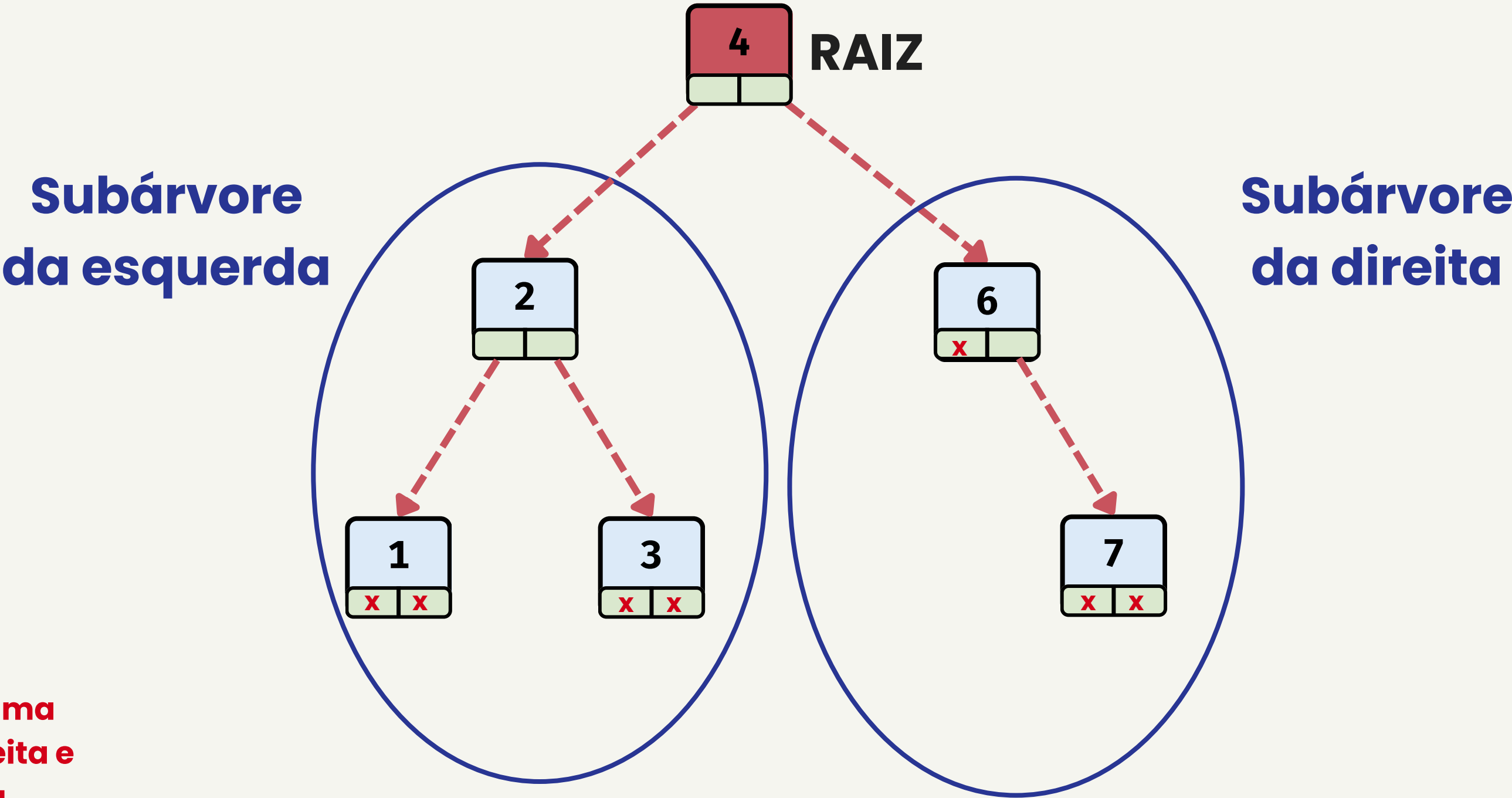
ÁRVORES BINÁRIAS DE BUSCA

Subárvore: Nó + seus descendentes



ÁRVORES BINÁRIAS DE BUSCA

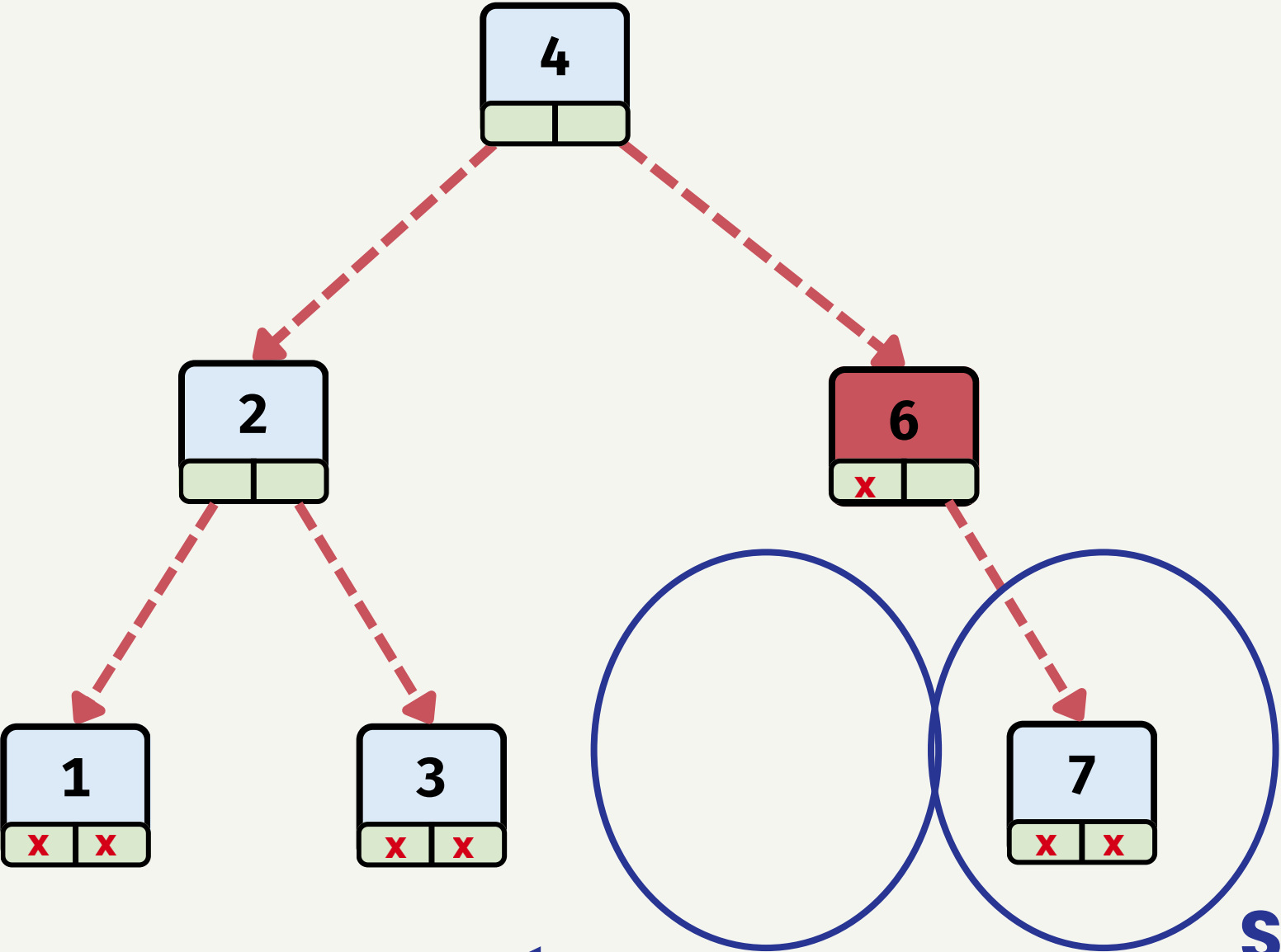
Subárvore: Nó + seus descendentes



Todo nó tem uma subárvore à direita e à esquerda

ÁRVORES BINÁRIAS DE BUSCA

Subárvore: Nó + seus descendentes



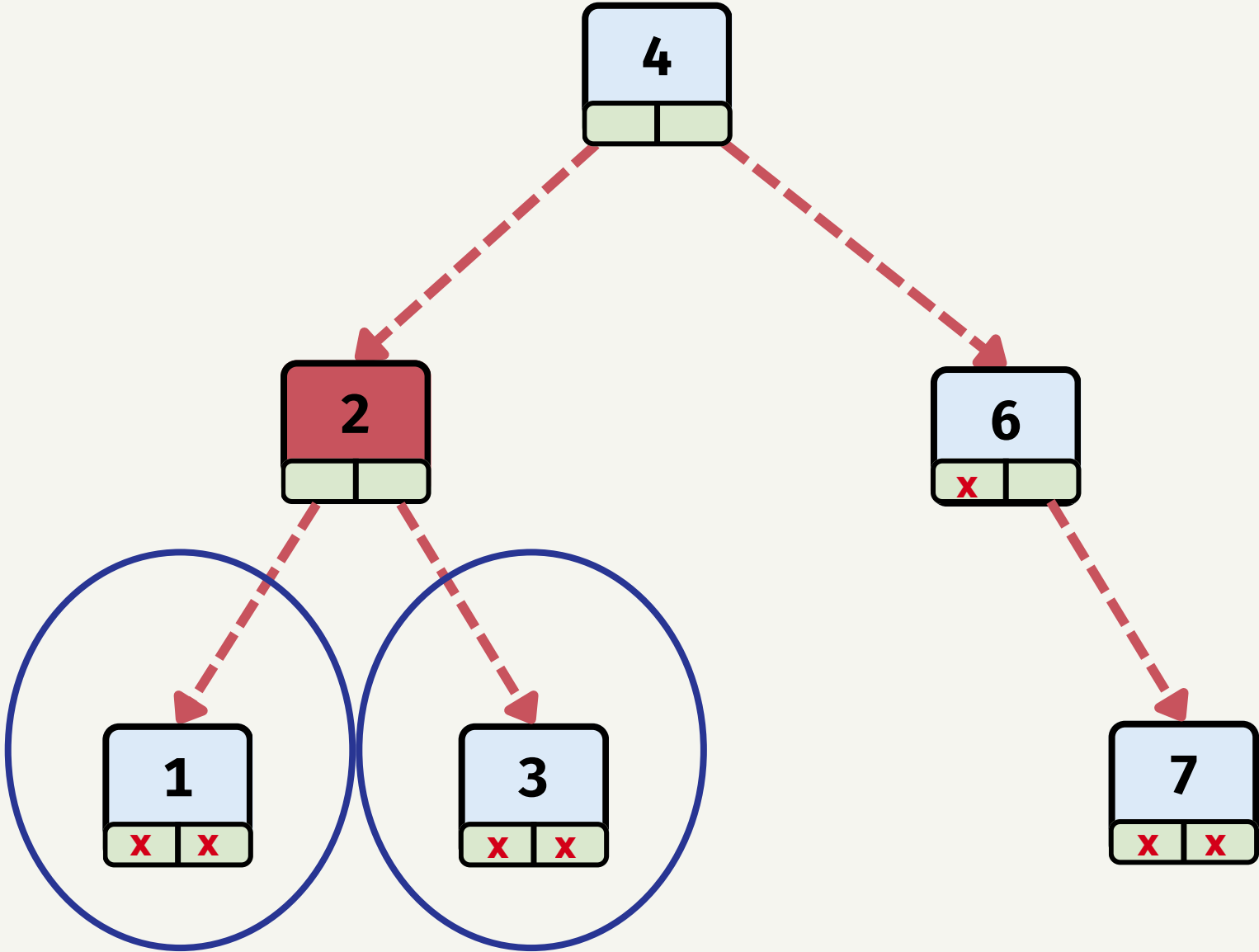
Todo nó tem uma subárvore à direita e à esquerda

Subárvore da esquerda

Subárvore da direita

ÁRVORES BINÁRIAS DE BUSCA

Subárvore: Nó + seus descendentes



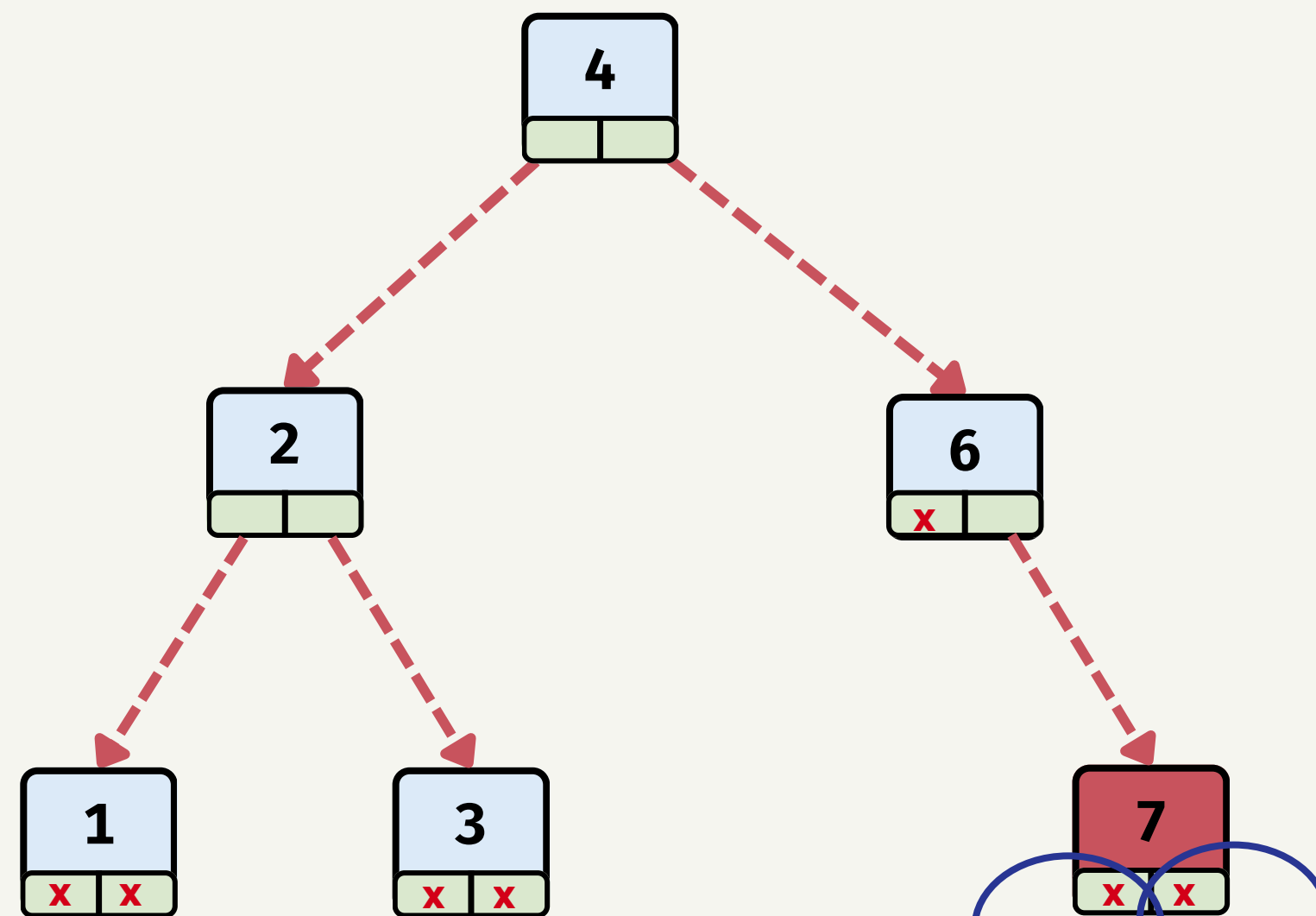
Subárvore da esquerda

Subárvore da direita

Todo nó tem uma subárvore à direita e à esquerda

ÁRVORES BINÁRIAS DE BUSCA

Subárvore: Nó + seus descendentes



Todo nó tem uma subárvore à direita e à esquerda

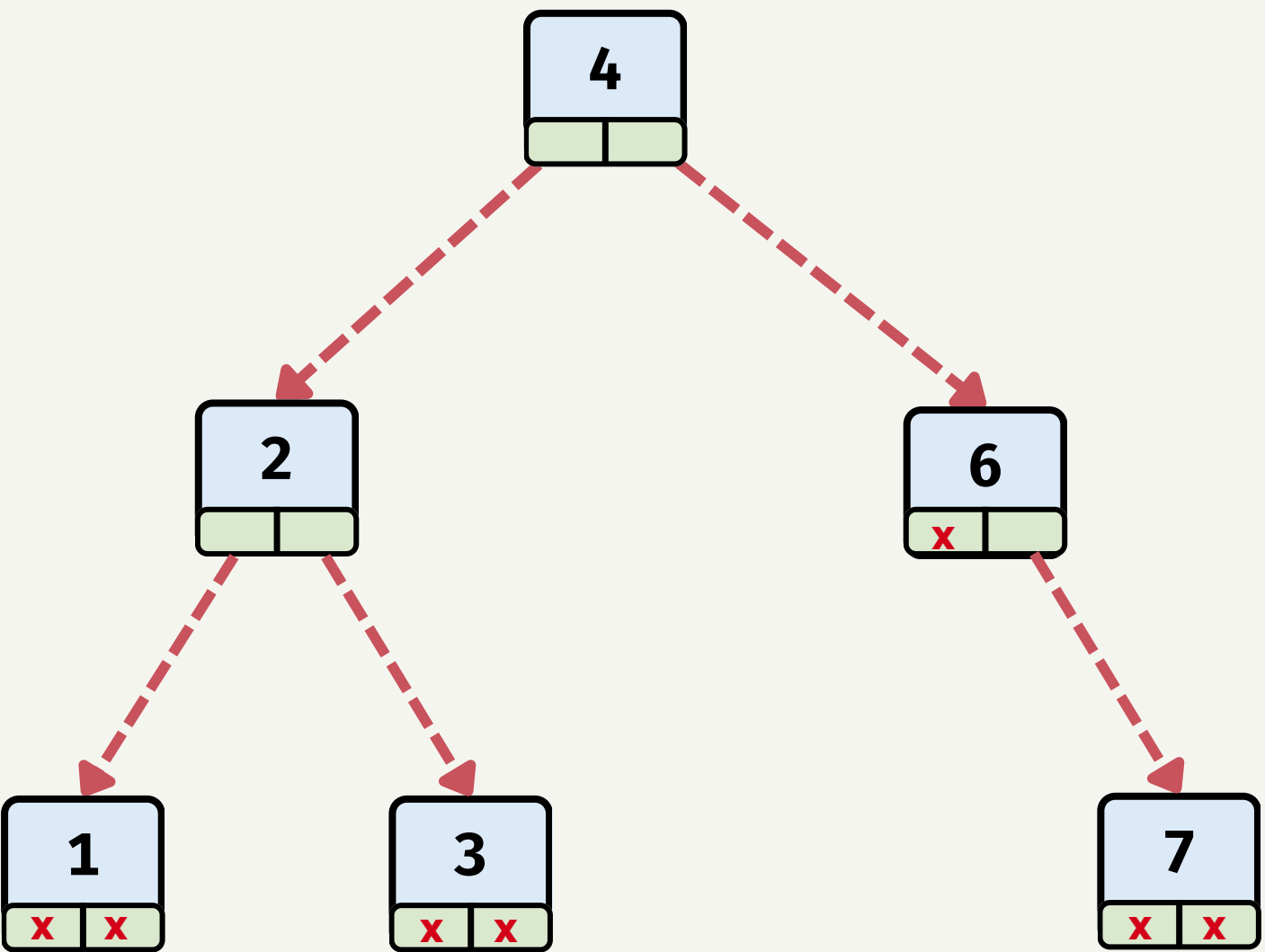
Subárvore da esquerda

Subárvore da direita

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.

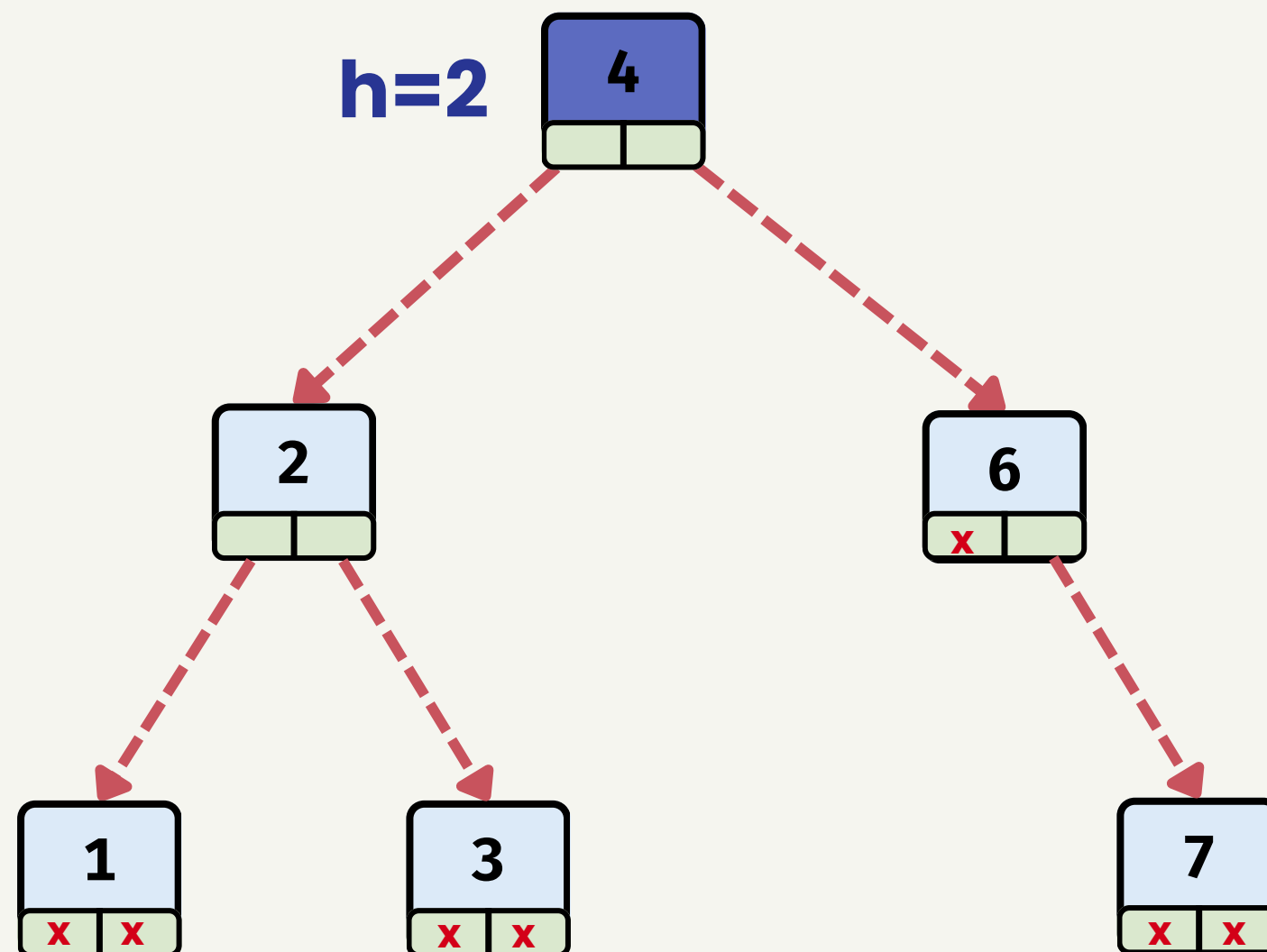
Distância do nó à folha



ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.

Distância do nó à folha

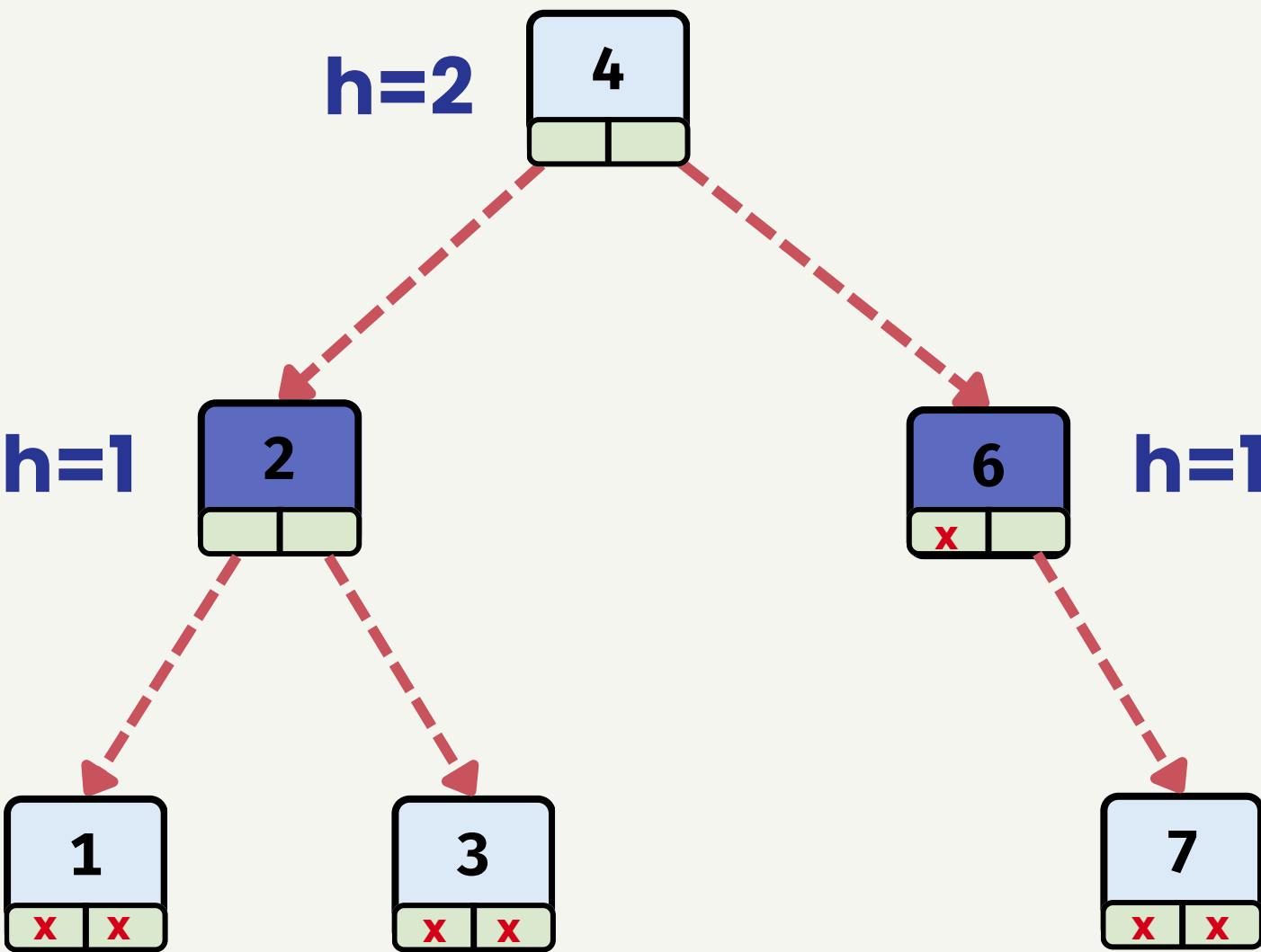


Altura da raiz = altura da árvore

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.

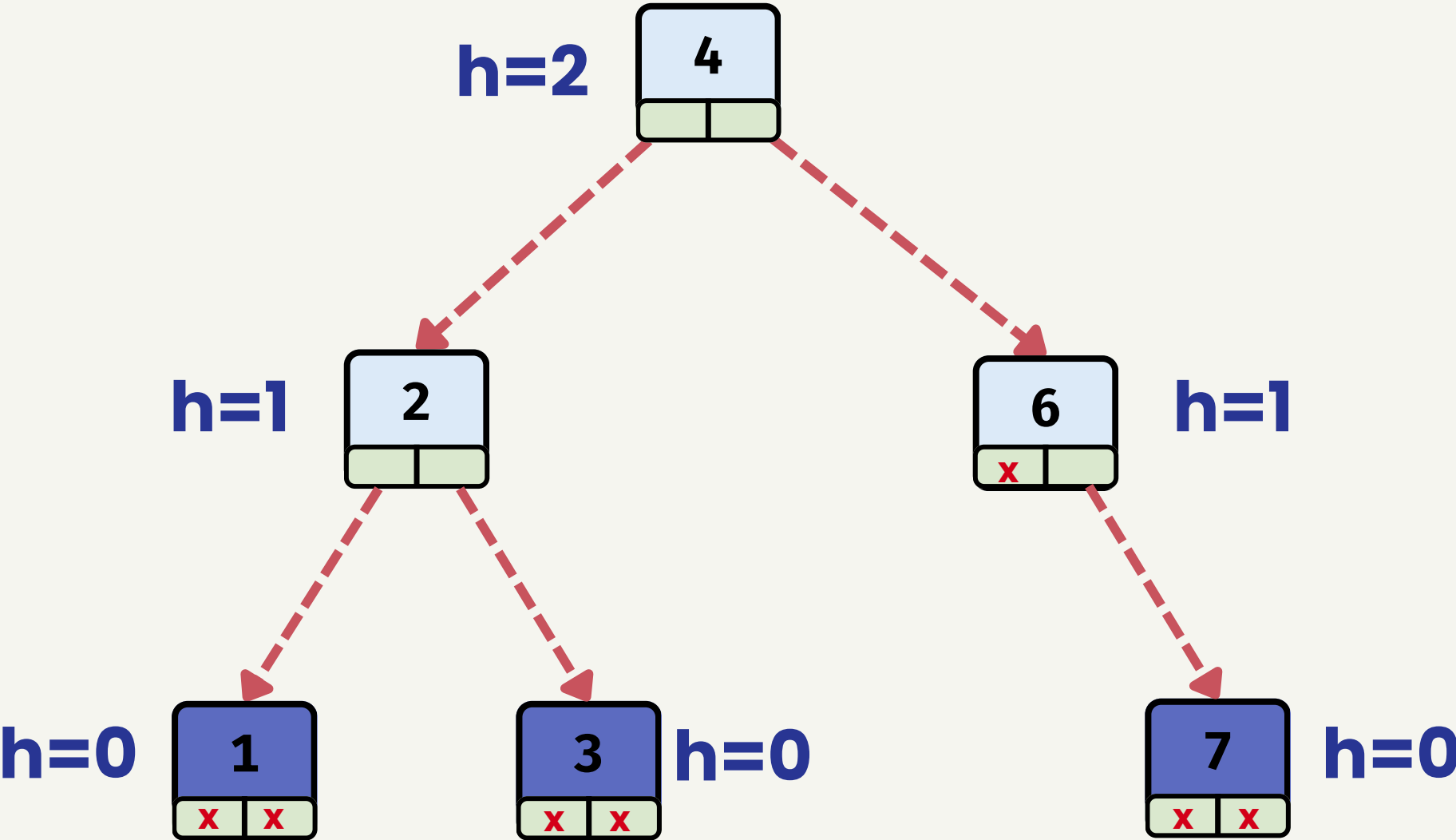
Distância do nó à folha



Altura da raiz = altura da árvore

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.
Distância do nó à folha

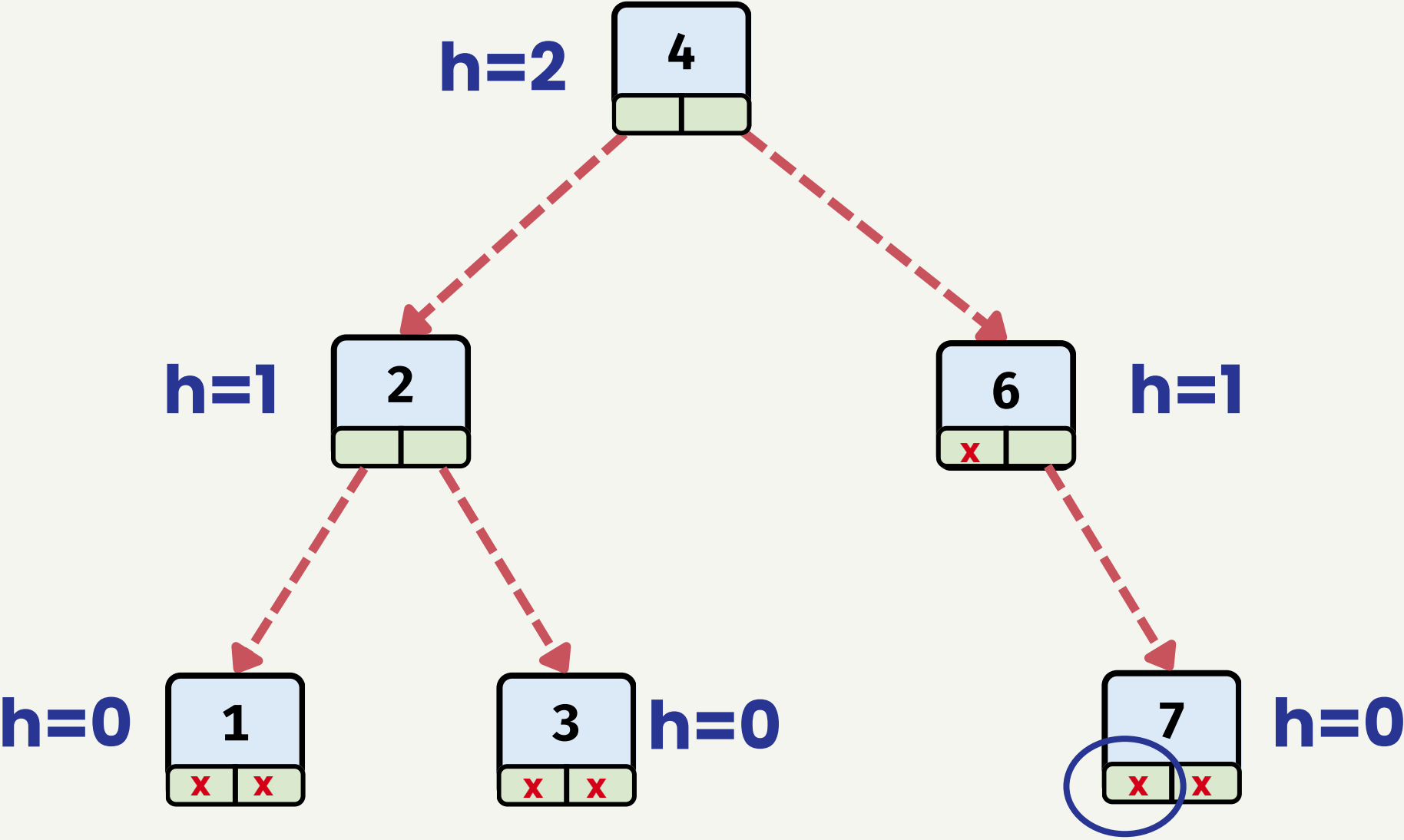


Altura da raiz = altura da árvore

Altura de uma folha= 0

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.
Distância do nó à folha



Altura da raiz = altura da árvore

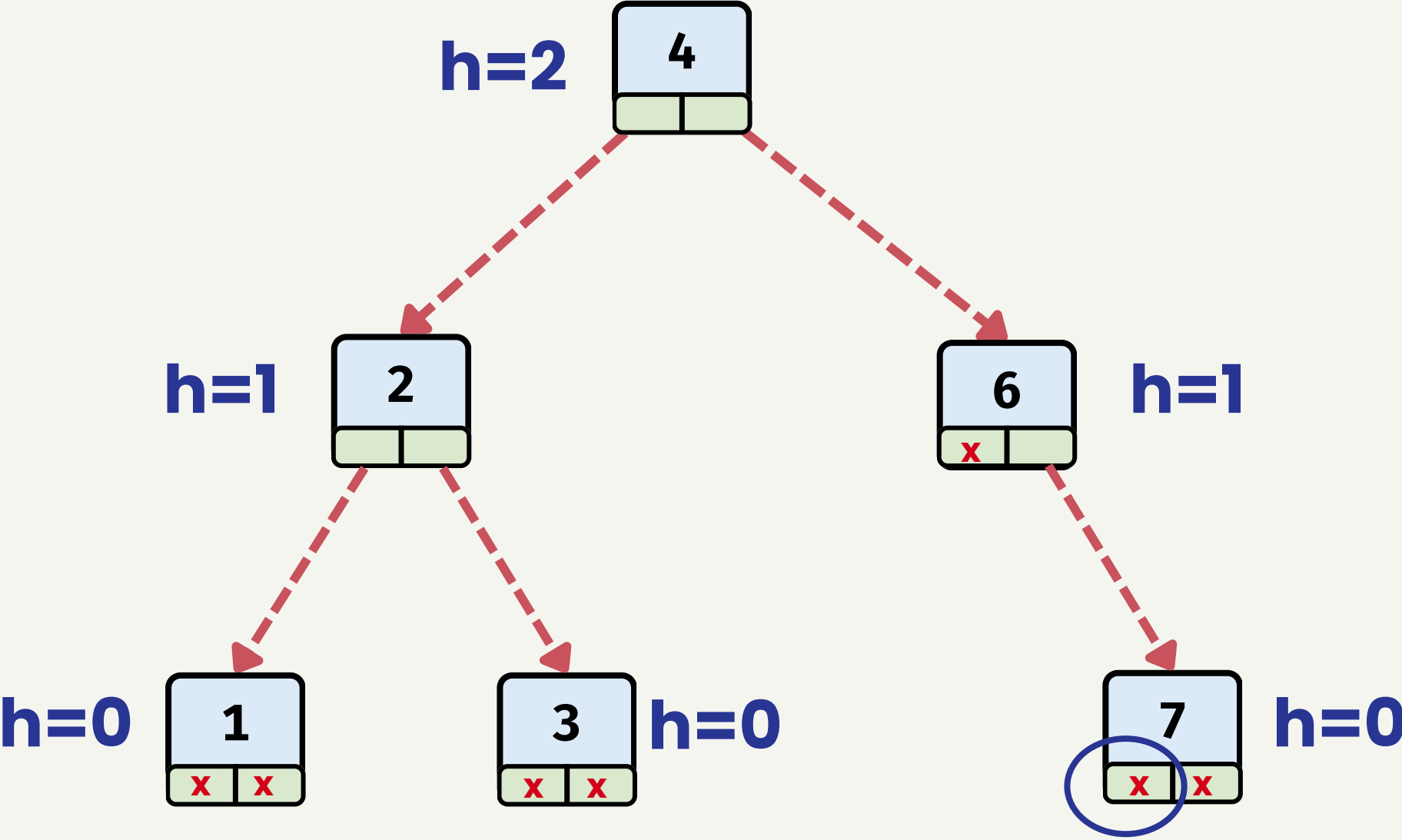
Altura de uma folha= 0

Altura de um nó NULL??

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.

Distância do nó à folha



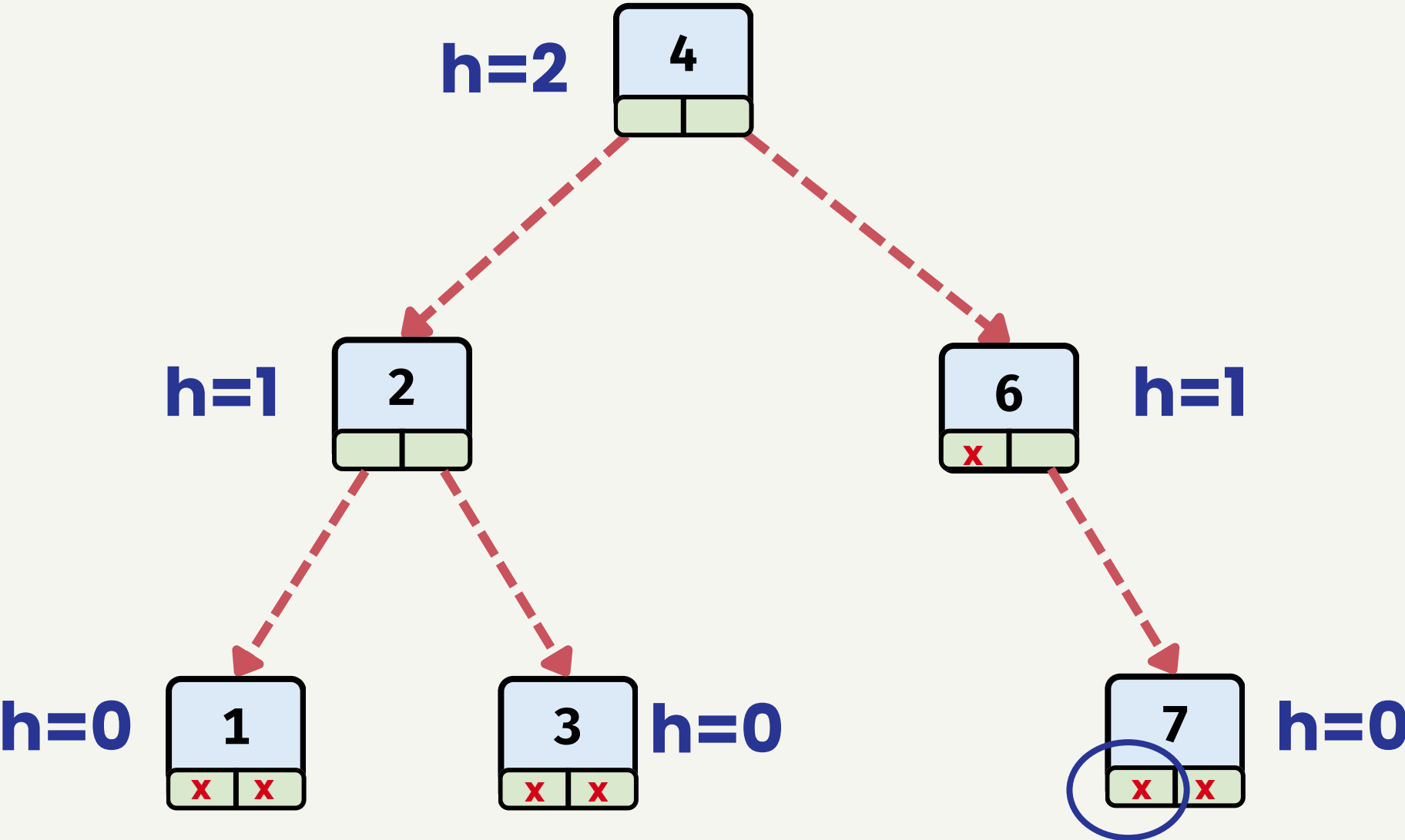
Altura da raiz = altura da árvore

Altura de uma folha= 0

Altura de um nó NULL=-1

ÁRVORES BINÁRIAS DE BUSCA

ALTURA DE UM NÓ: número de arestas no maior caminho desse nó até uma folha.
Distância do nó à folha



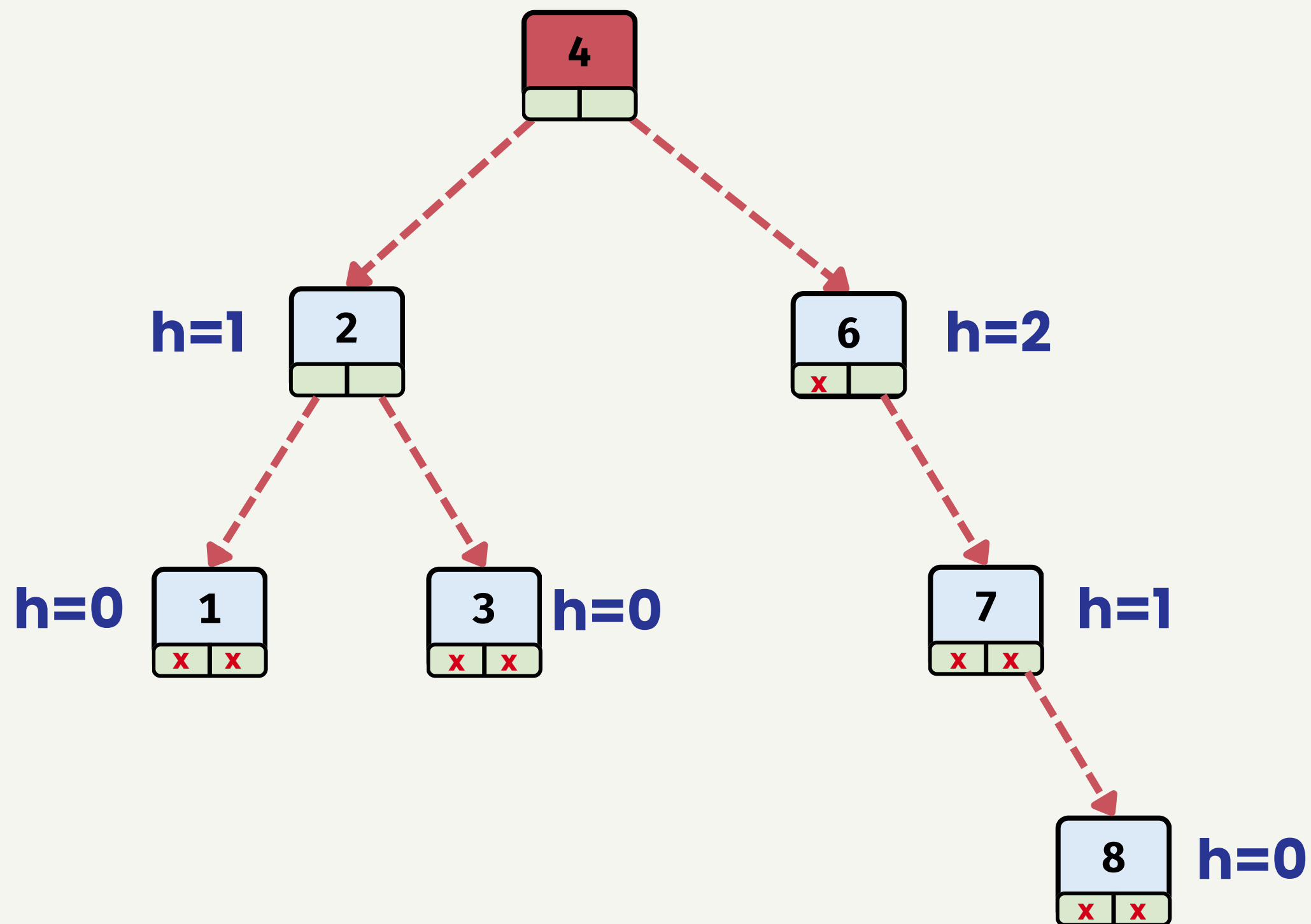
Altura da raiz = altura da árvore

Altura de uma folha= 0

Altura de um nó NULL=-1

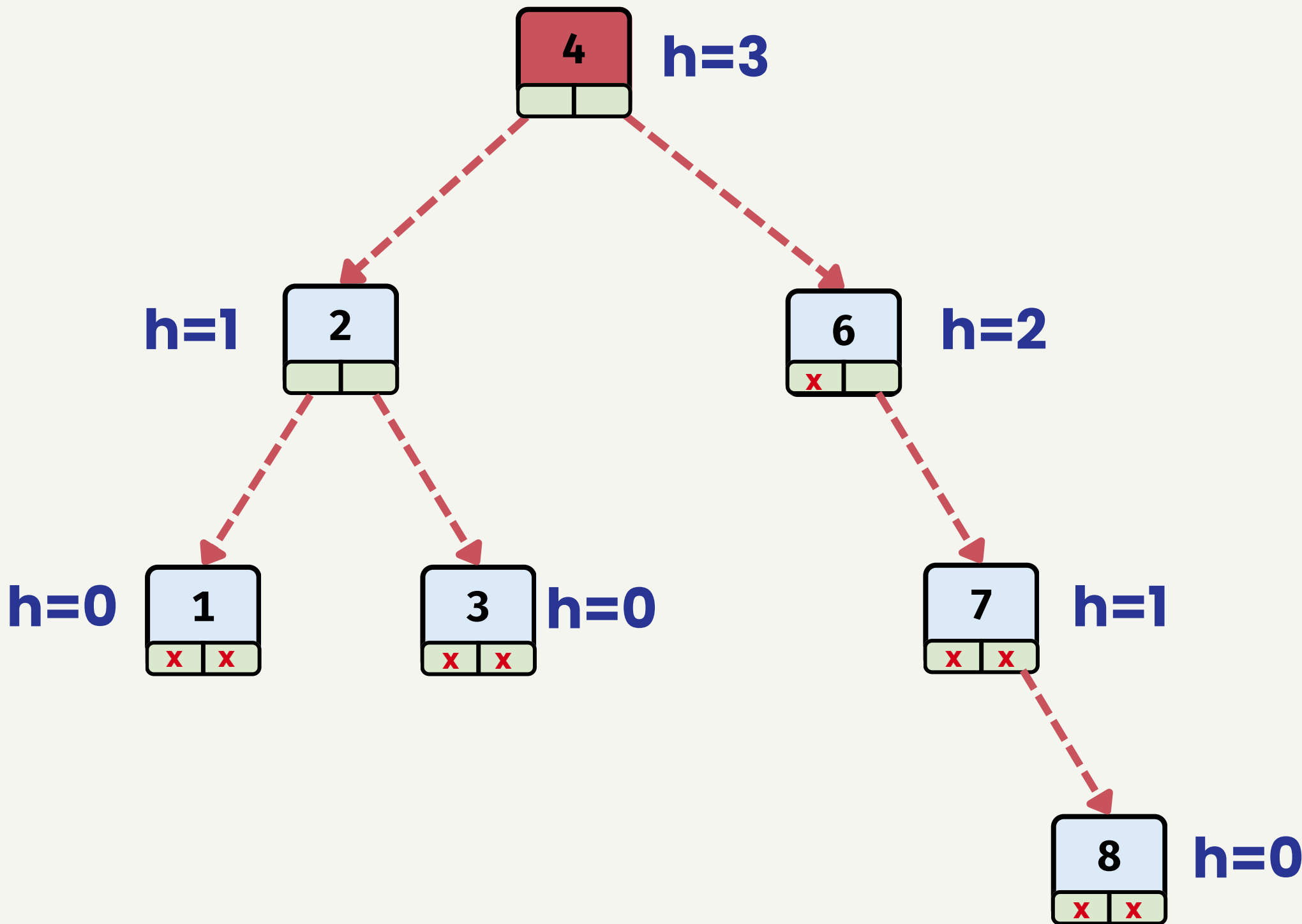
ALTURA: propriedade do NÓ!!!

ÁRVORES BINÁRIAS DE BUSCA



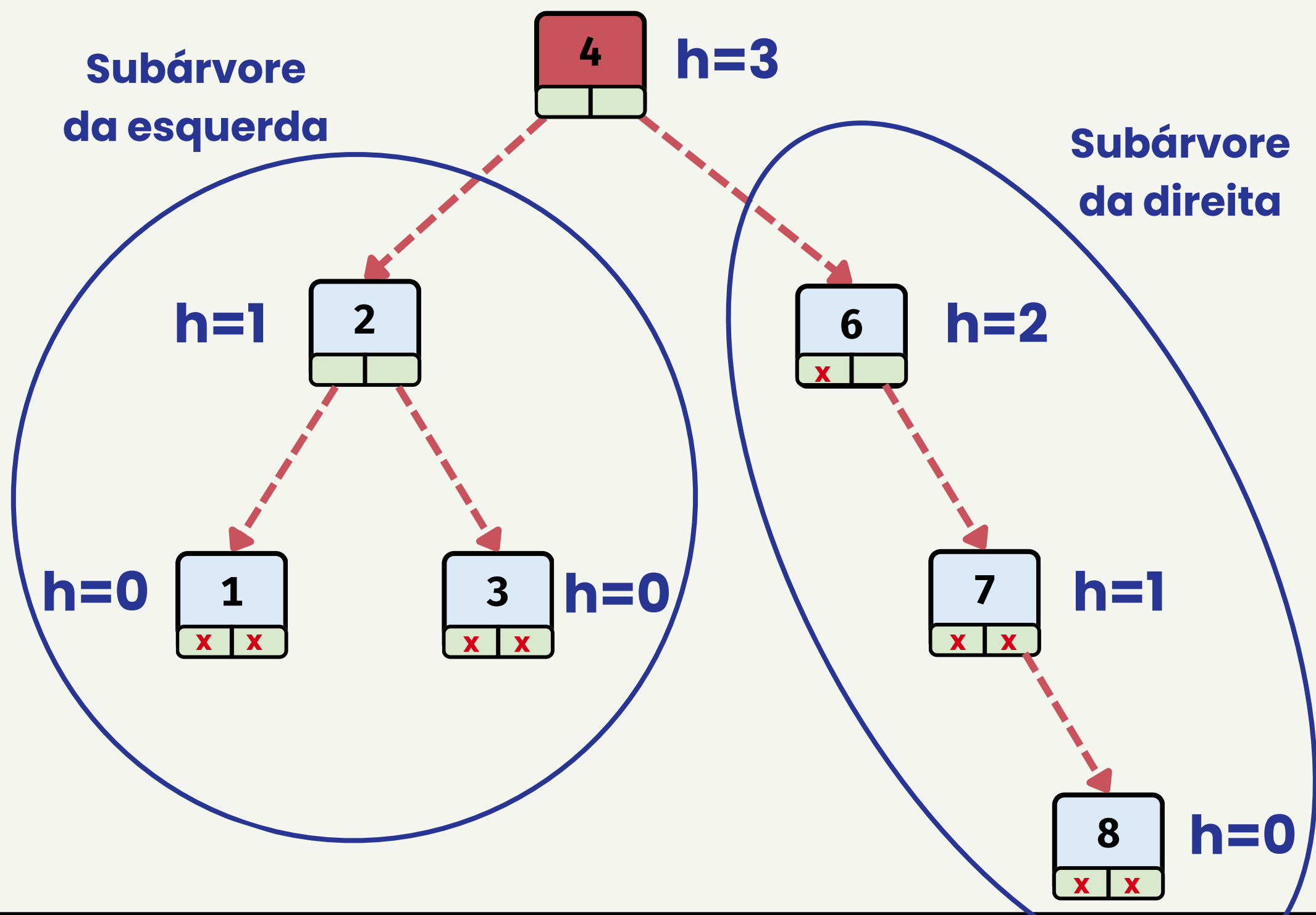
Altura do nó 4 = ?

ÁRVORES BINÁRIAS DE BUSCA



Altura do nó 4 = 3

ÁRVORES BINÁRIAS DE BUSCA



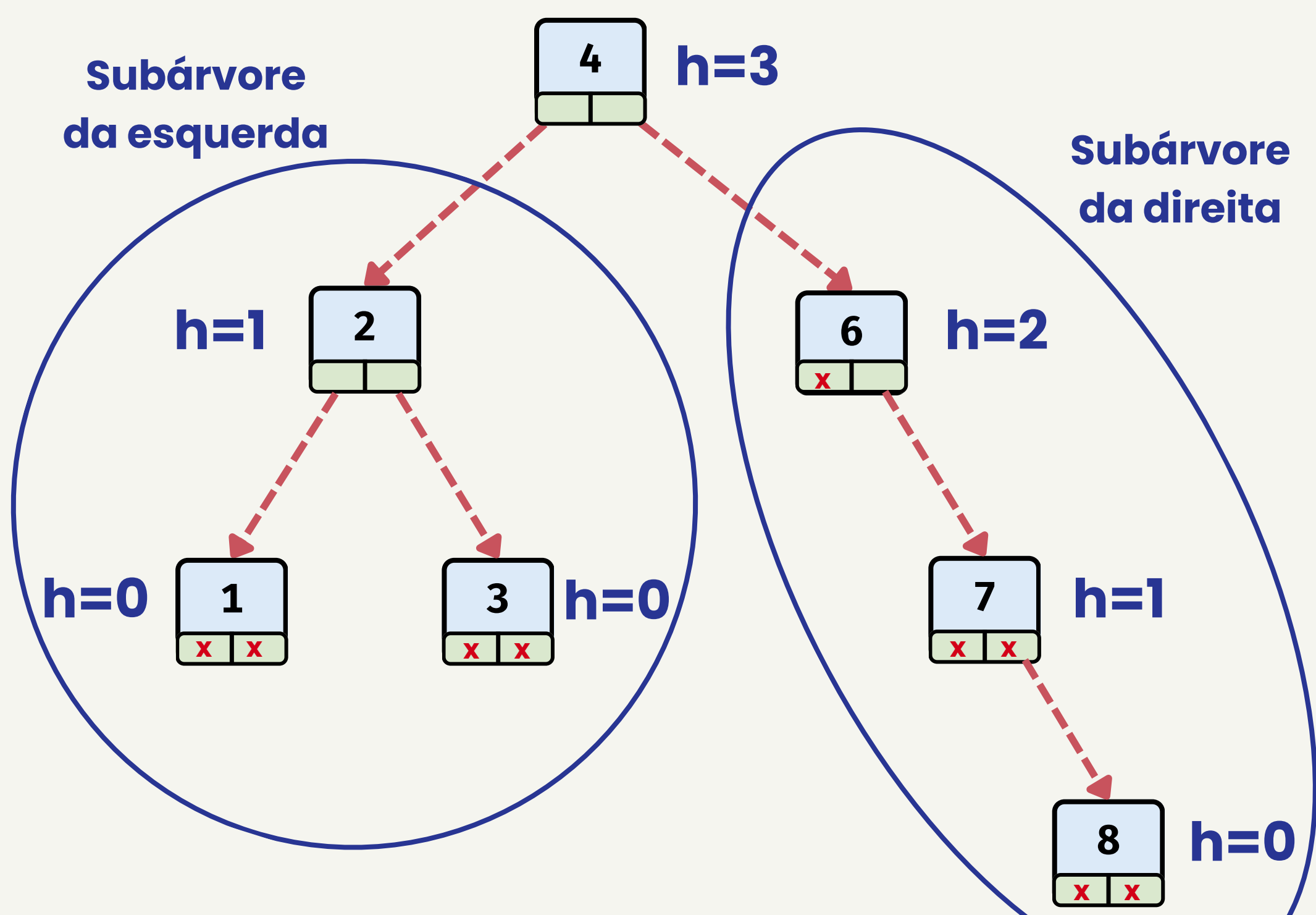
Altura do nó 4 = 3

=

Altura da subárvore da direita +1

ÁRVORES BINÁRIAS DE BUSCA

Convenção Altura: Maior altura entre a subárvore da direita e esquerda +1



Altura do nó 4 = 3

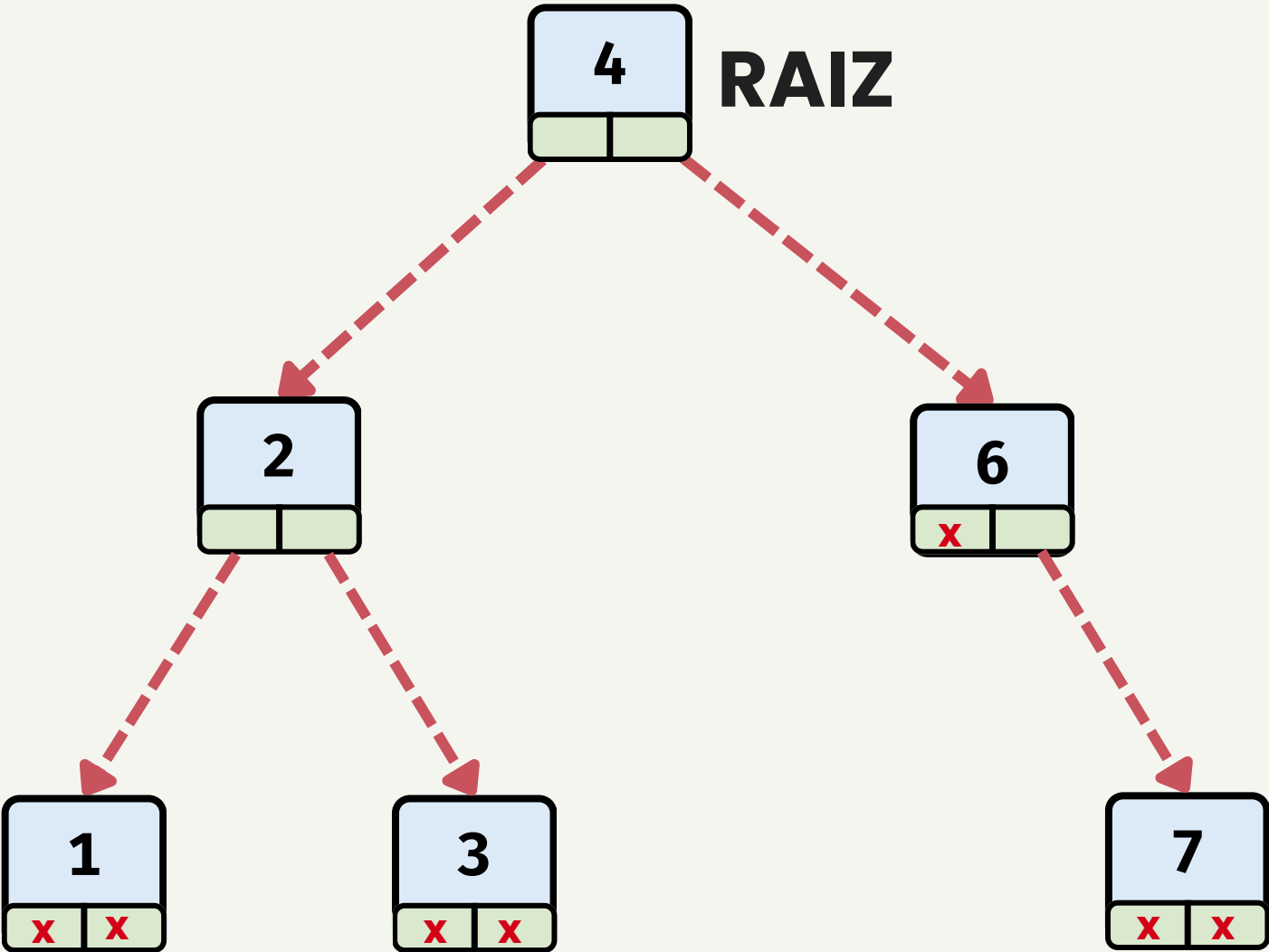
=

Altura da subárvore da direita +1

AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

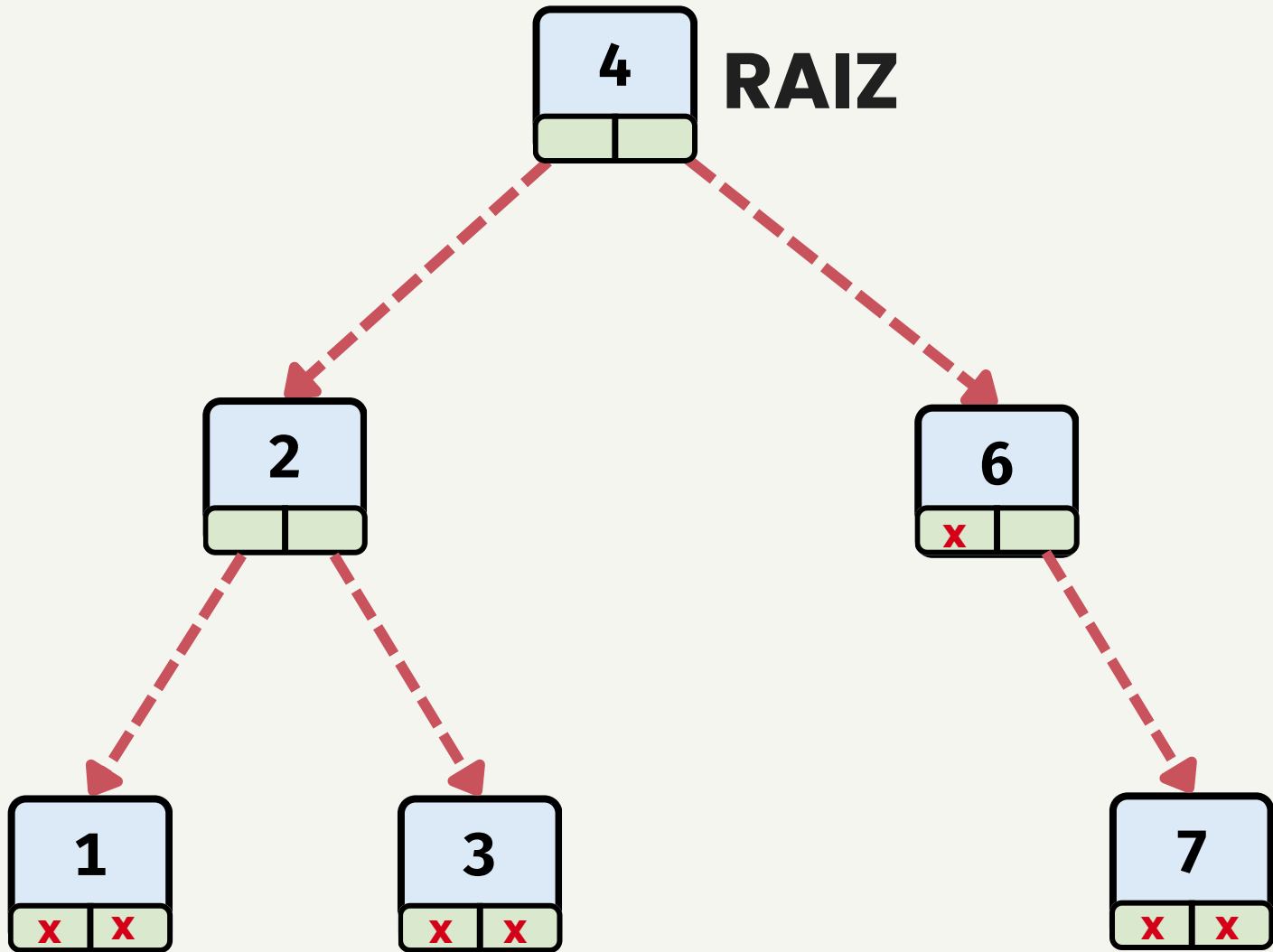


AVL

AVL – Árvore Binária de Busca Balanceada Adelson-Velsky e Landis

REGRA DE ORDENAÇÃO:

Para **qualquer** nó da árvore:
Elementos **MENORES: ESQUERDA**
Elementos **MAIORES: DIREITA**



AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

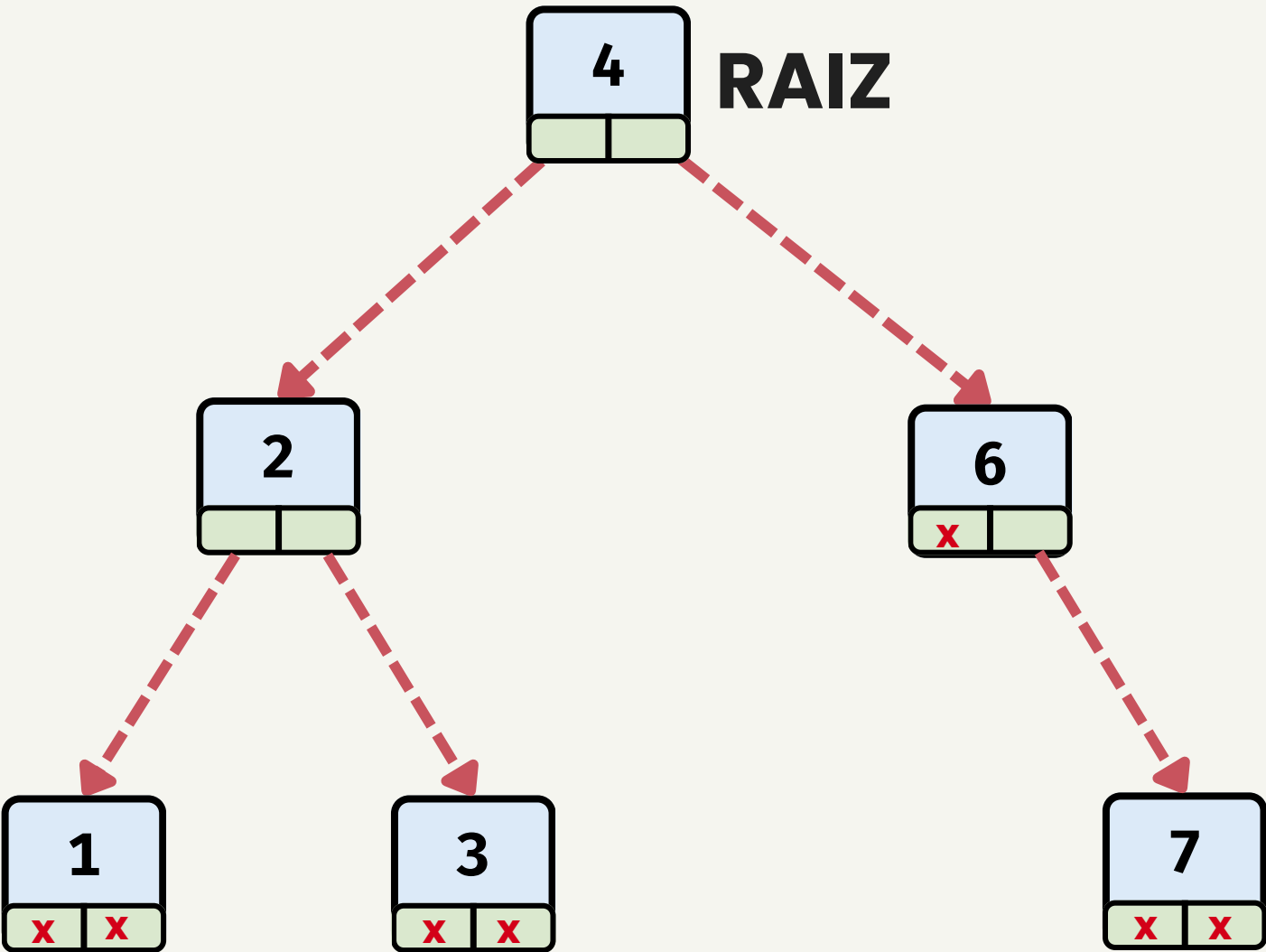
REGRA DE ORDENAÇÃO:

Para **qualquer** nó da árvore:

Elementos **MENORES: ESQUERDA**

Elementos **MAIORES: DIREITA**

Balanceado = equilibrado



AVL

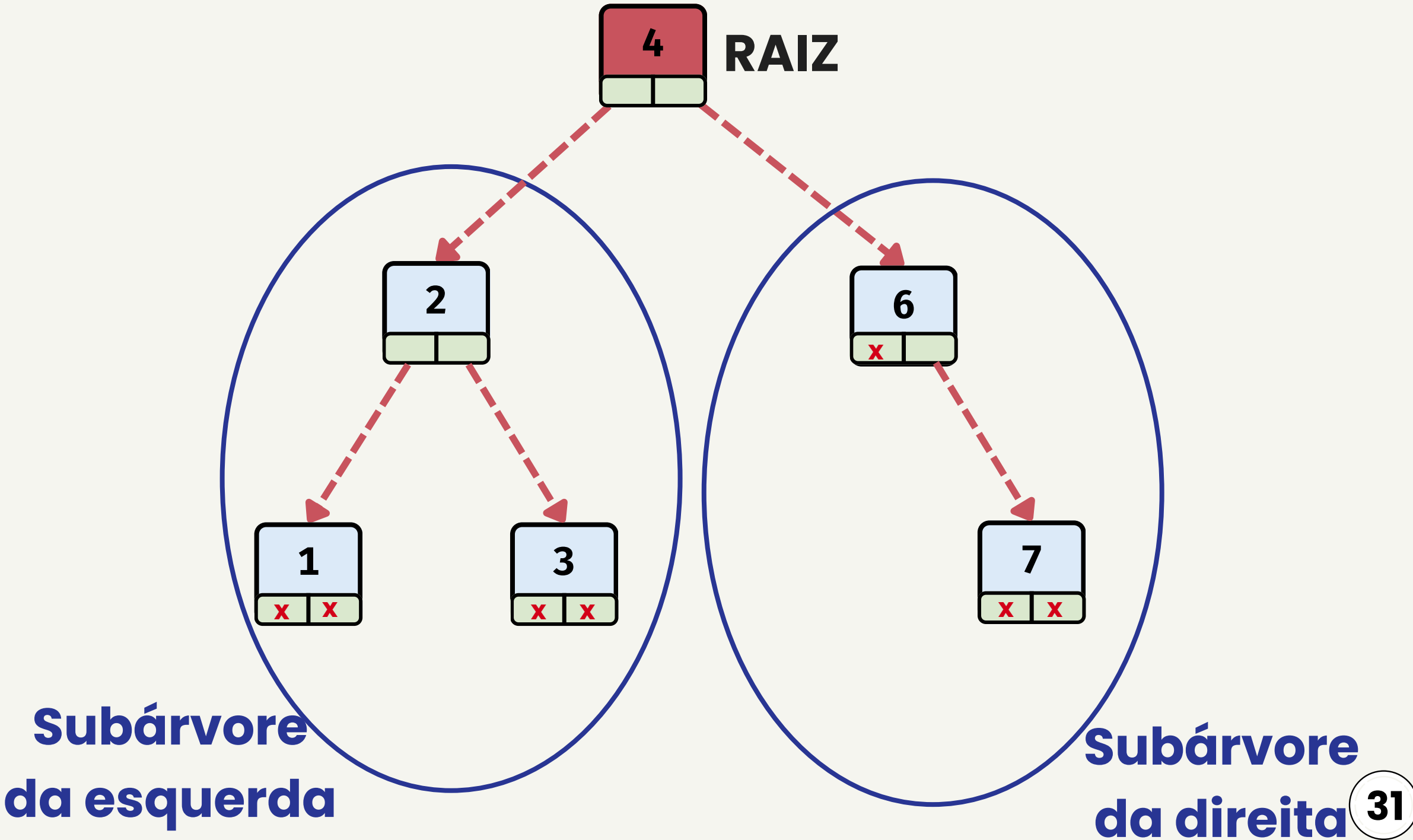
AVL – Árvore Binária de Busca Balanceada Adelson-Velsky e Landis

REGRA DE ORDENAÇÃO:

Para **qualquer** nó da árvore:
Elementos **MENORES: ESQUERDA**
Elementos **MAIORES: DIREITA**

Balanceado = equilibrado

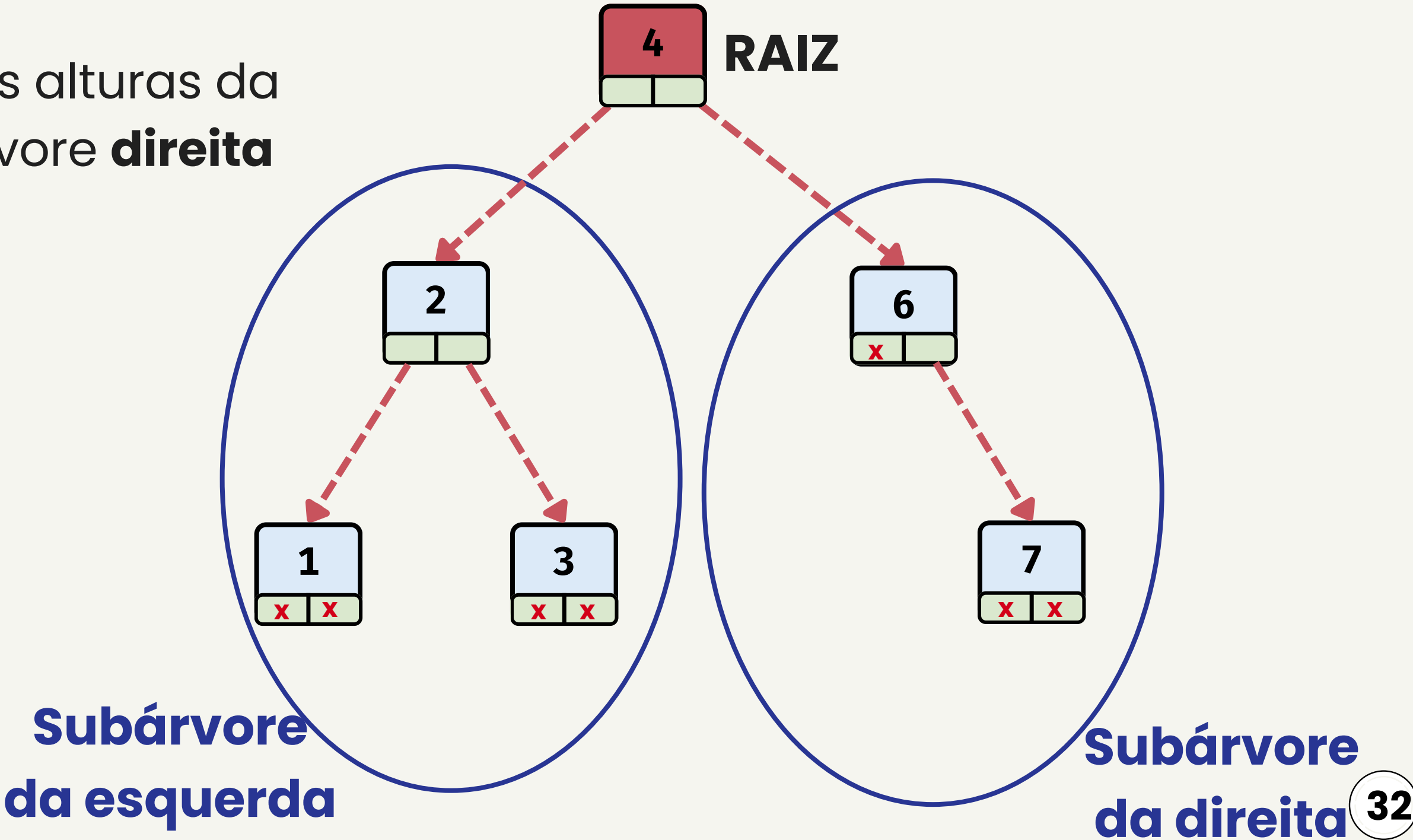
Manter as alturas de **TODAS**
subárvores da esquerda e direita
com a menor diferença possível



AVL

AVL – Árvore Binária de Busca Balanceada Adelson-Velsky e Landis

Em cada nó, a **diferença** entre as alturas da subárvore **esquerda** e da subárvore **direita** deve ser **-1, 0** ou **1**



AVL

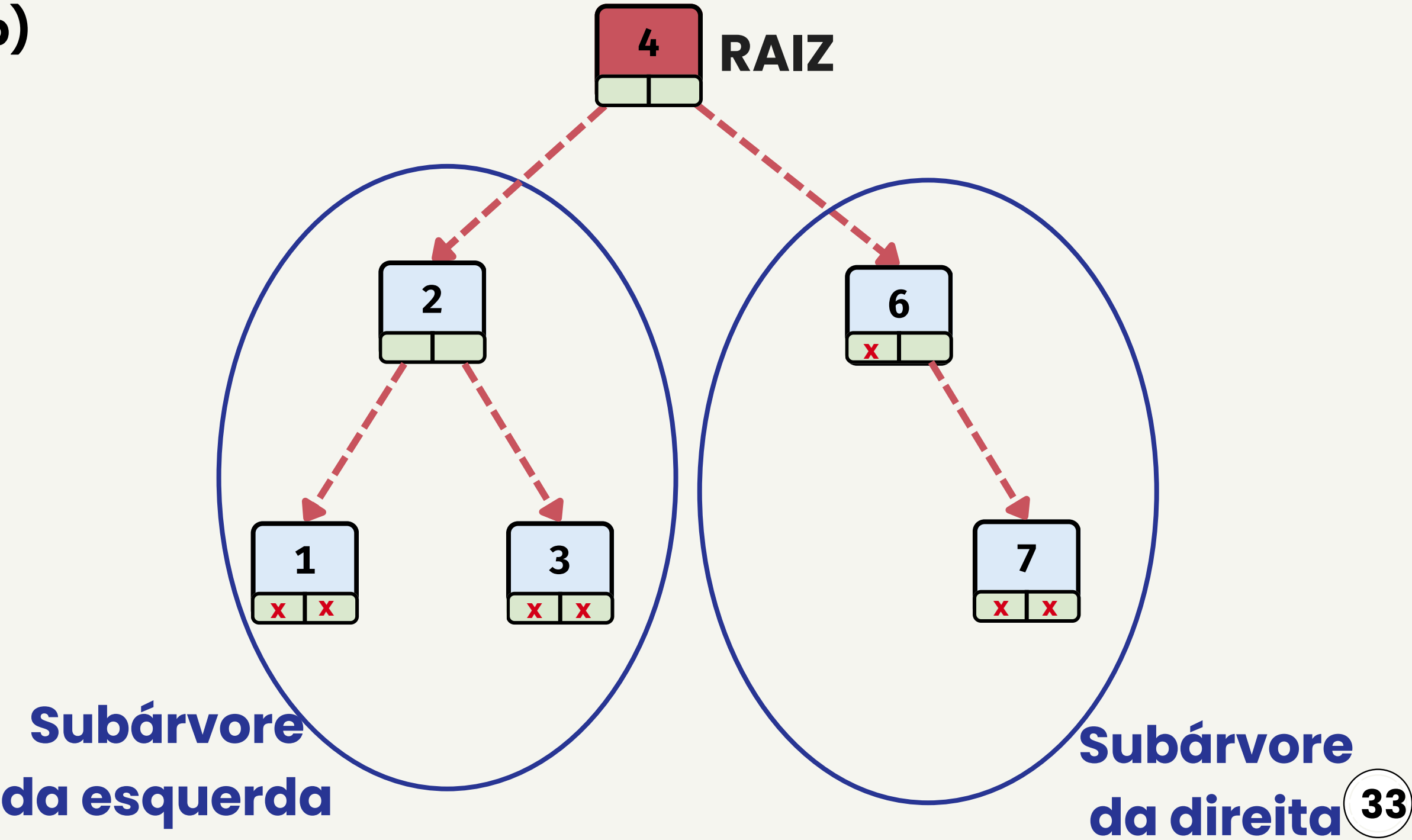
AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada



AVL

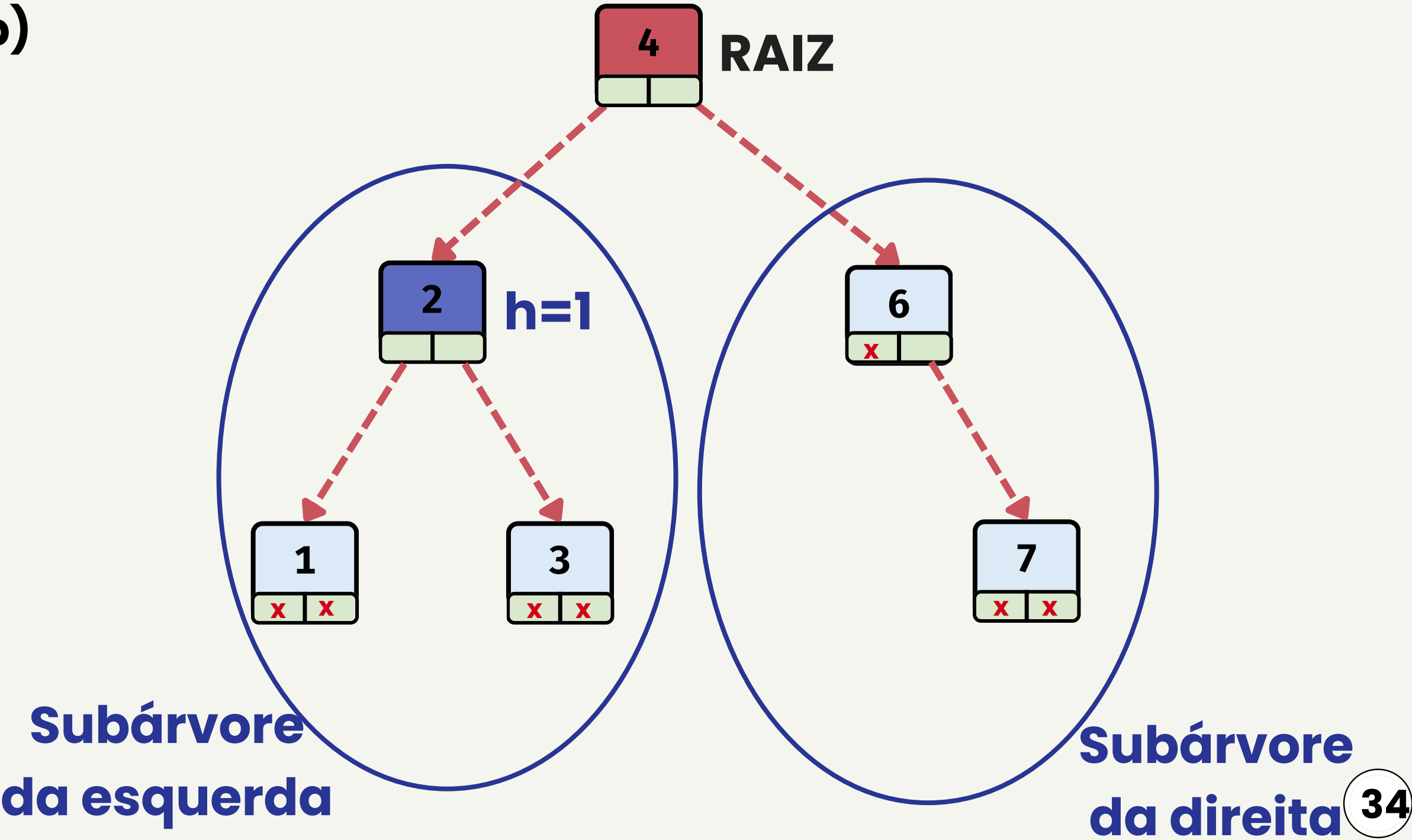
AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada



AVL

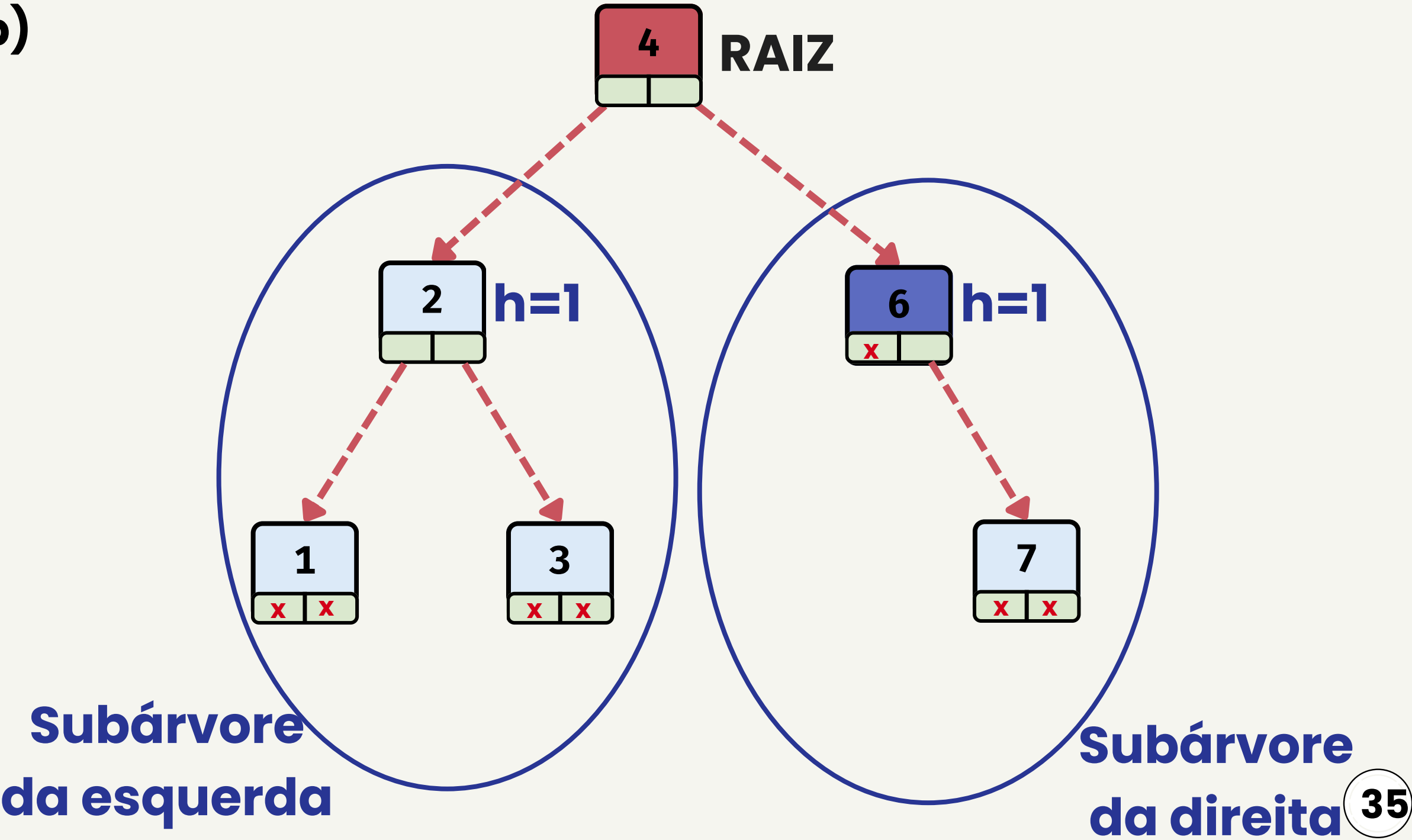
AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada



AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

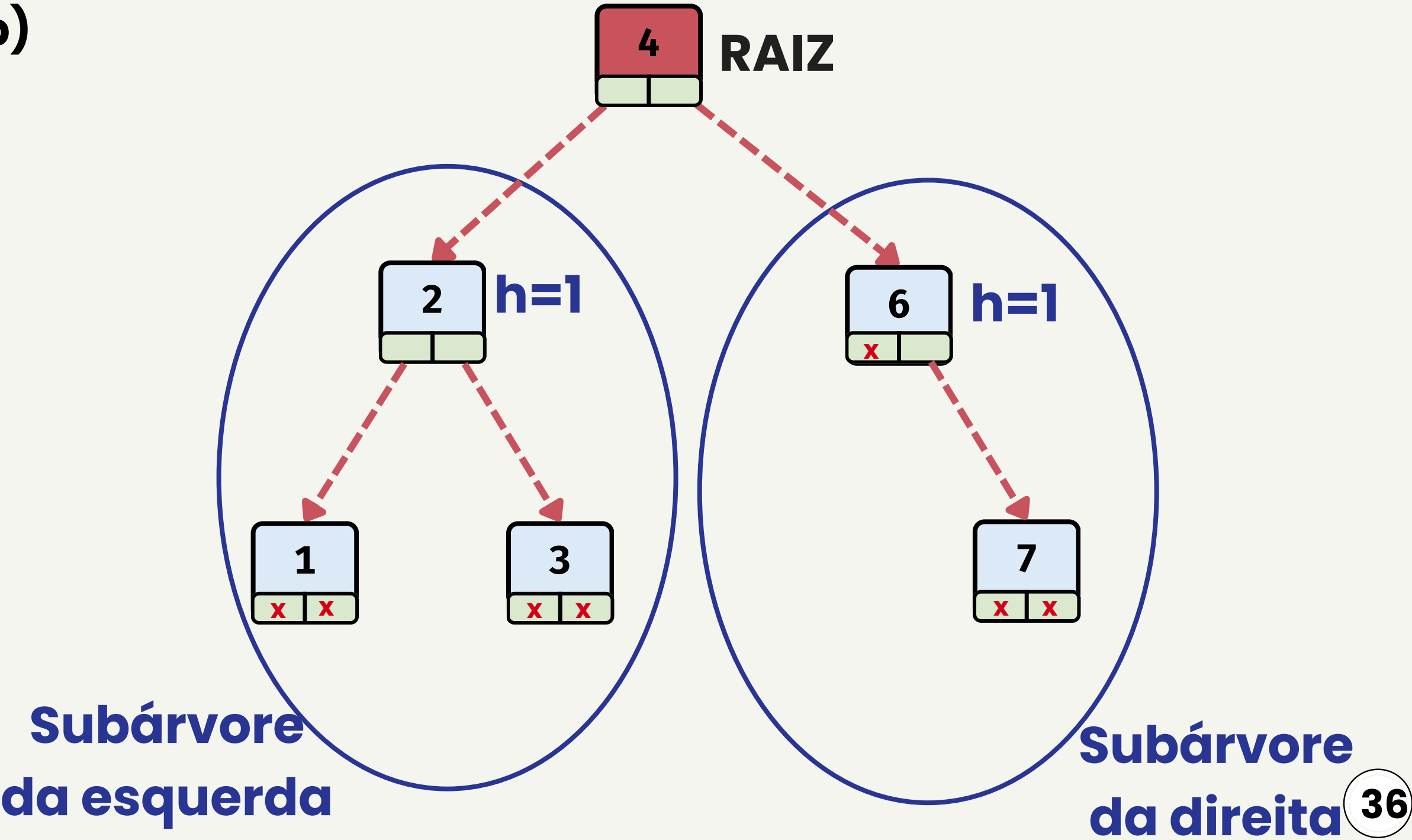
FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada

$fb = 1 - 1 = 0$

Nó 4 balanceado



AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

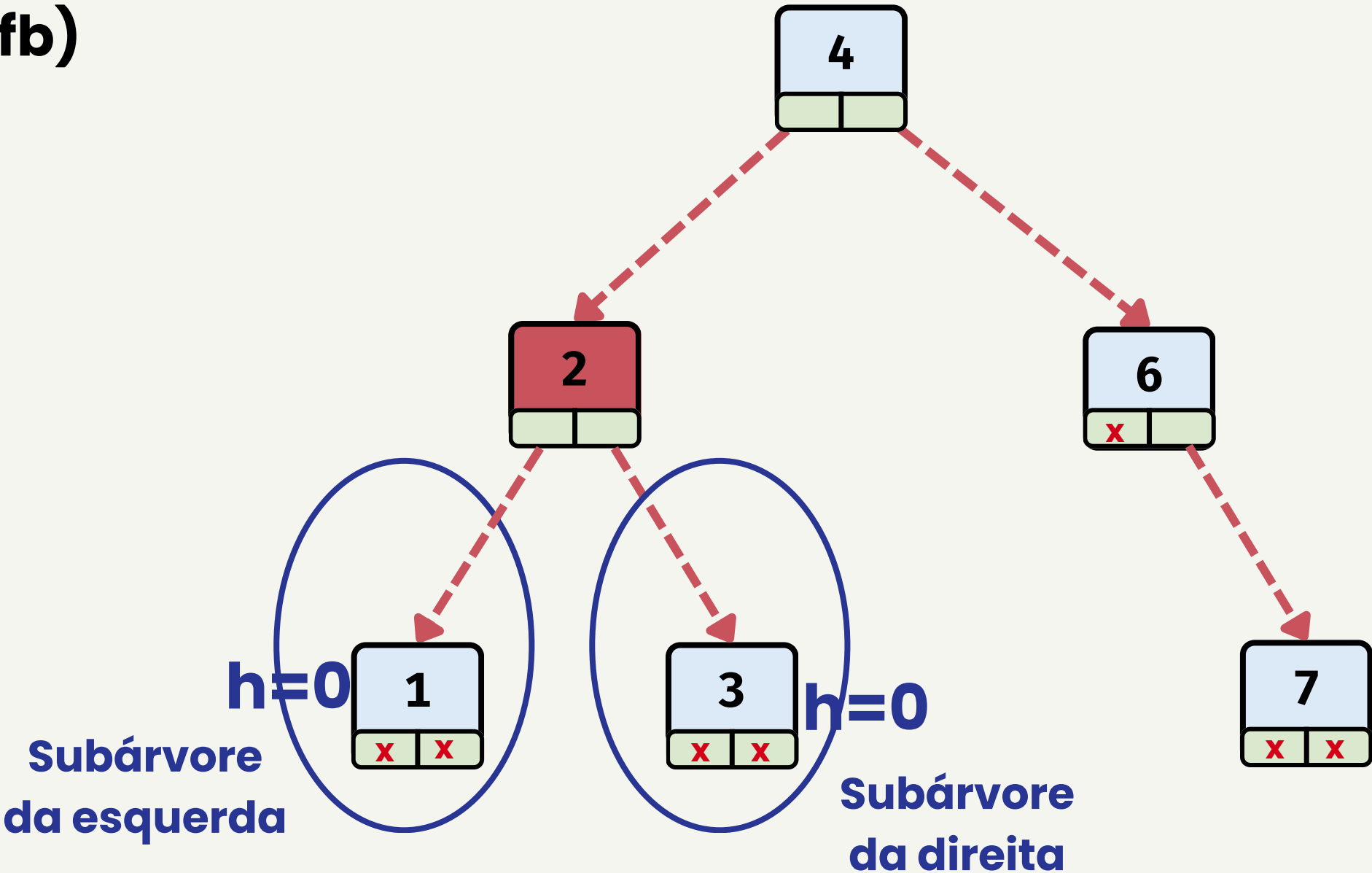
FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada

$fb = 0 - 0 = 0$

Nó 2 balanceado



AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

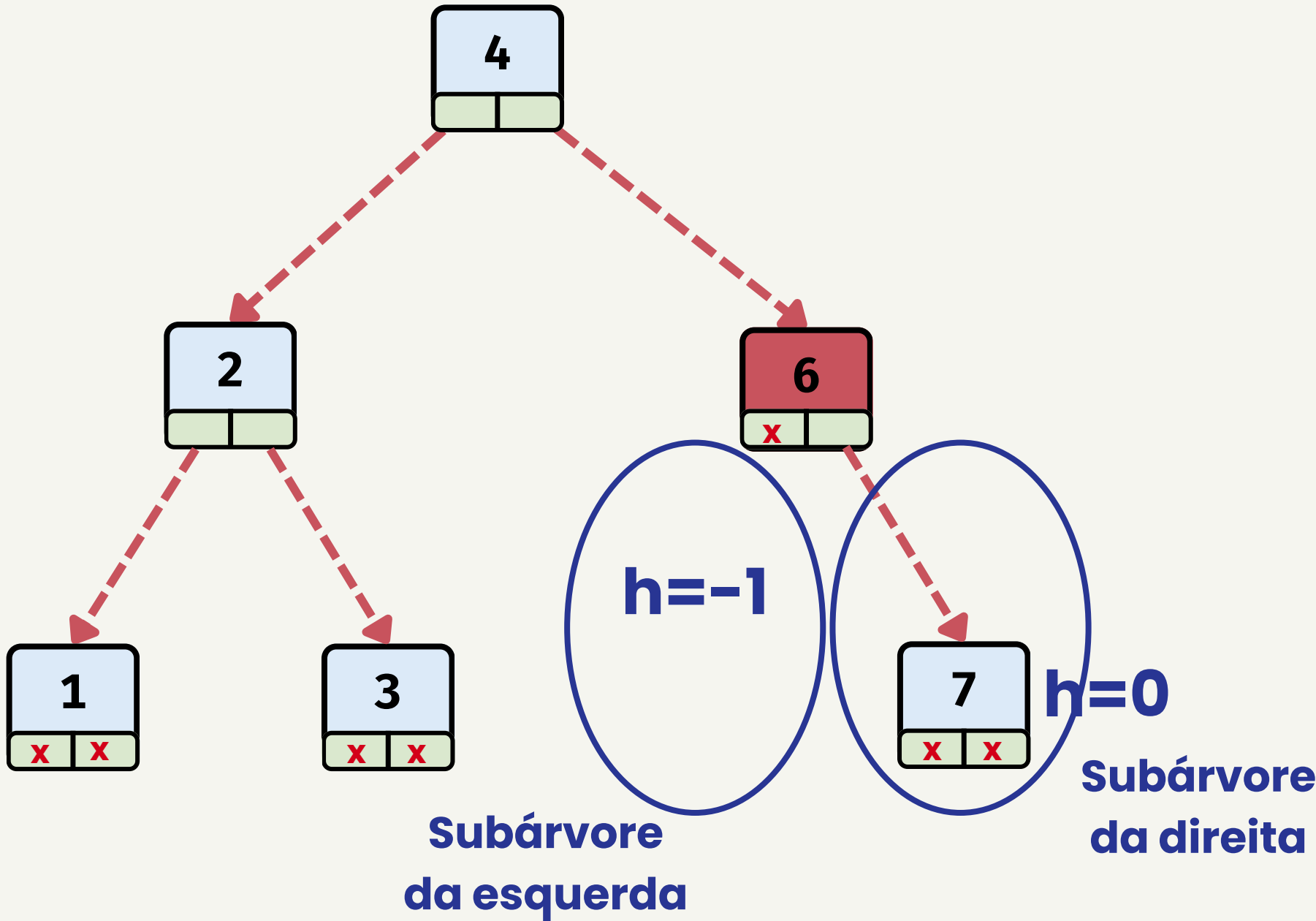
FATOR DE BALANCEAMENTO (fb)

$fb = altEsq - altDir$

-1, 0 ou 1 → balanceada

$fb = -1 - 0 = -1$

Nó 6 balanceado



AVL

AVL – Árvore Binária de Busca Balanceada

Adelson-Velsky e Landis

FATOR DE BALANCEAMENTO (fb)

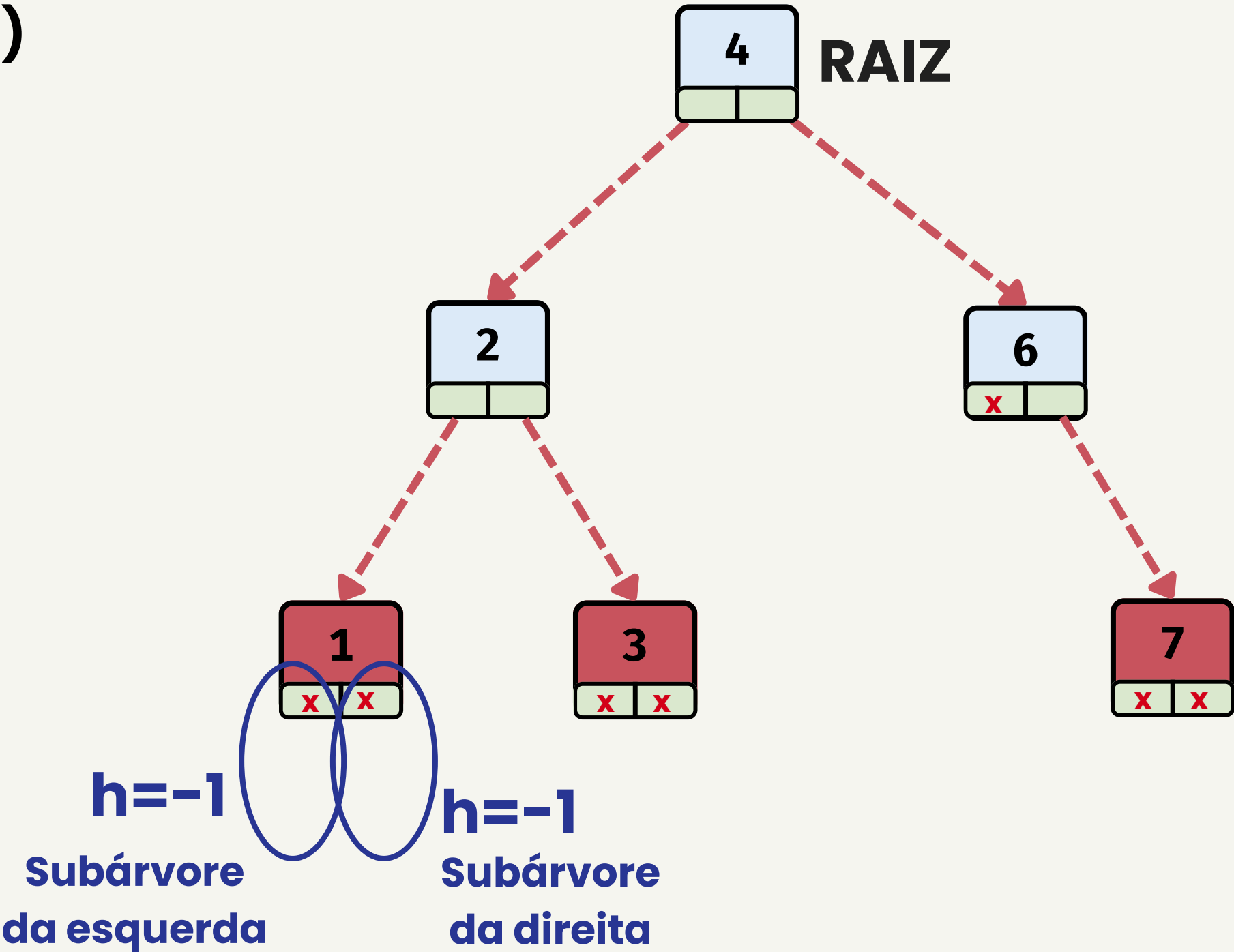
$$fb = altEsq - altDir$$

-1, 0 ou 1 → balanceada

$$fb = -1 - (-1) = 0$$

Nós-folha

Nós 1, 3 e 7 balanceados



AVL

Se $fb > 1$ OU $fb < -1$



Desbalanceada

AVL

Se $fb > 1$ OU $fb < -1$



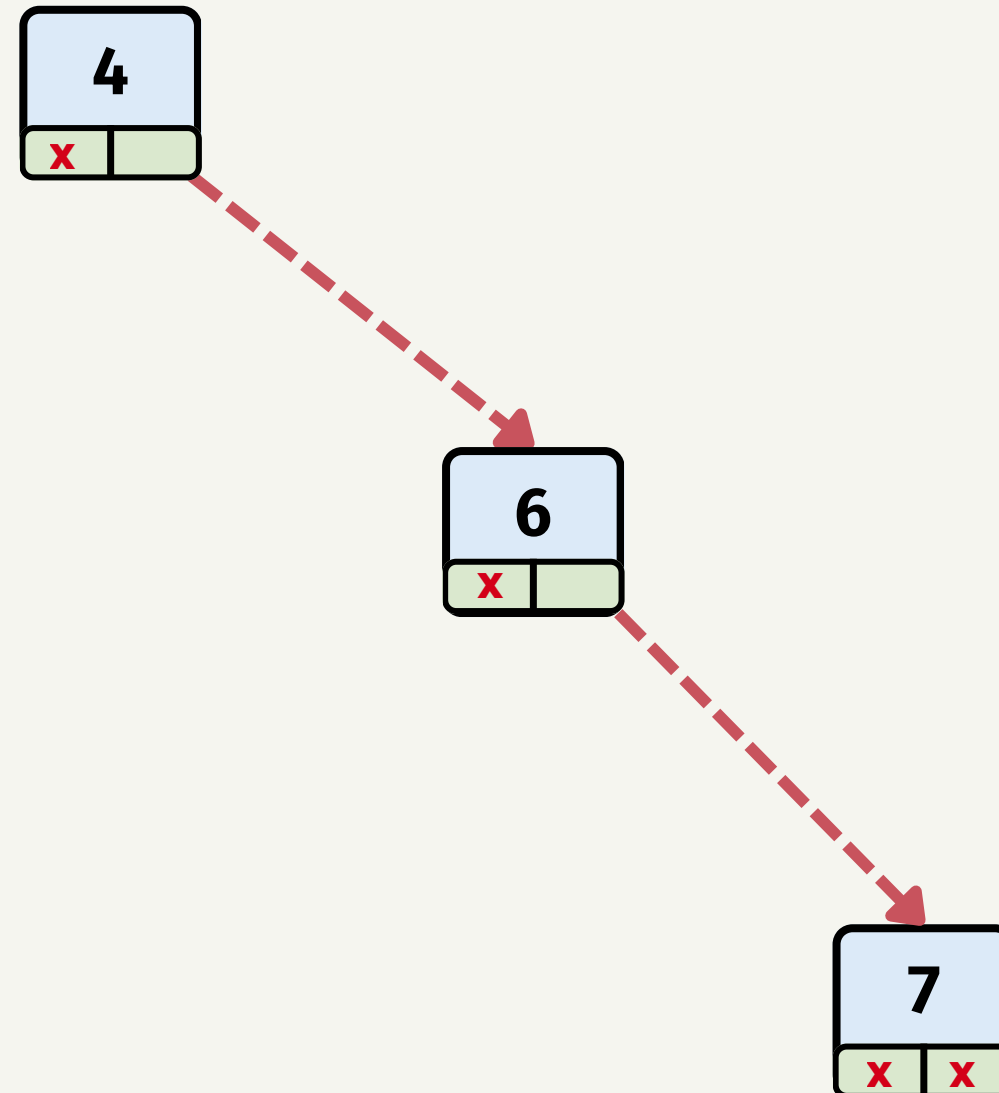
Desbalanceada



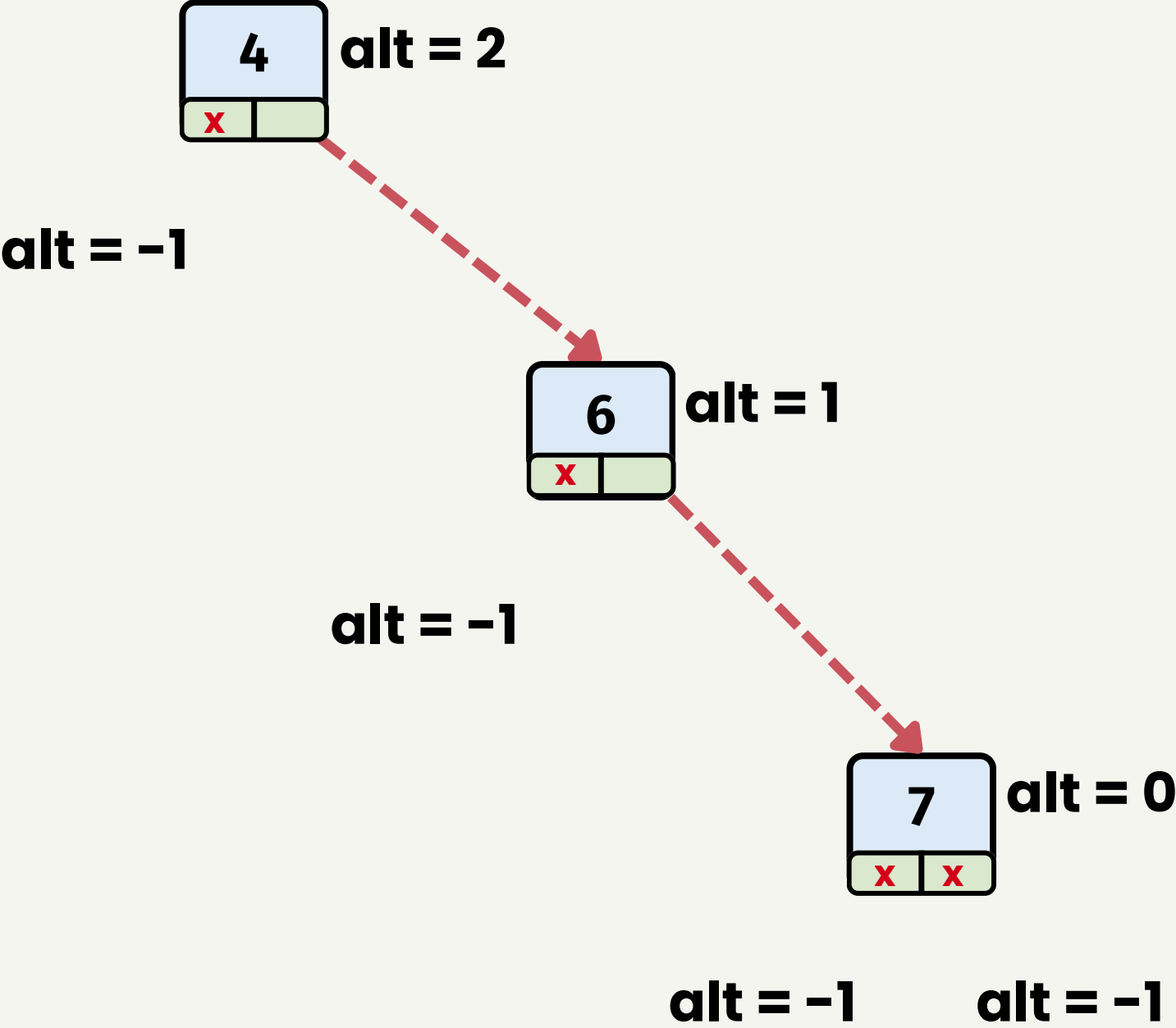
Rotações para rebalancear a árvore



AVL

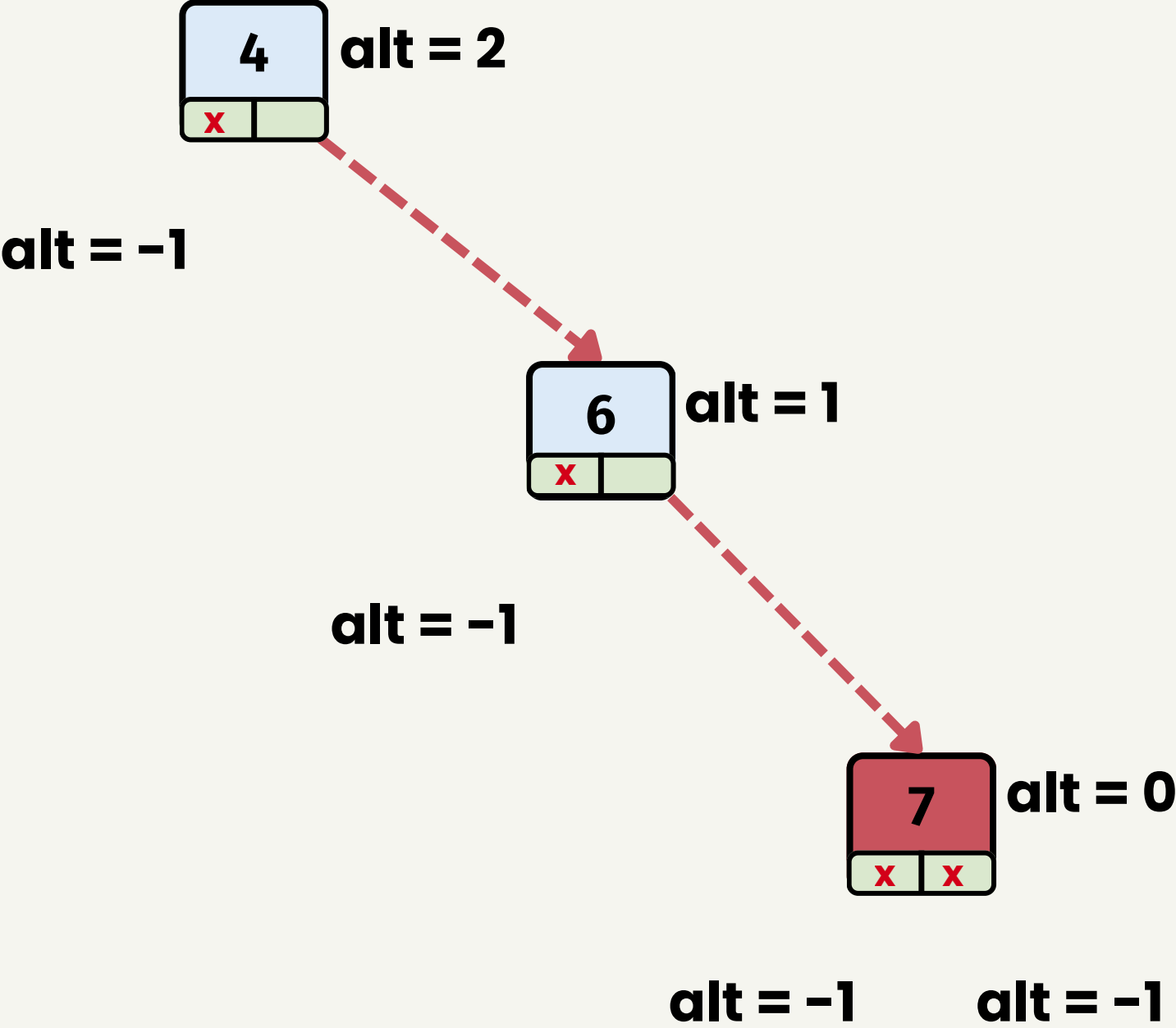


AVL



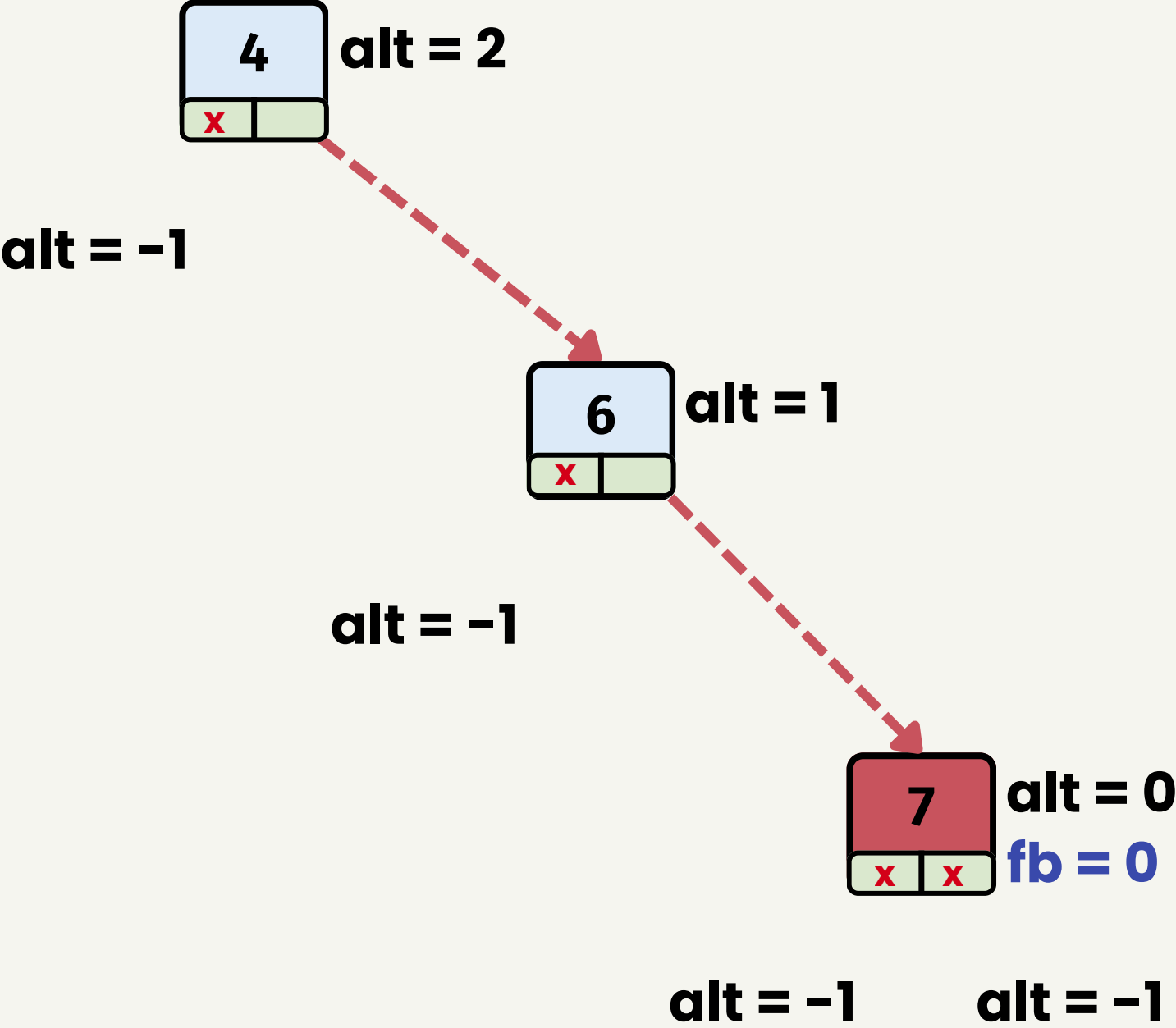
AVL

$fb = altEsq - altDir$



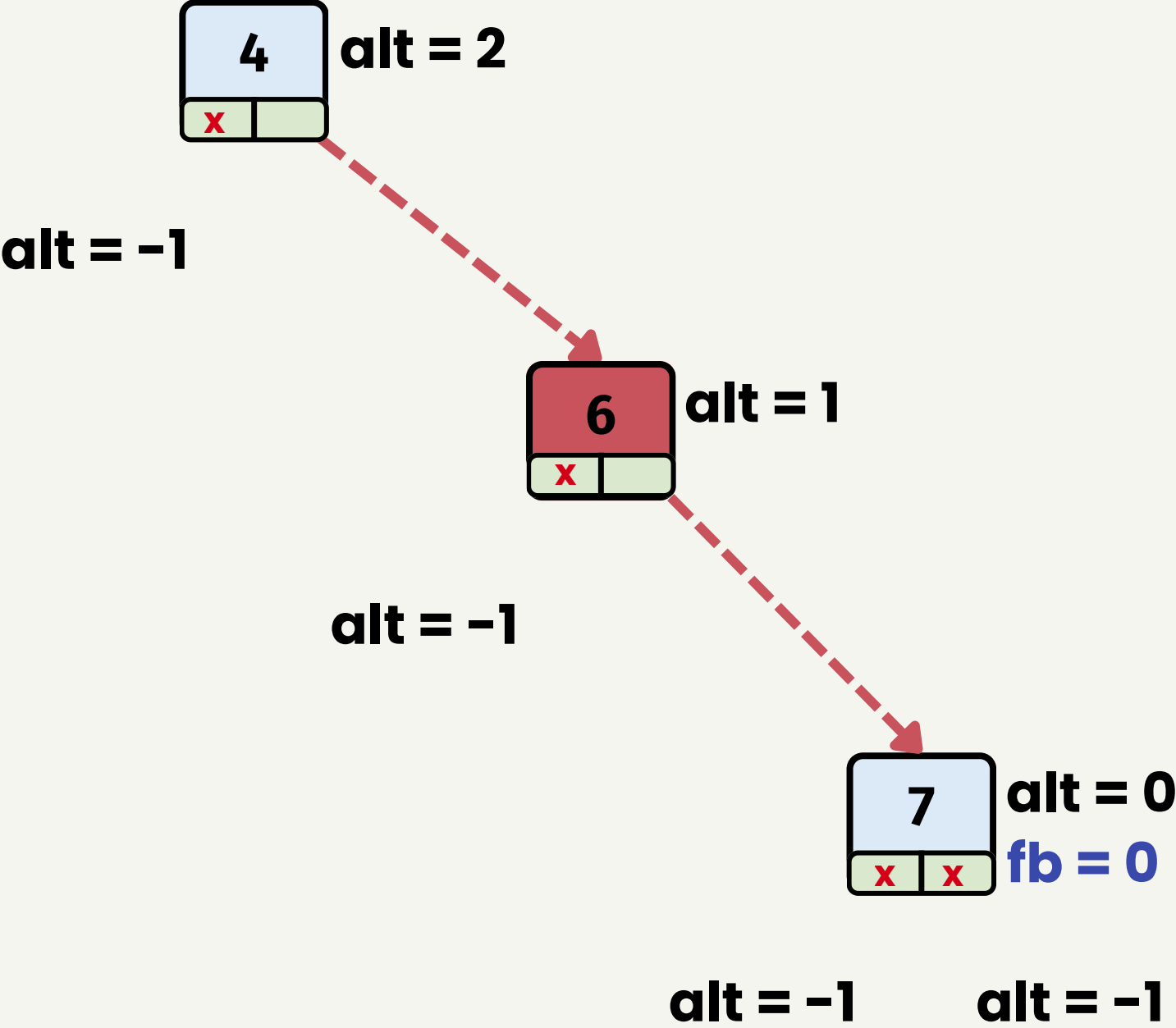
AVL

$fb = altEsq - altDir$



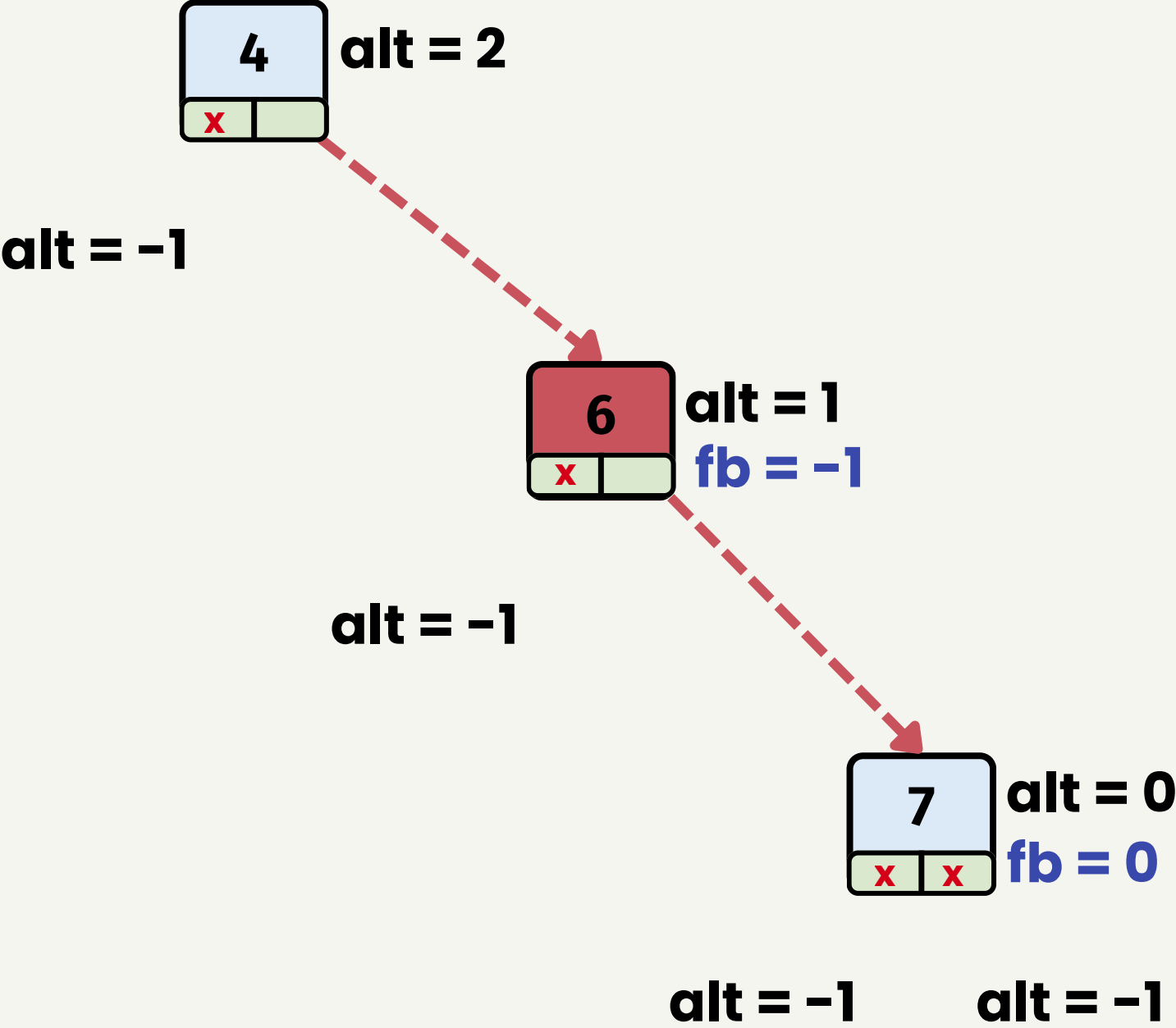
AVL

$fb = altEsq - altDir$



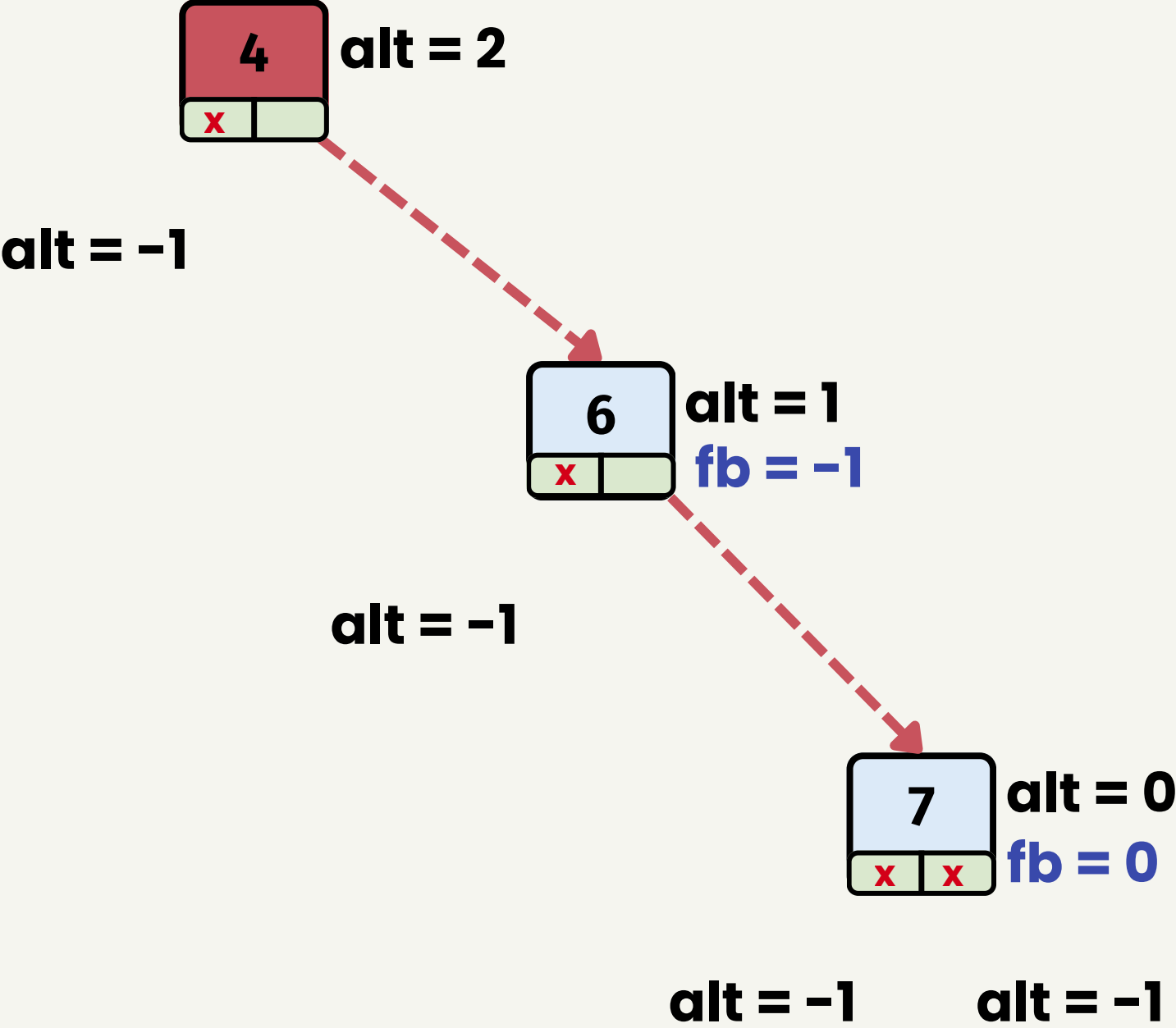
AVL

$fb = altEsq - altDir$



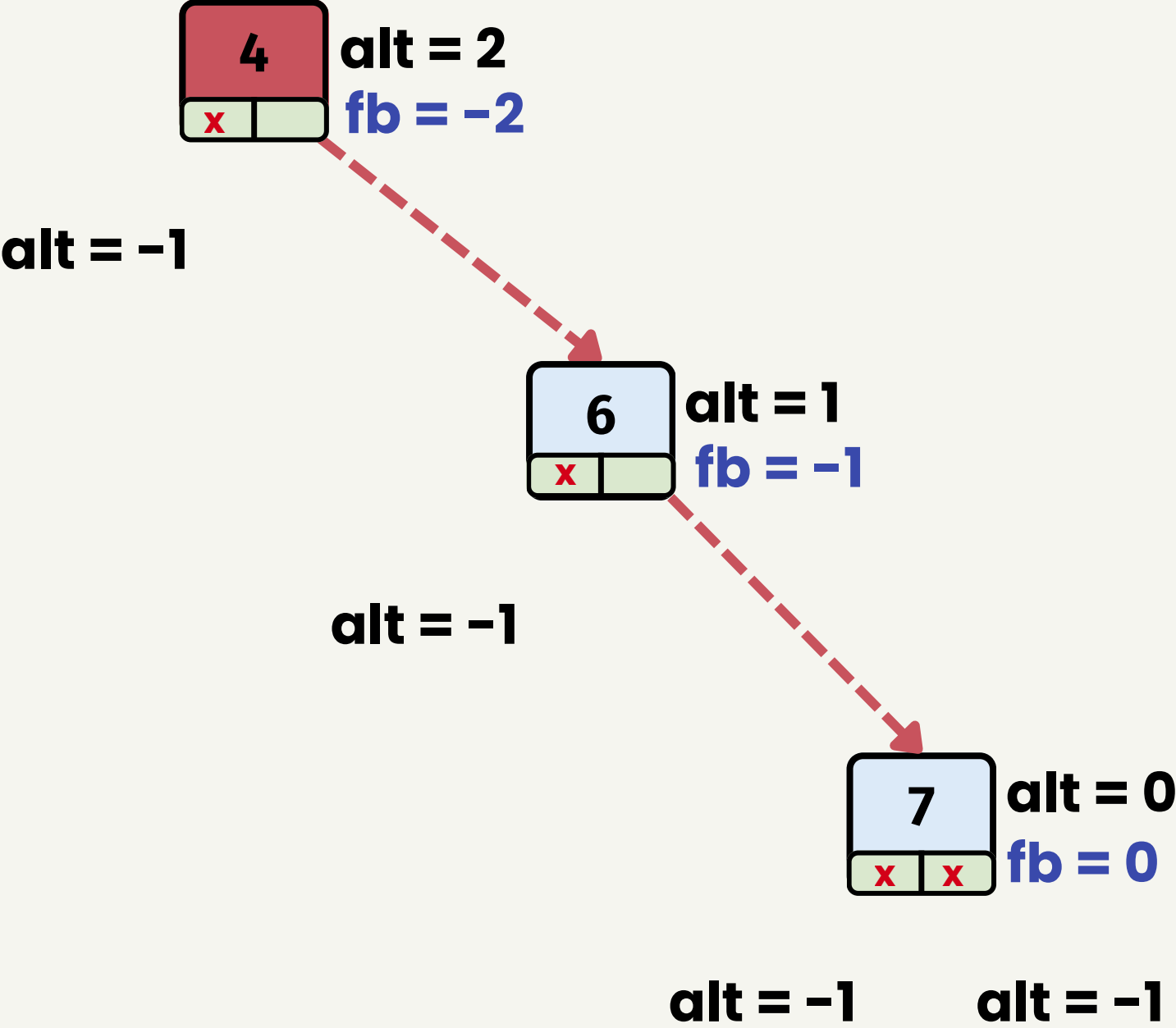
AVL

$fb = altEsq - altDir$

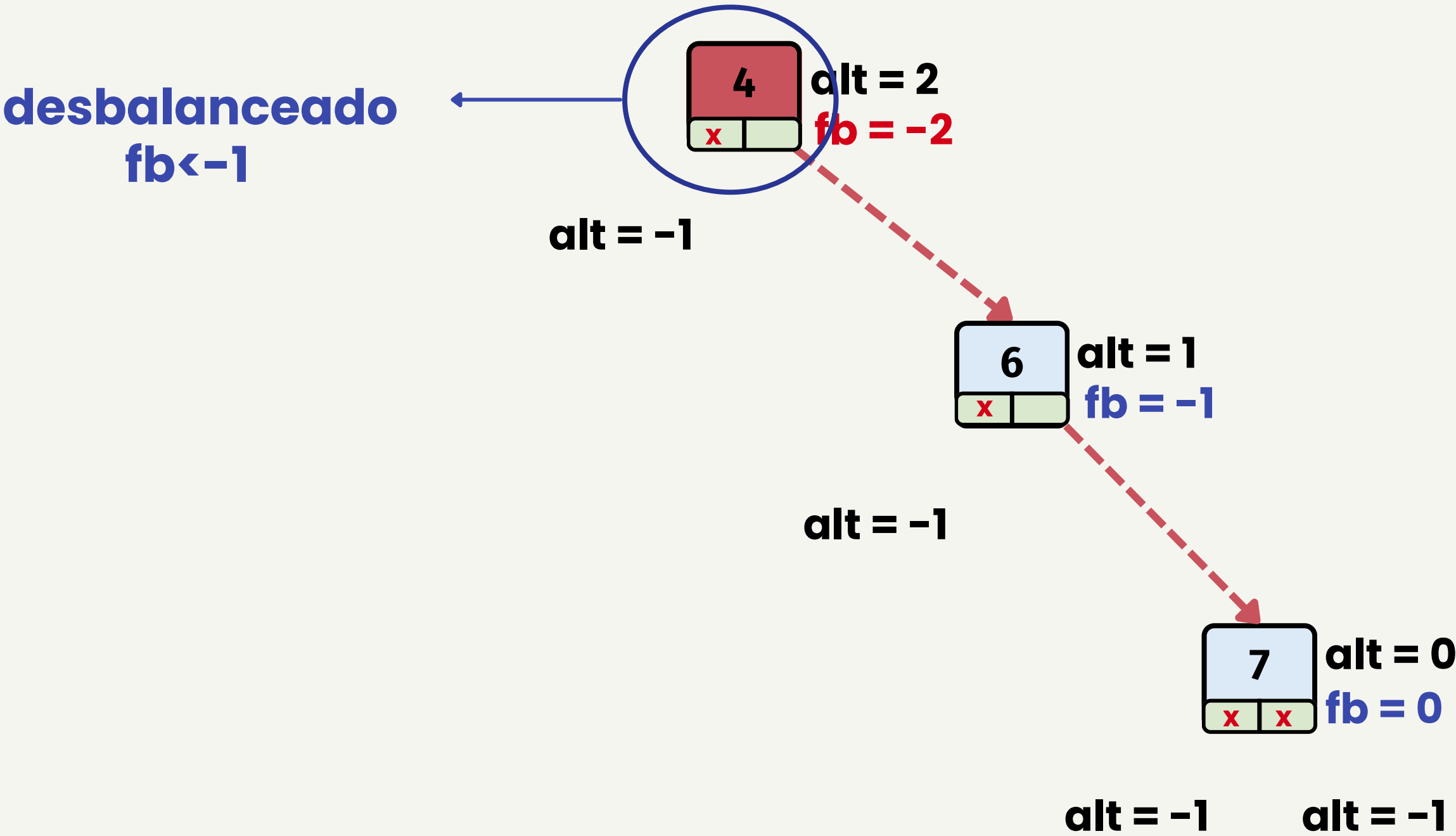


AVL

$fb = altEsq - altDir$

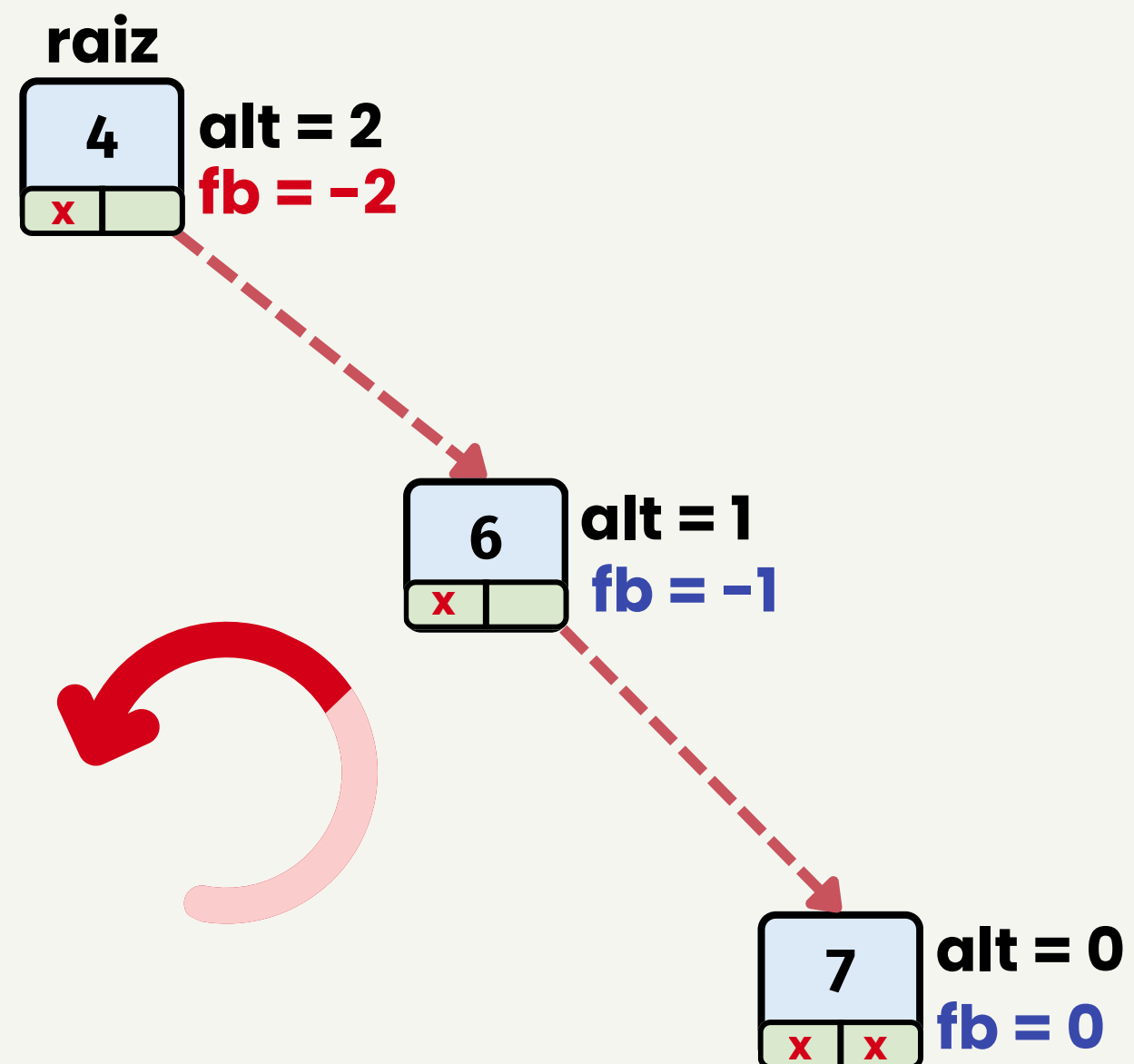


AVL



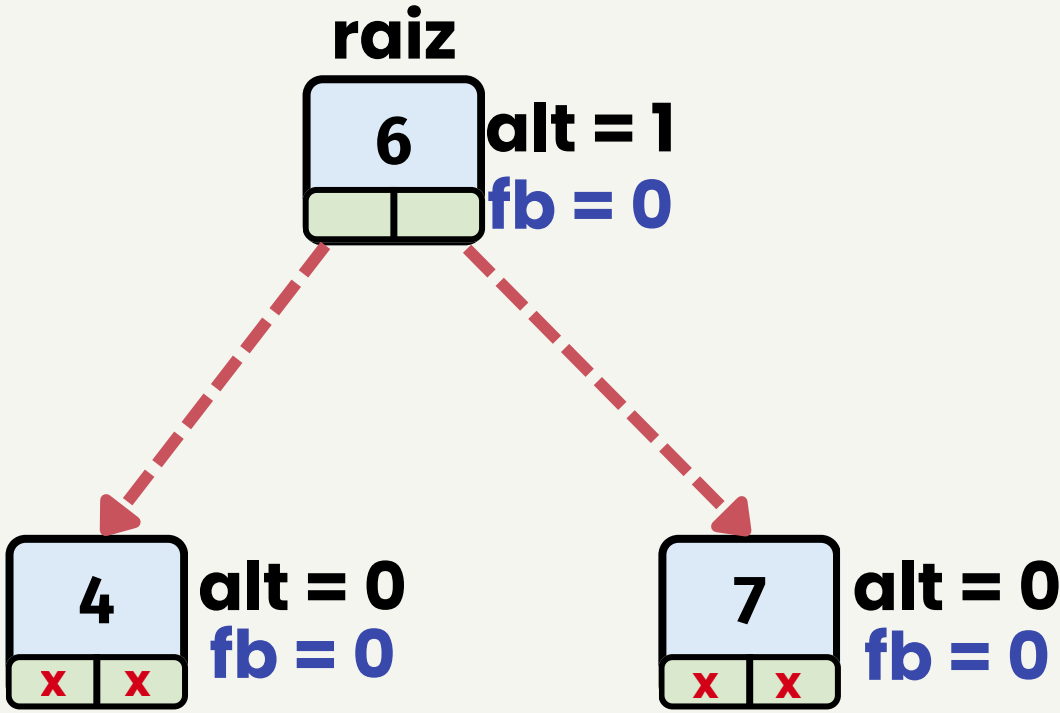
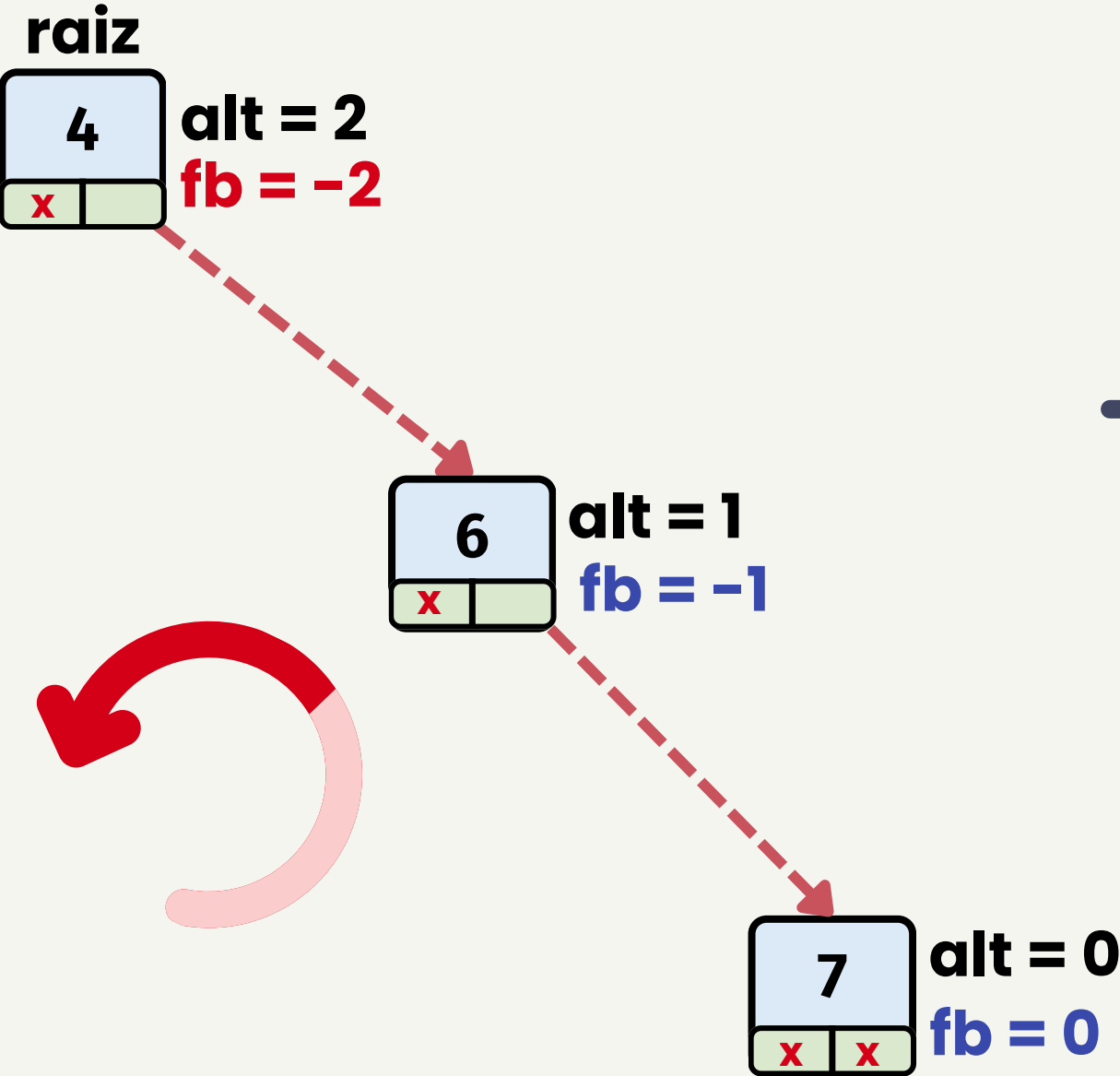
AVL

ROTAÇÃO À ESQUERDA



AVL

ROTAÇÃO À ESQUERDA

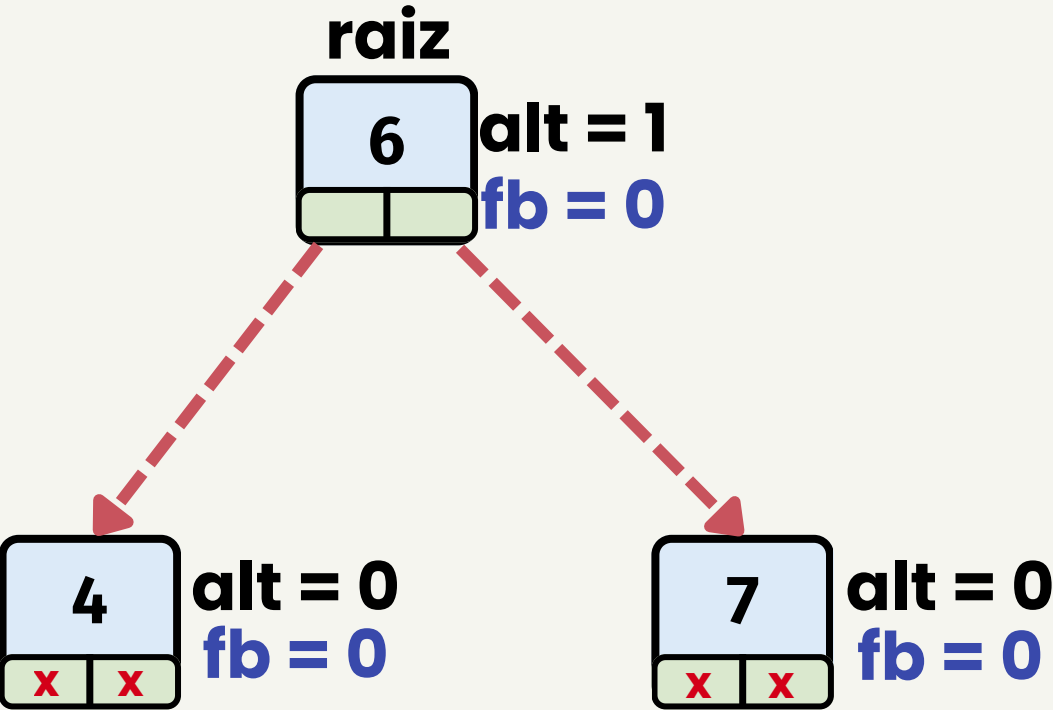
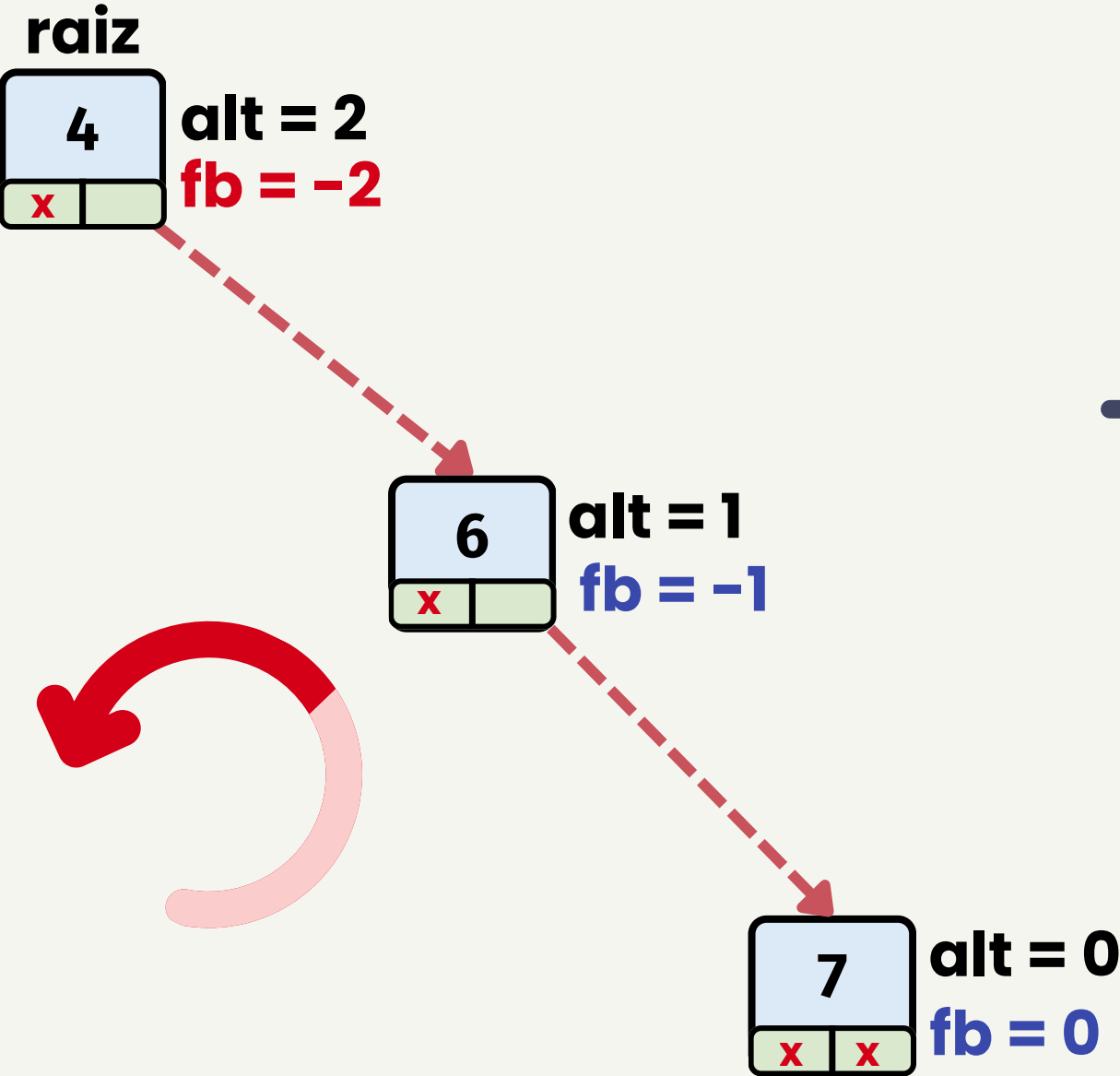


ÁRVORE BALANCEADA

AVL

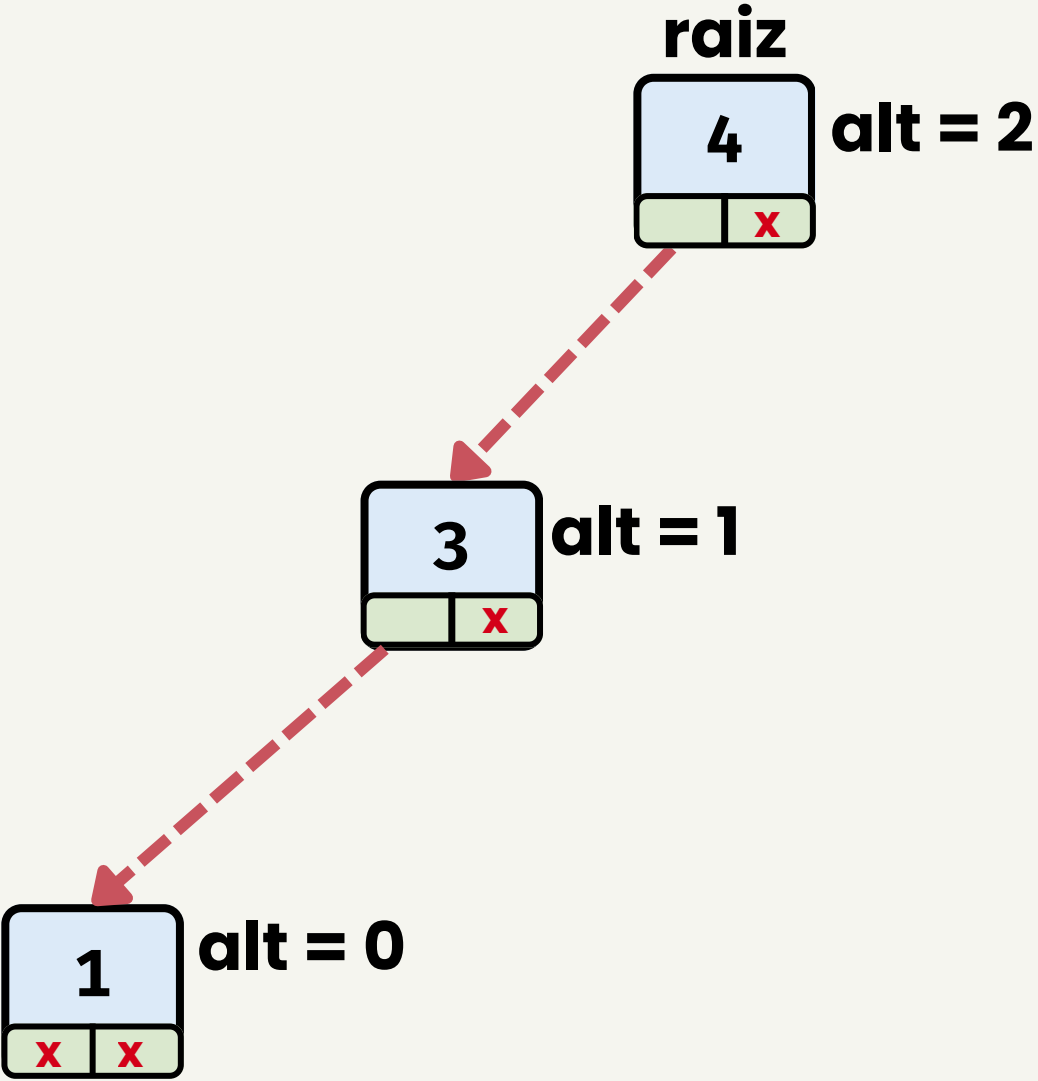
ROTAÇÃO À ESQUERDA

fb < -1

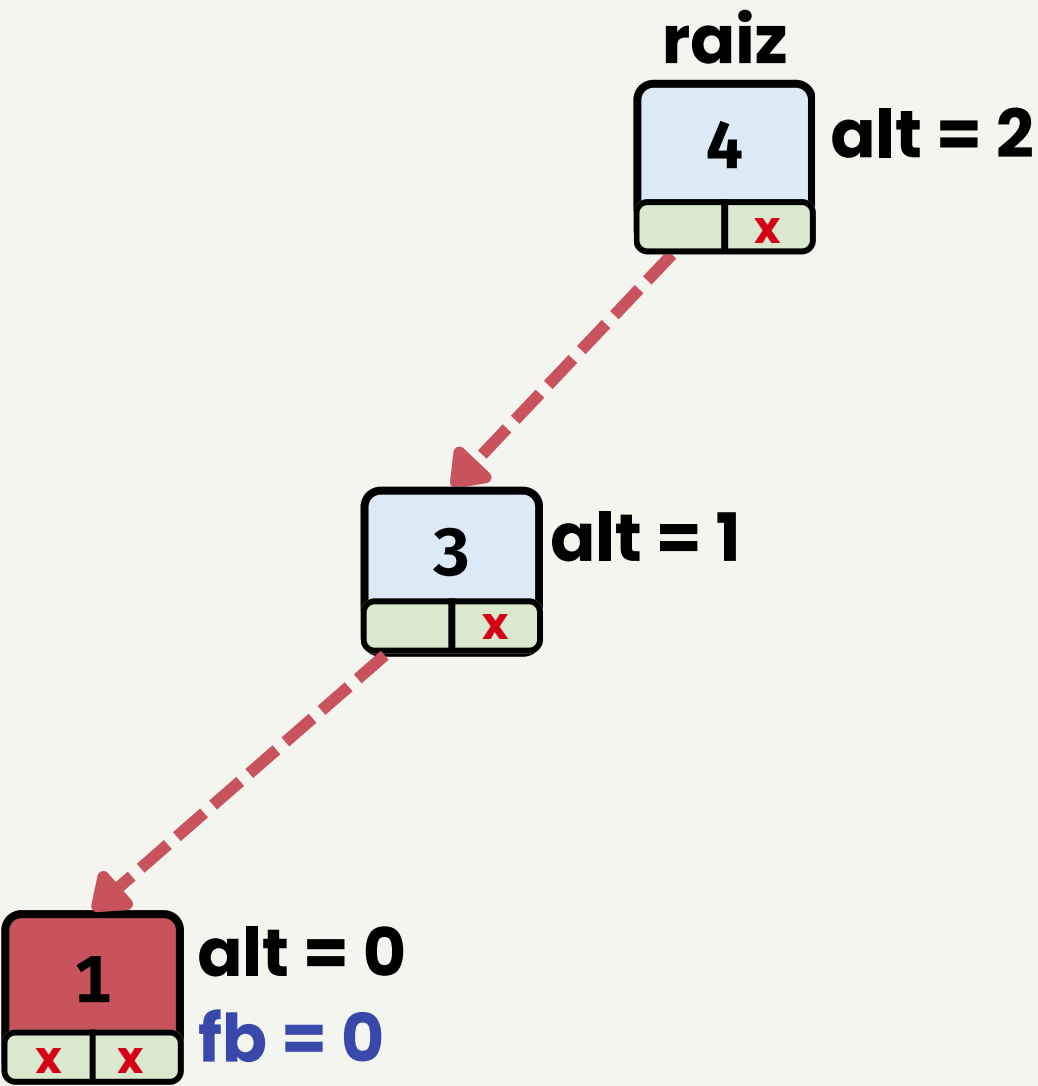


ÁRVORE BALANCEADA

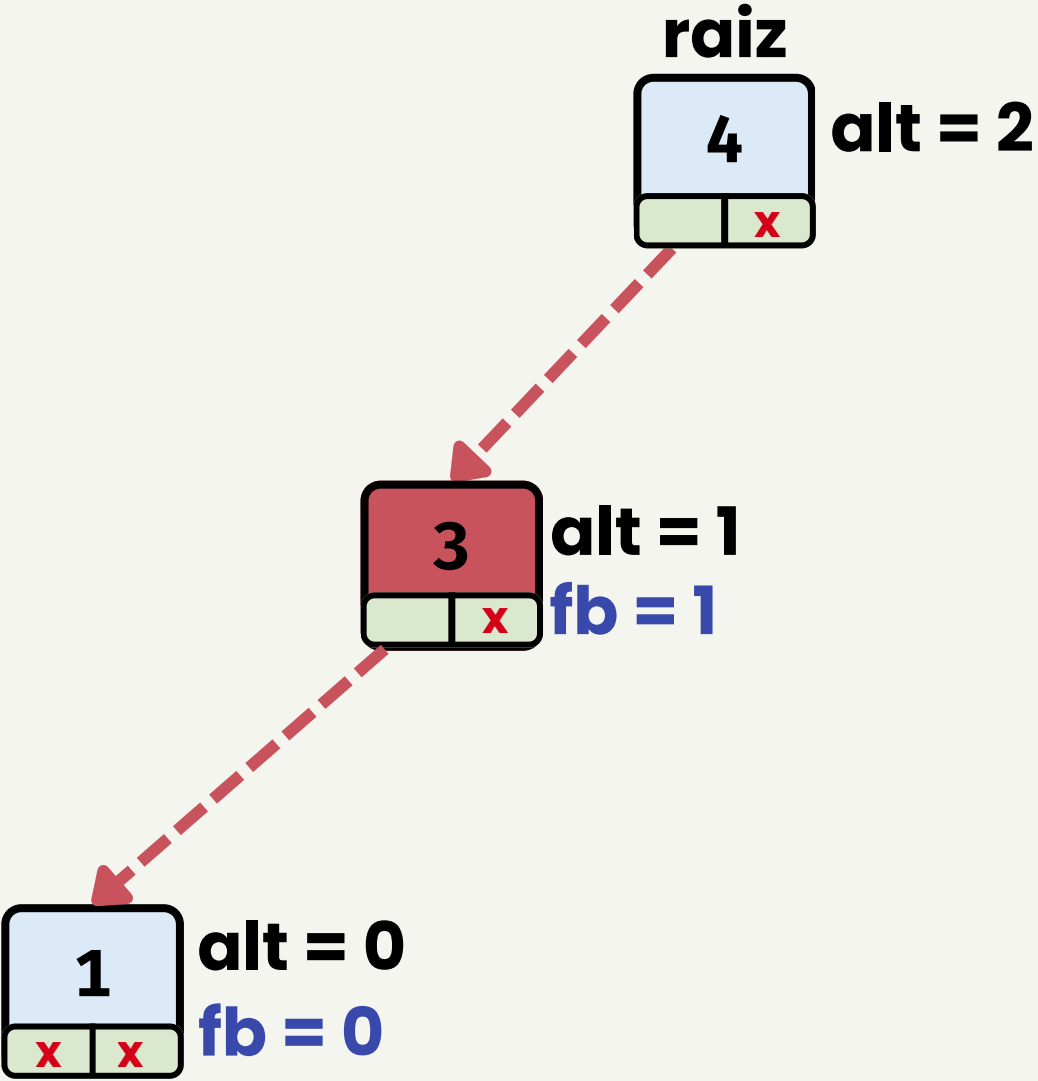
AVL



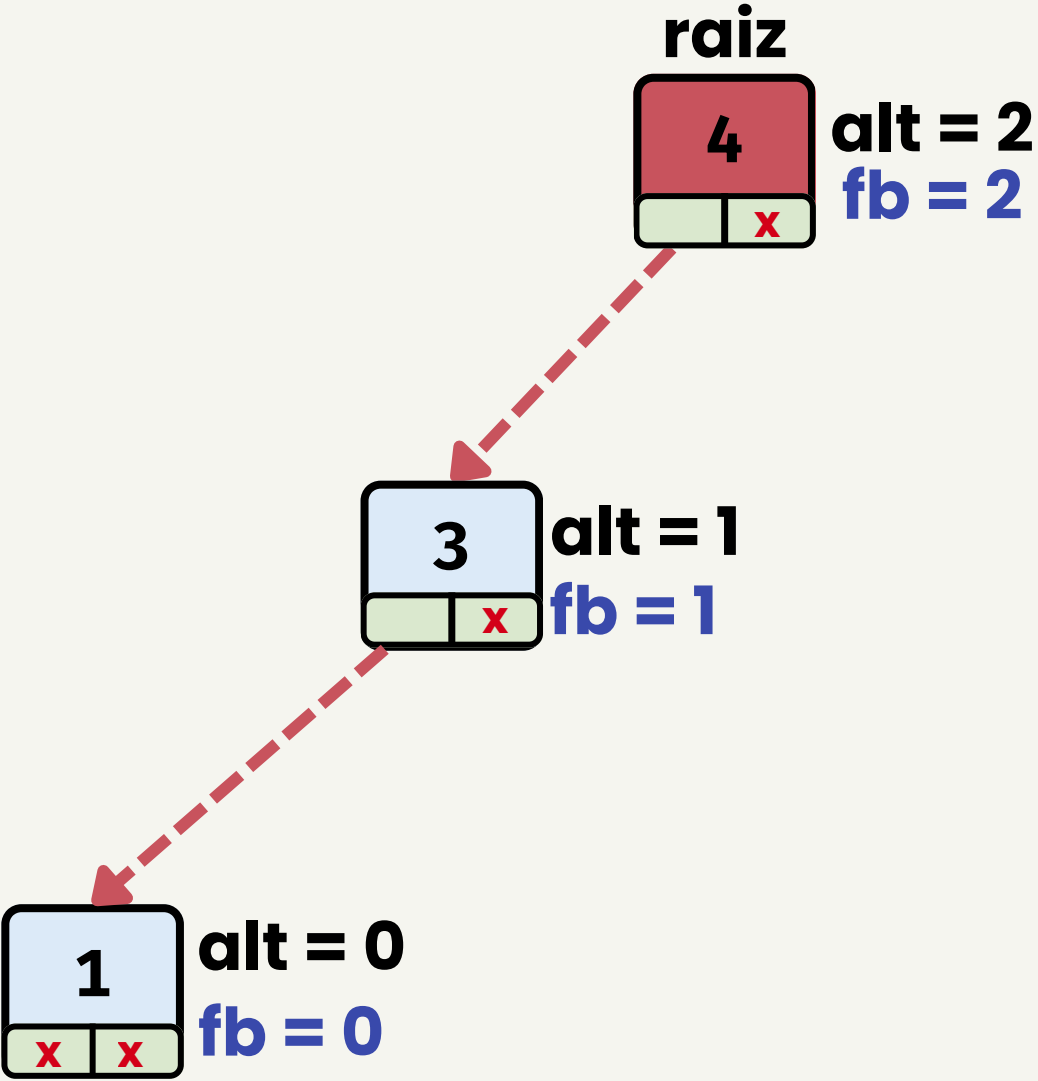
AVL



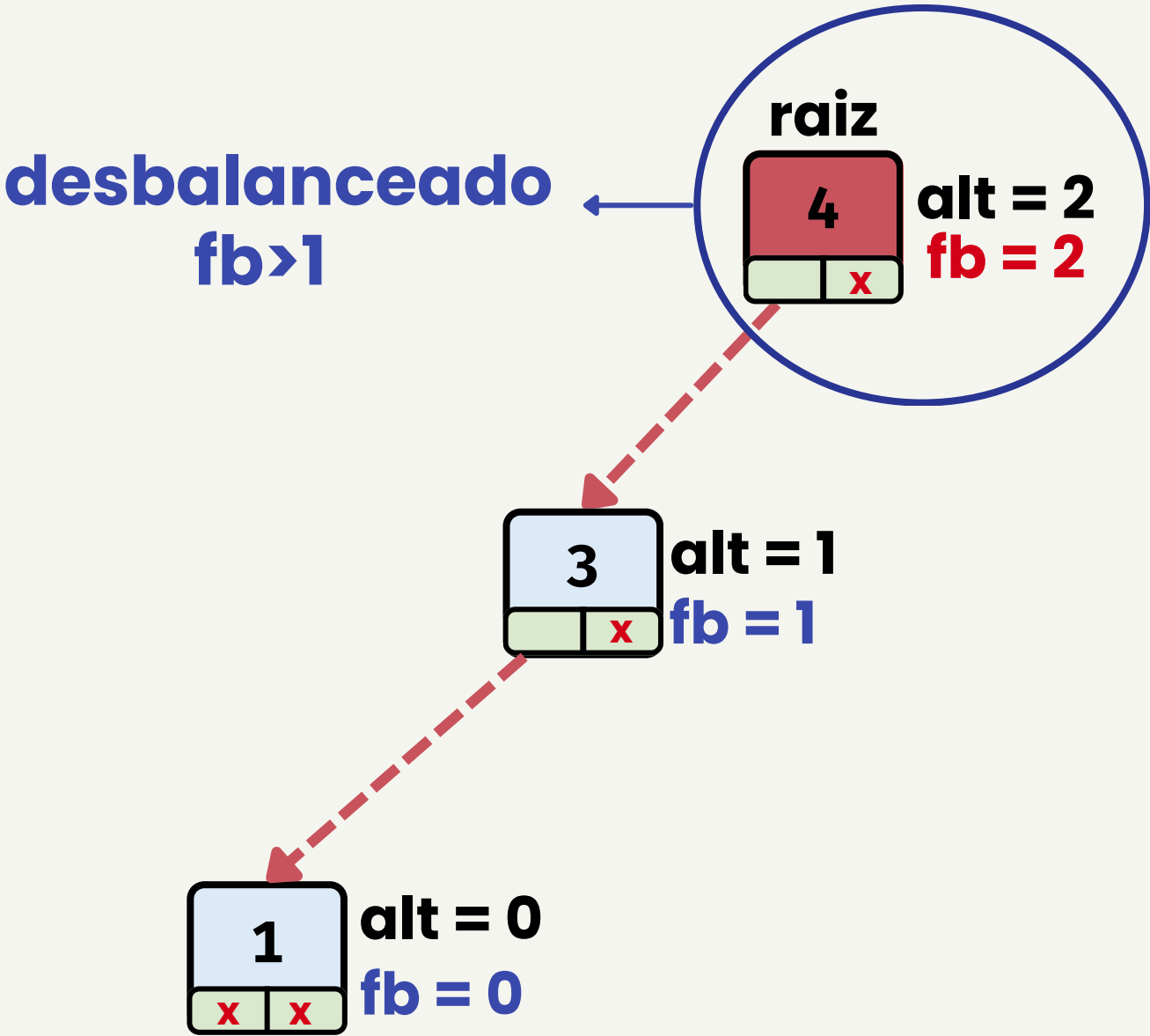
AVL



AVL

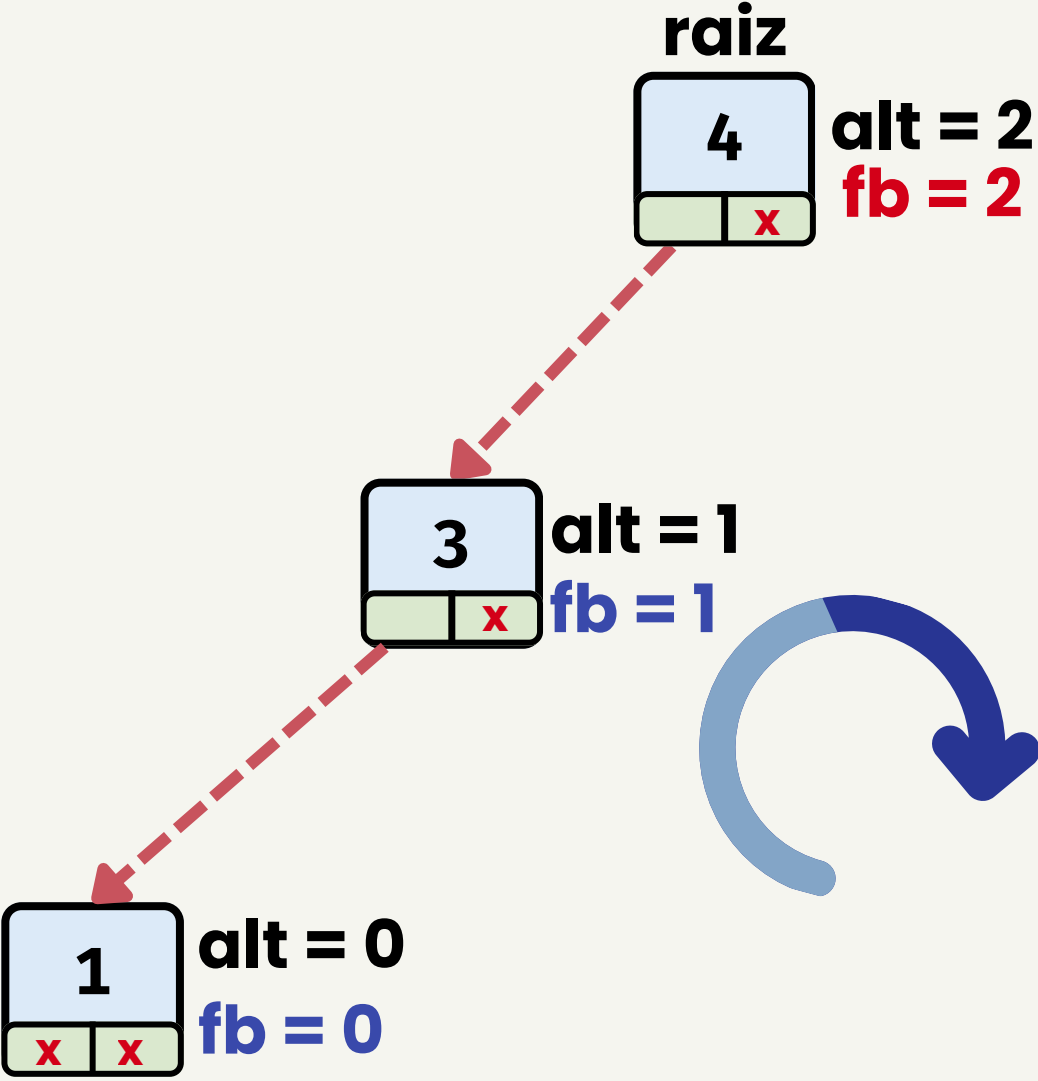


AVL



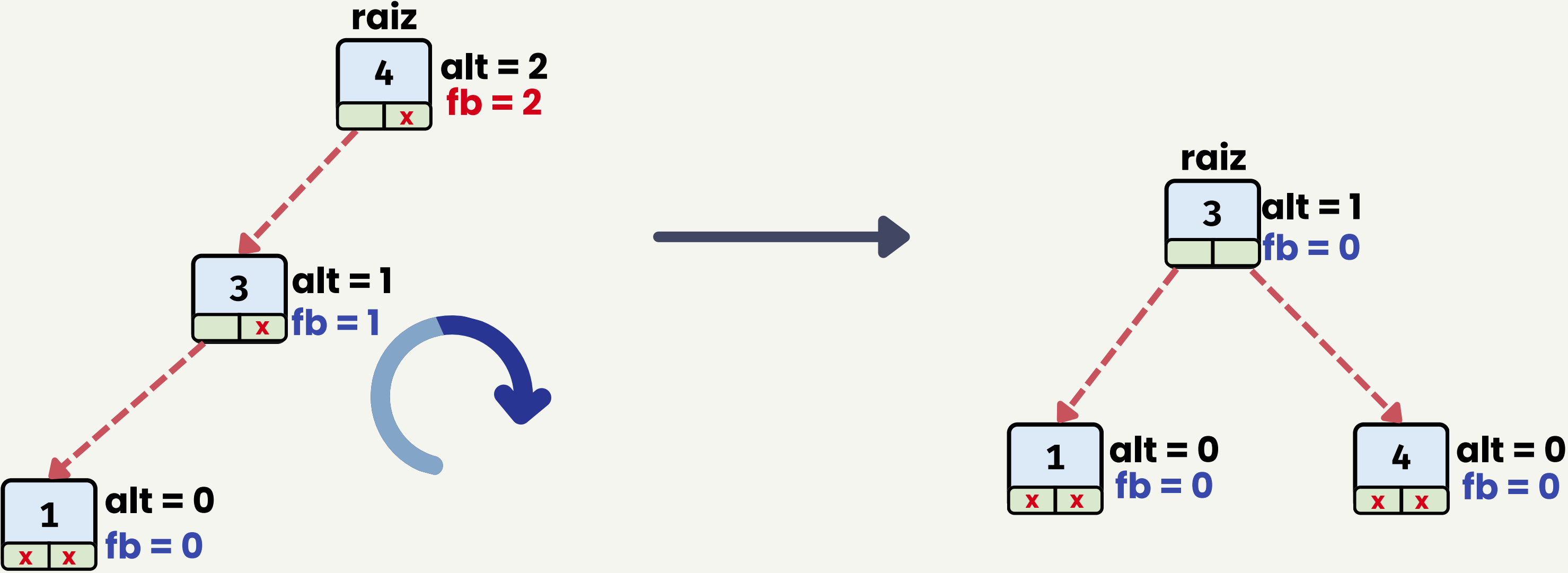
AVL

ROTAÇÃO À DIREITA



AVL

ROTAÇÃO À DIREITA

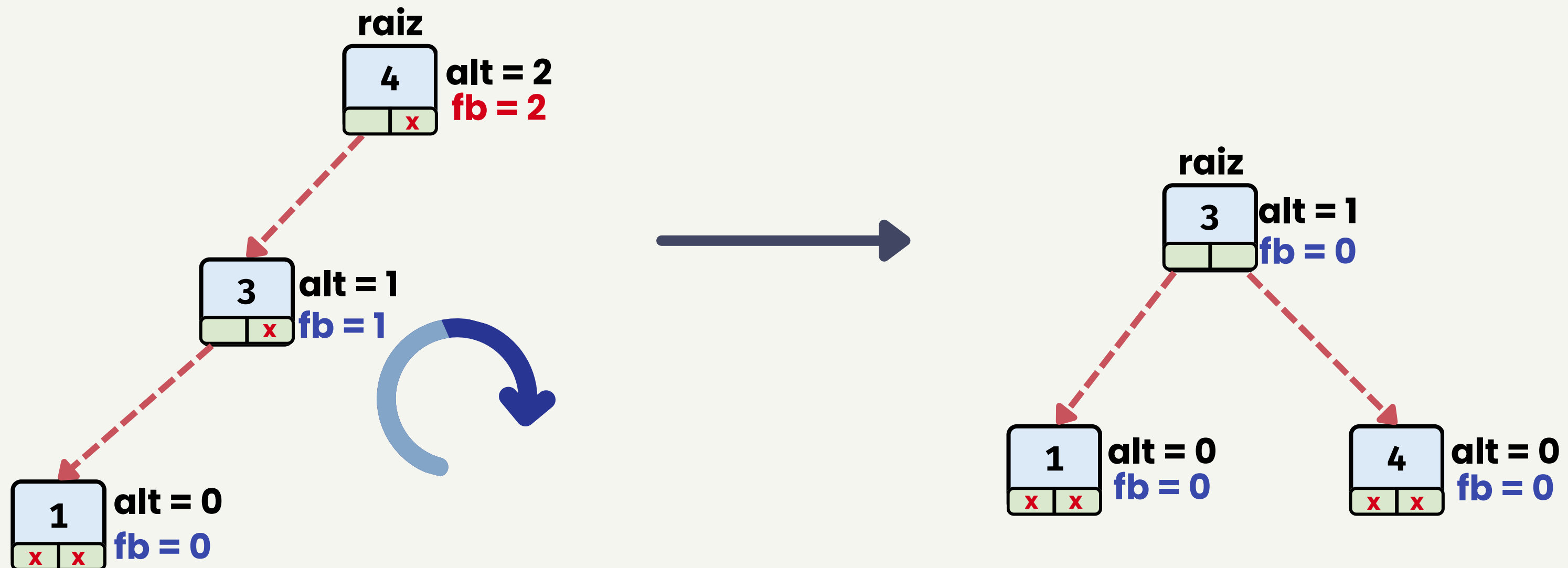


ÁRVORE BALANCEADA

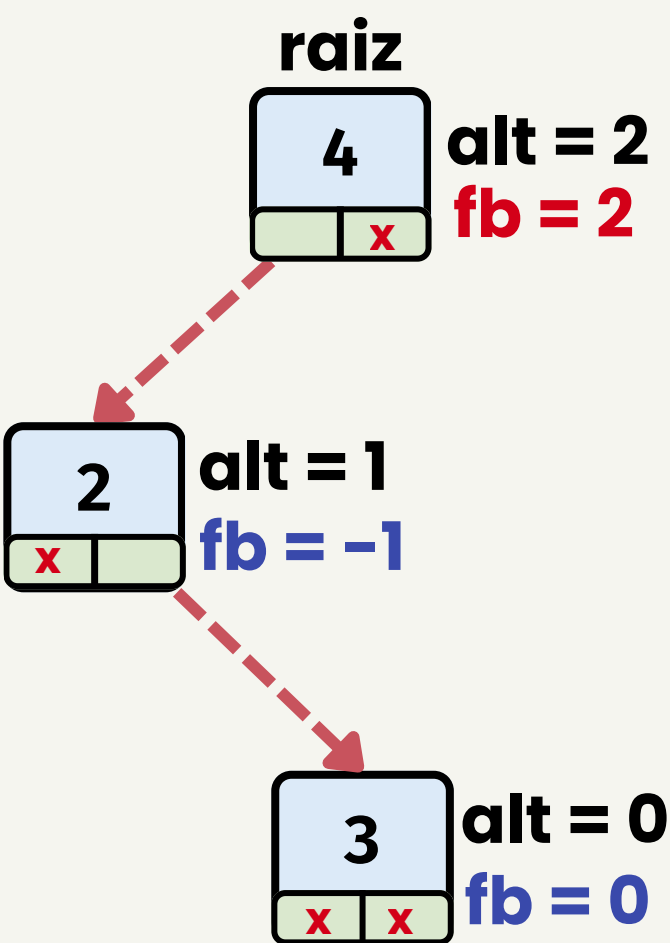
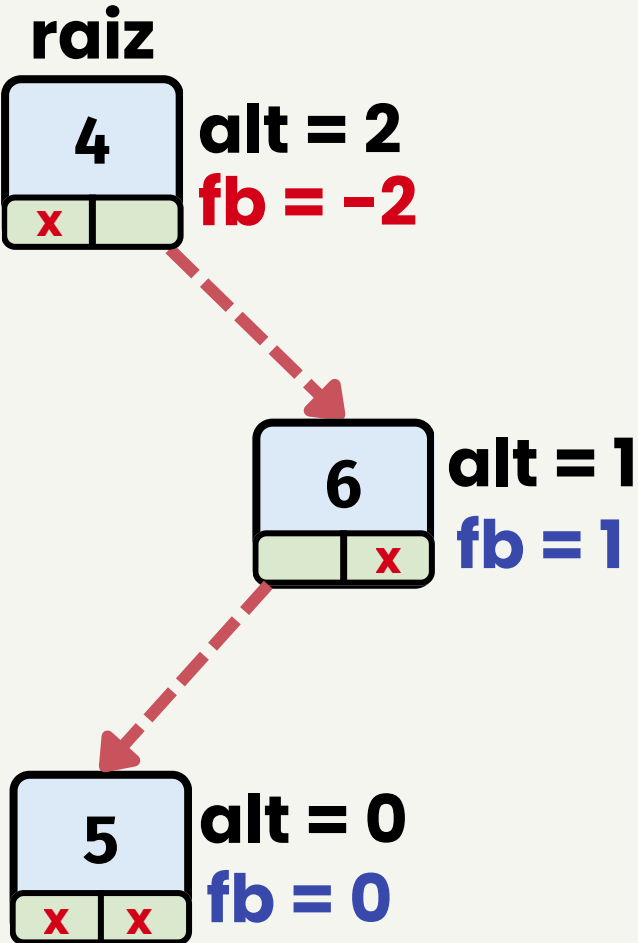
AVL

ROTAÇÃO À DIREITA

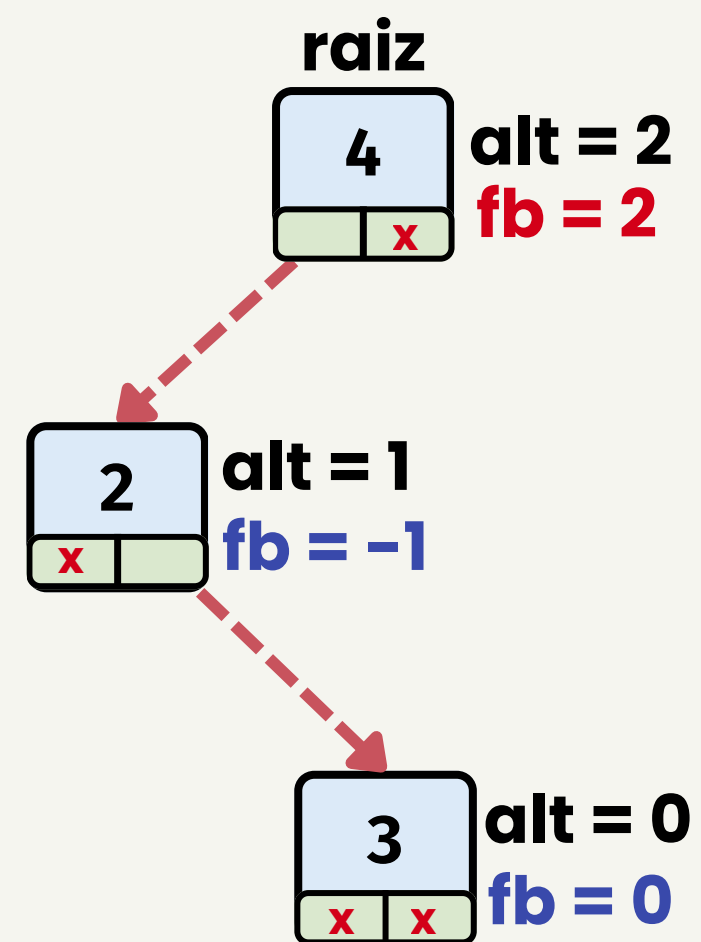
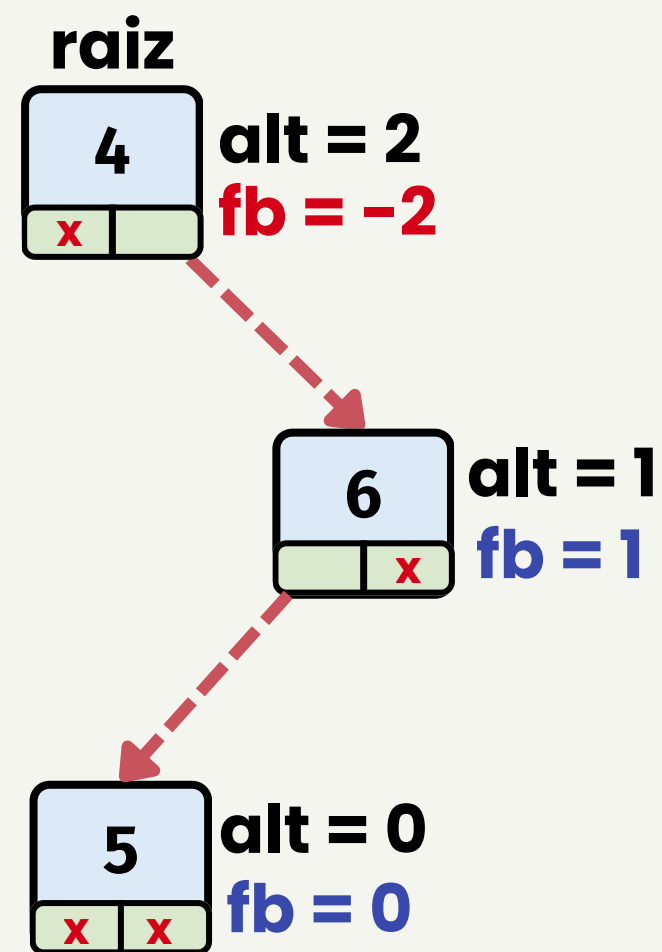
fb > 1



AVL



AVL

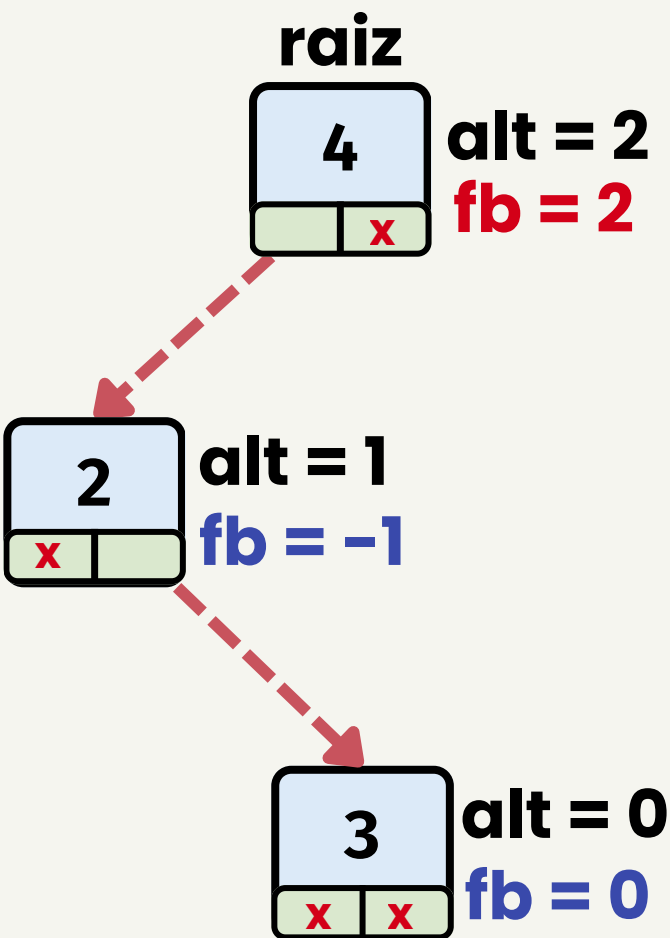
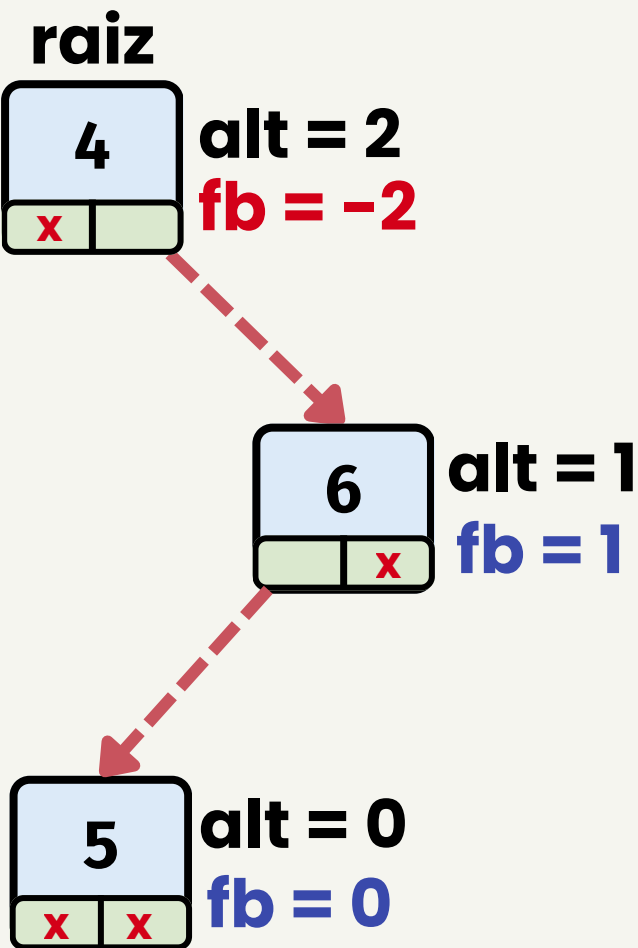


Apenas uma rotação não vai balancear a árvore

AVL

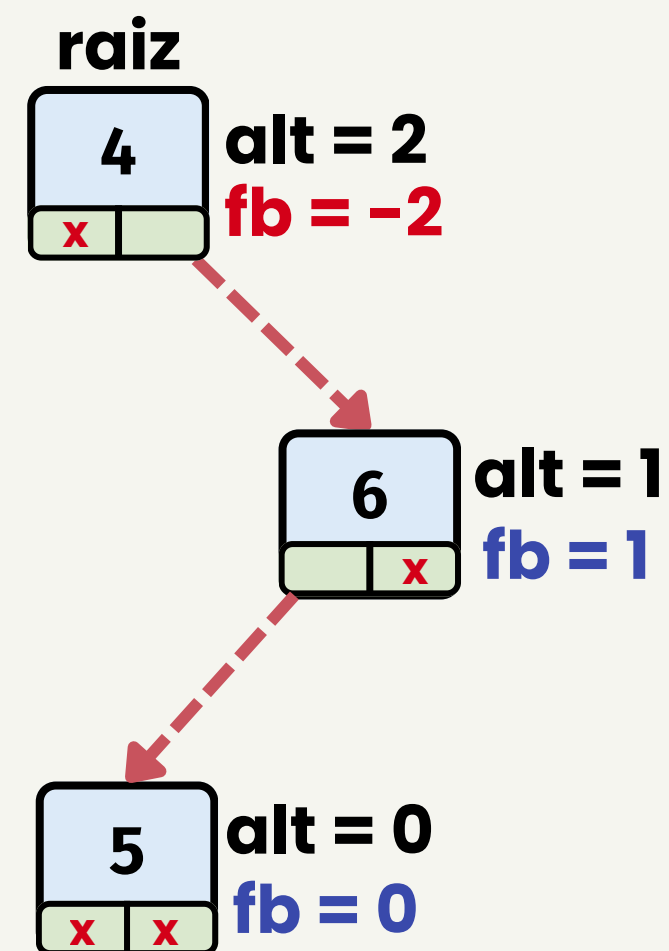
ROTAÇÃO DUPLA

Transformar em um dos casos base e depois aplicá-los



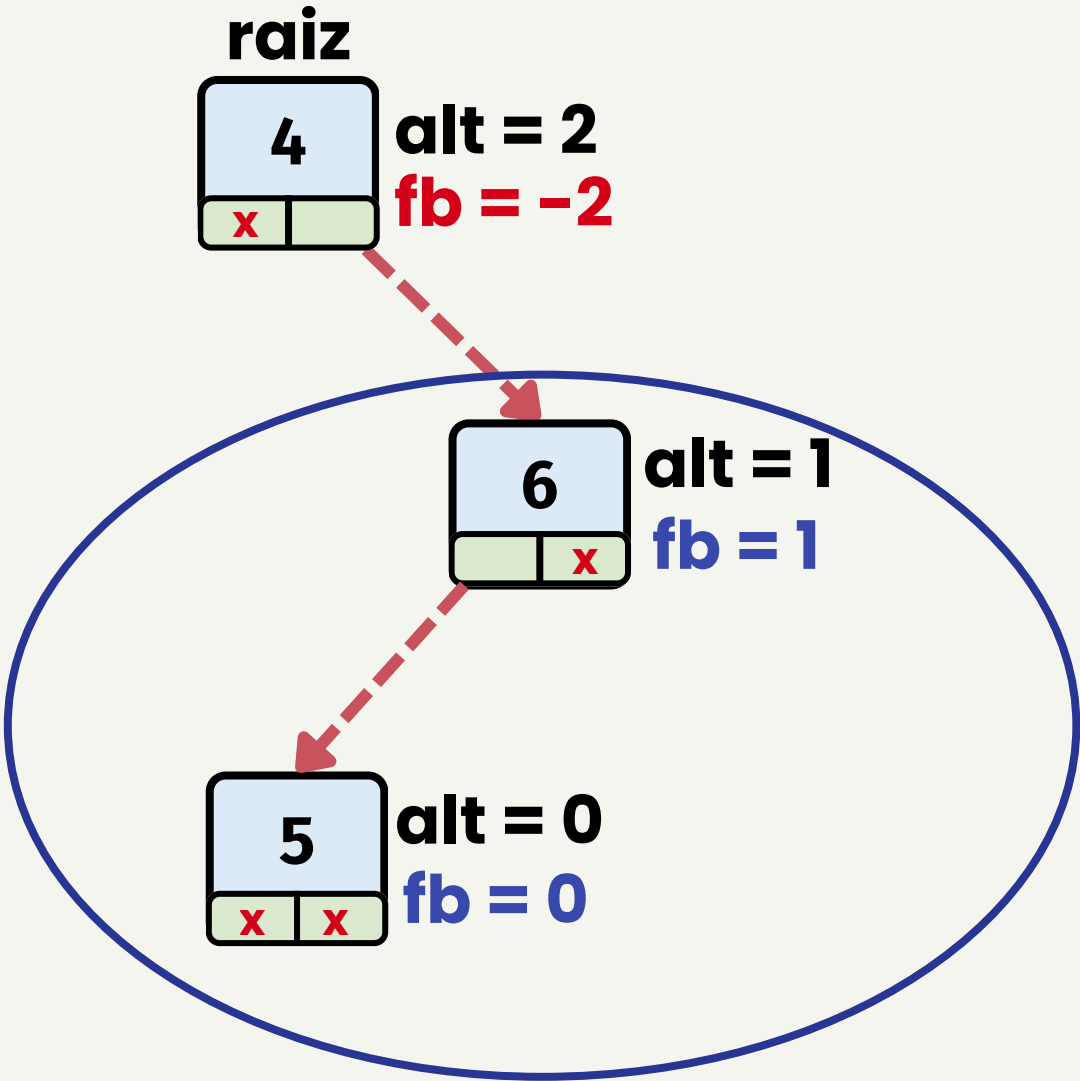
AVL

ROTAÇÃO DIREITA-ESQUERDA (RDE)



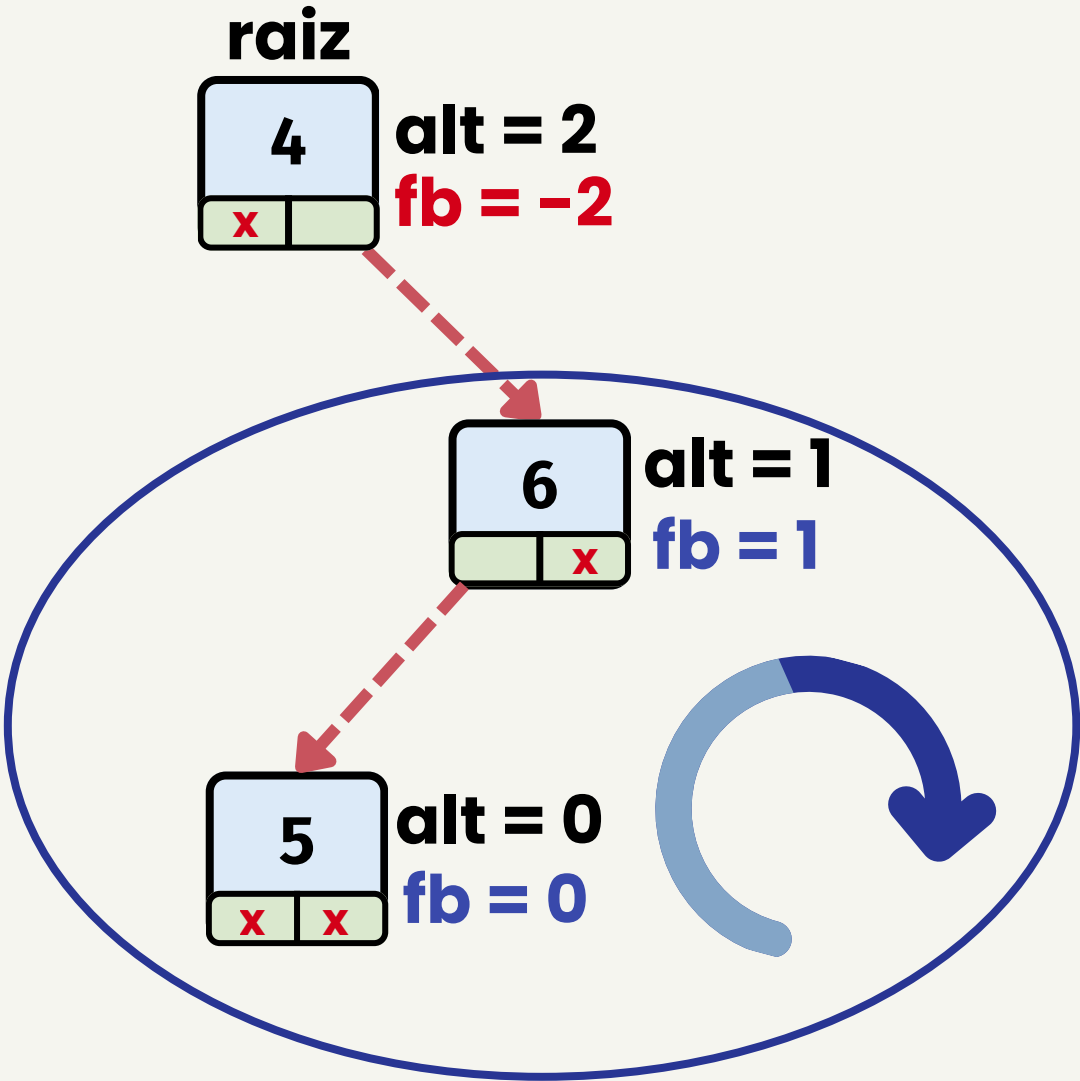
AVL

ROTAÇÃO DIREITA-ESQUERDA (RDE)



AVL

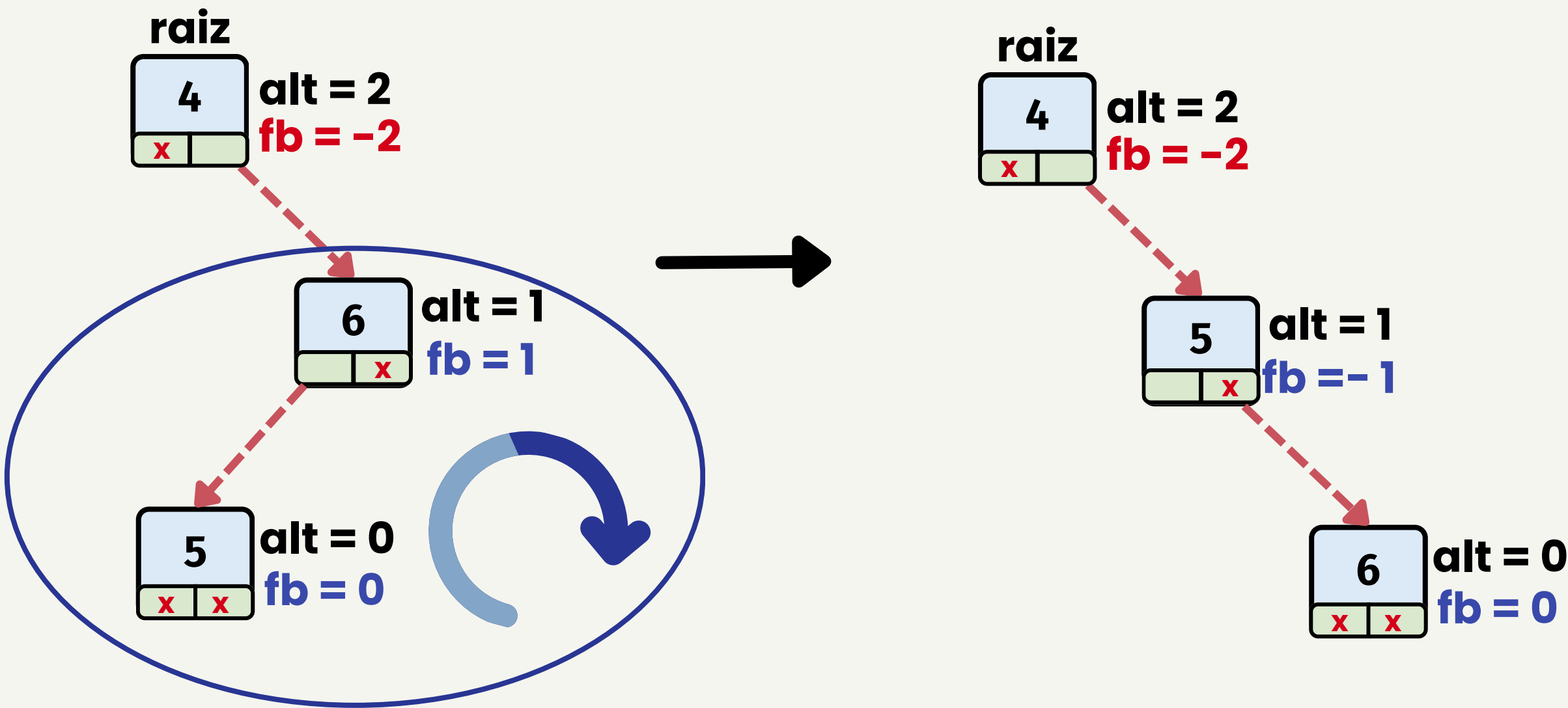
ROTAÇÃO DIREITA-ESQUERDA (RDE)



ROTAÇÃO À DIREITA

AVL

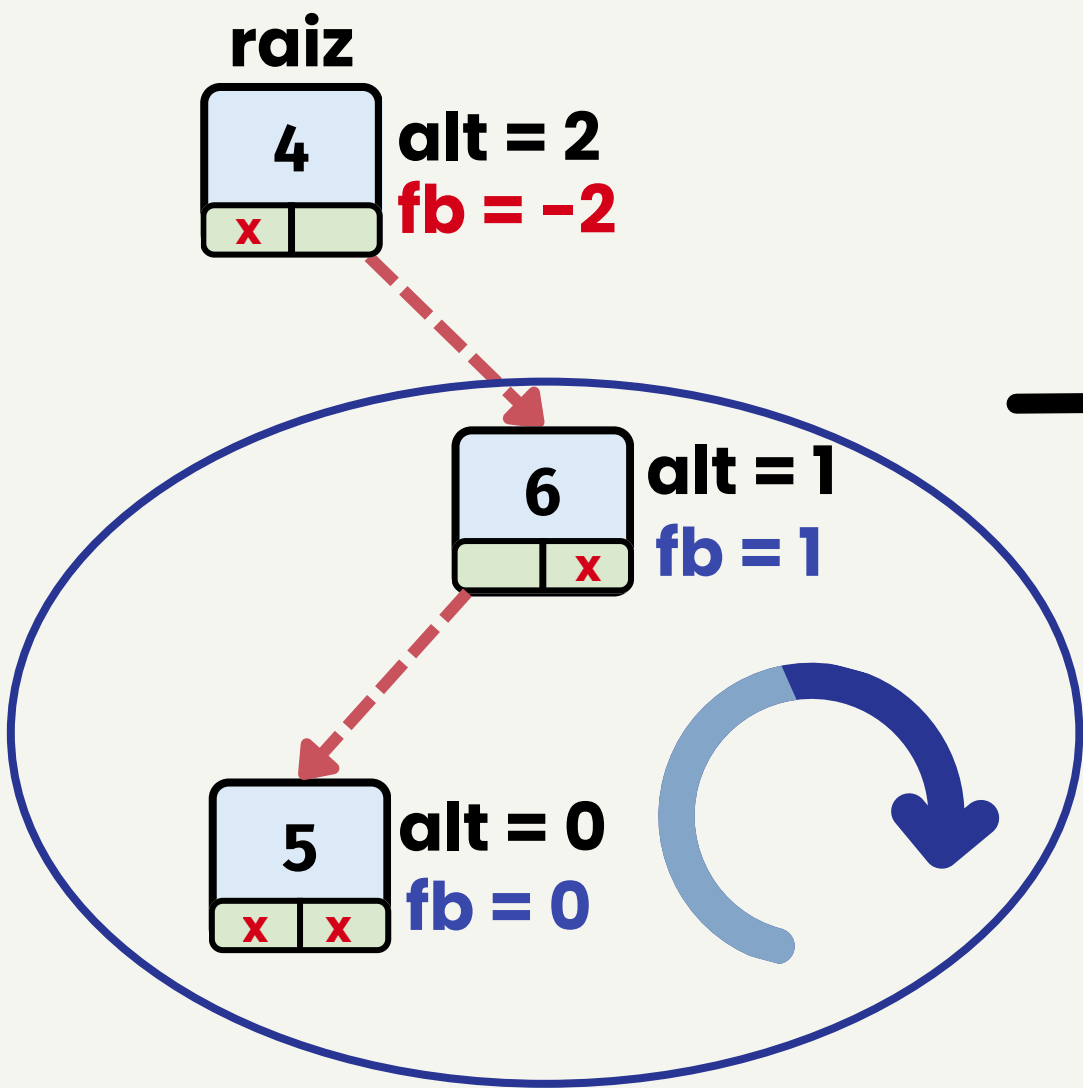
ROTAÇÃO DIREITA-ESQUERDA (RDE)



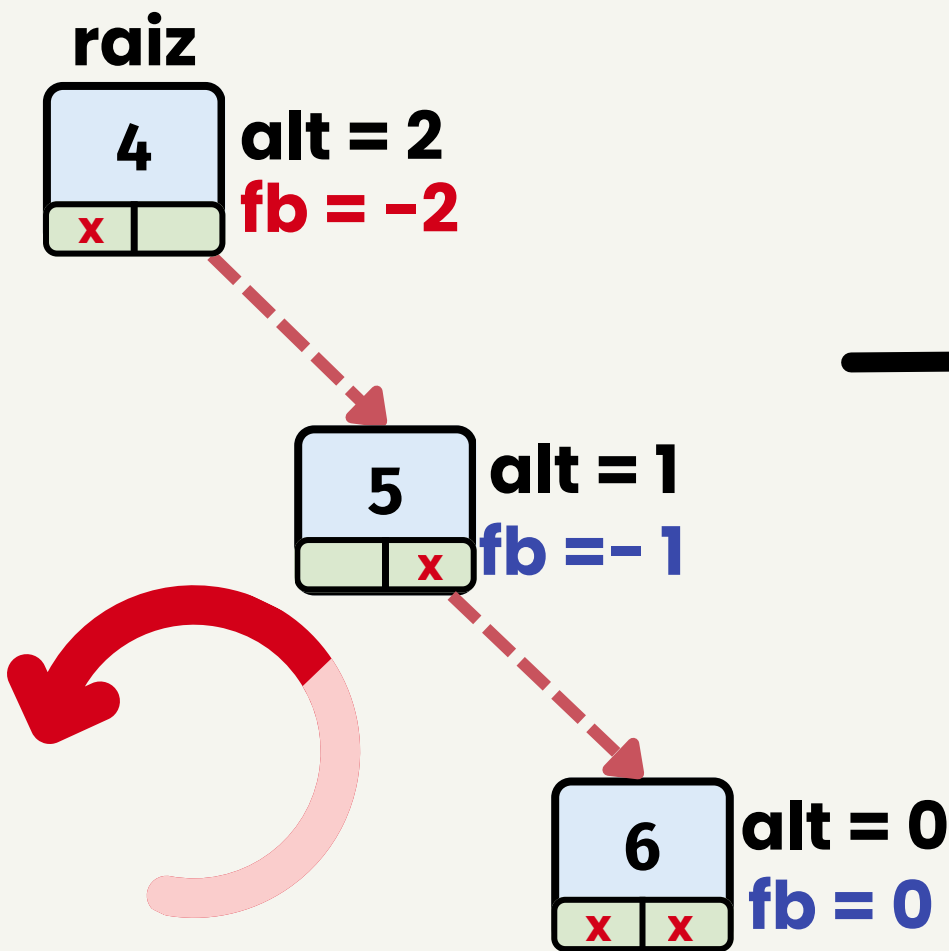
ROTAÇÃO À DIREITA

AVL

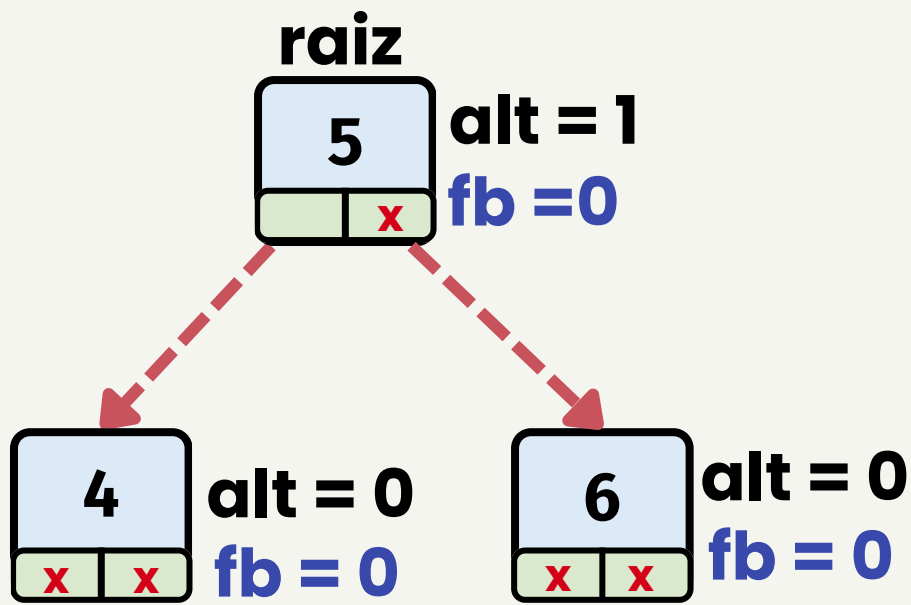
ROTAÇÃO DIREITA-ESQUERDA (RDE)



ROTAÇÃO À DIREITA



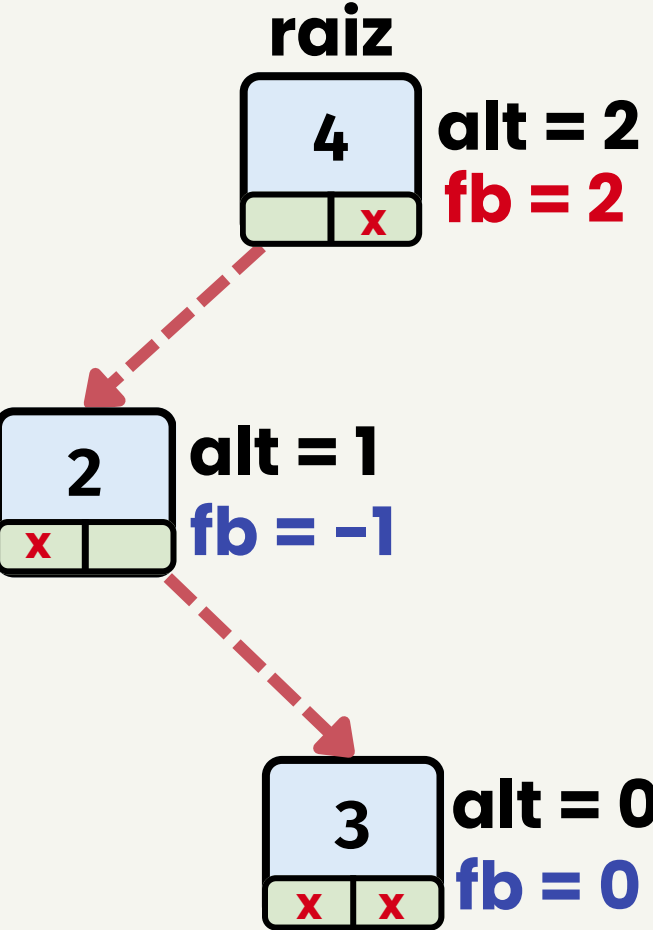
ROTAÇÃO À ESQUERDA



ÁRVORE BALANCEADA

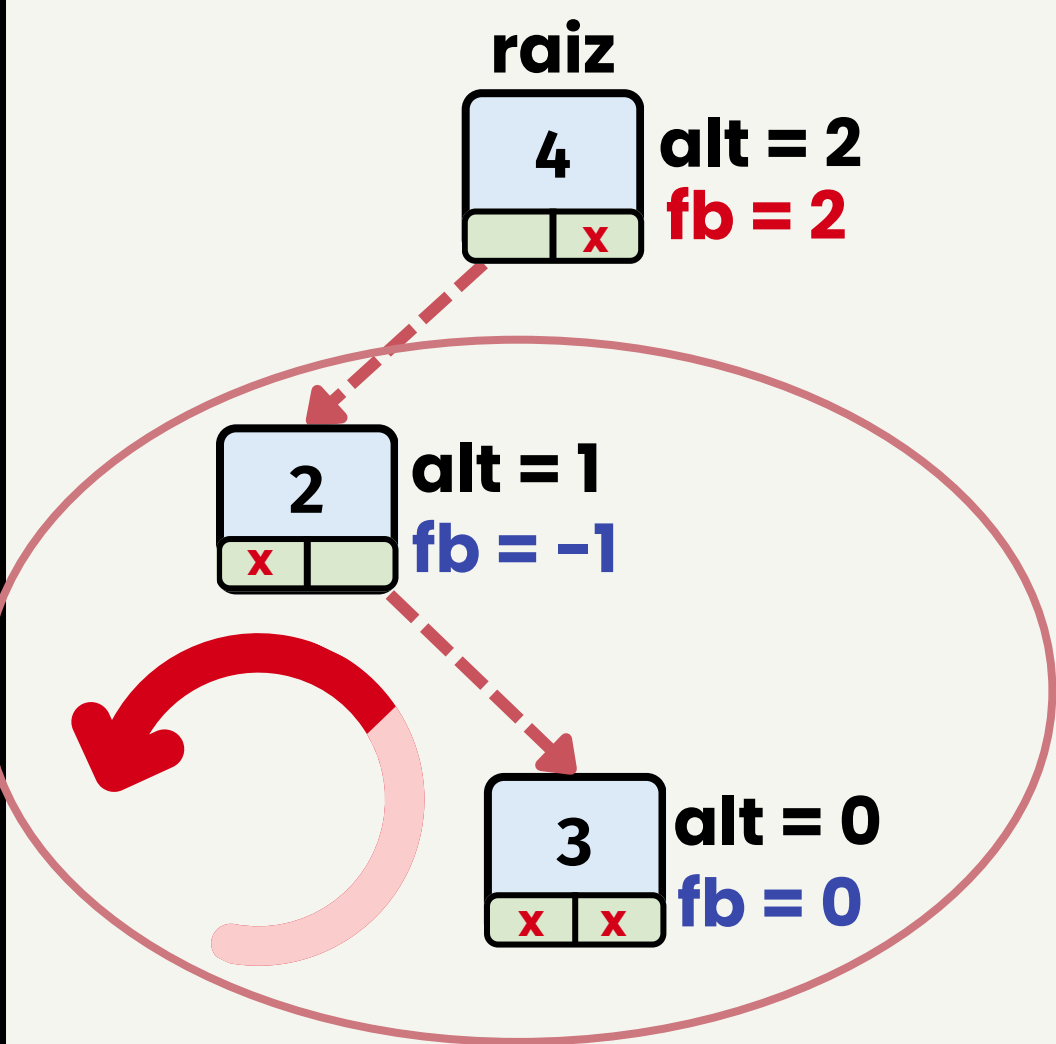
AVL

ROTAÇÃO ESQUERDA-DIREITA (RED)



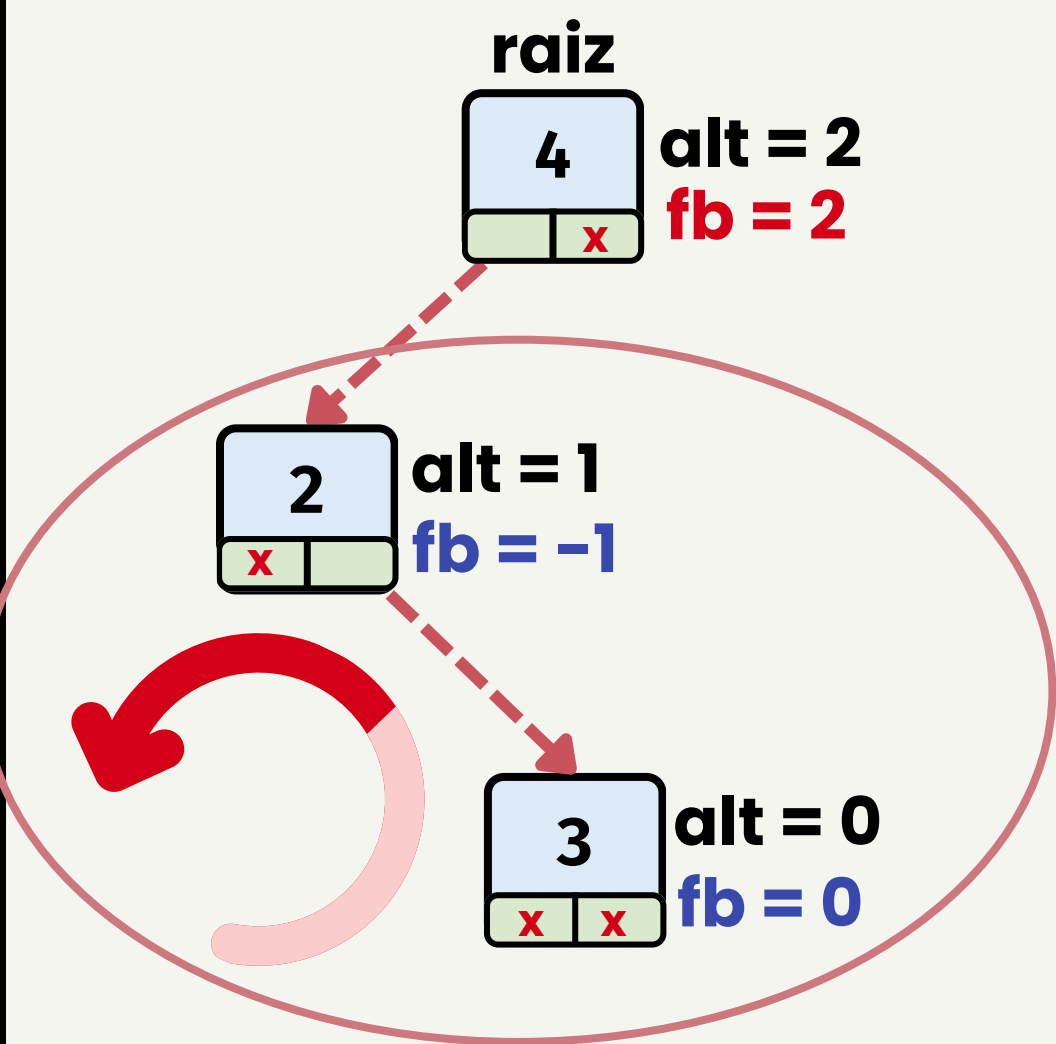
AVL

ROTAÇÃO ESQUERDA-DIREITA (RED)

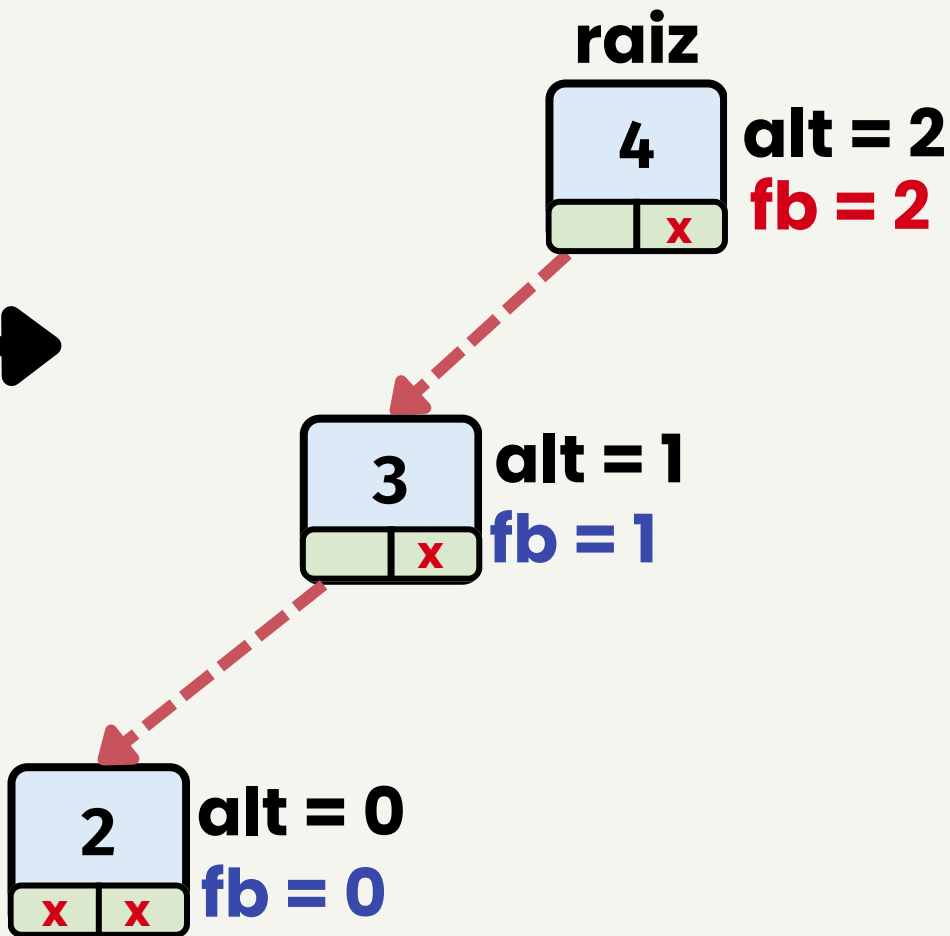


AVL

ROTAÇÃO ESQUERDA-DIREITA (RED)

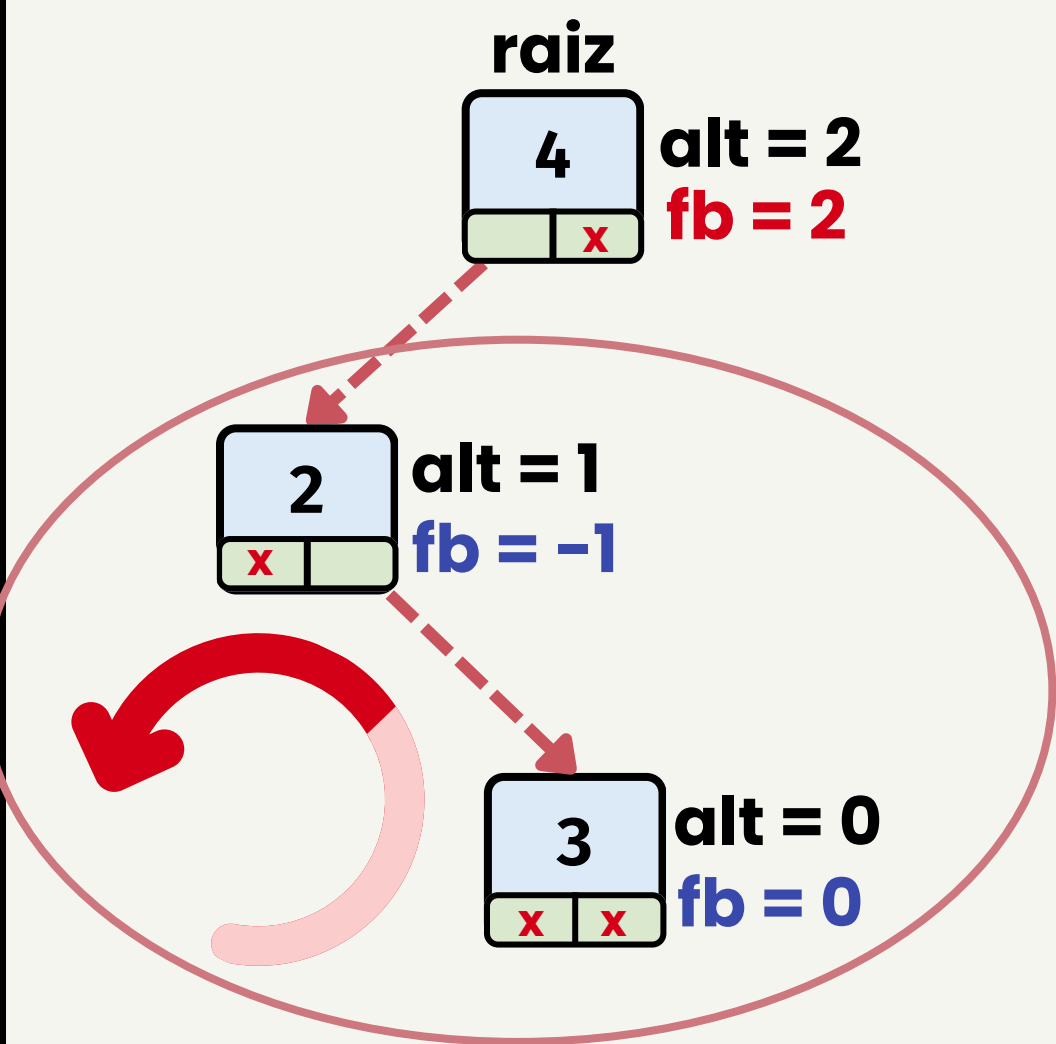


ROTAÇÃO À ESQUERDA

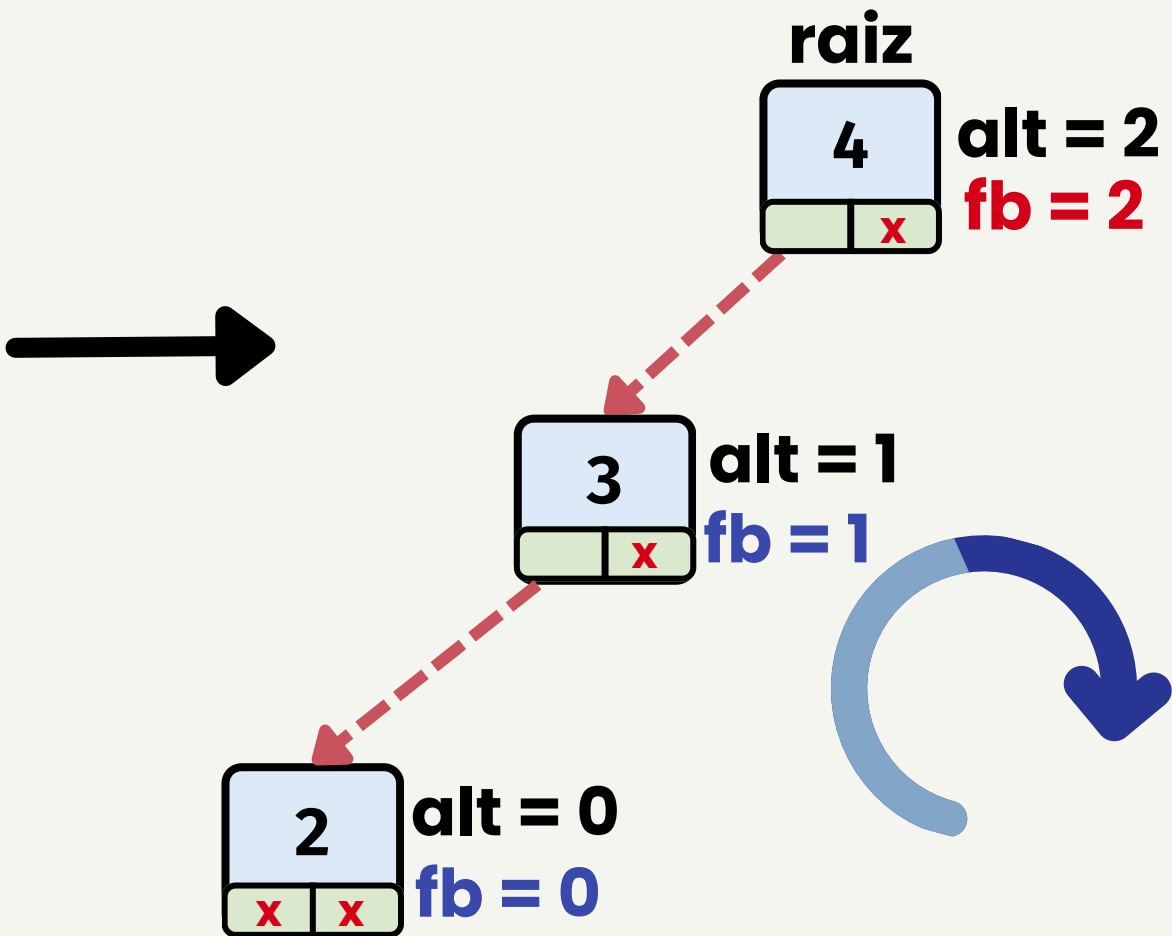


AVL

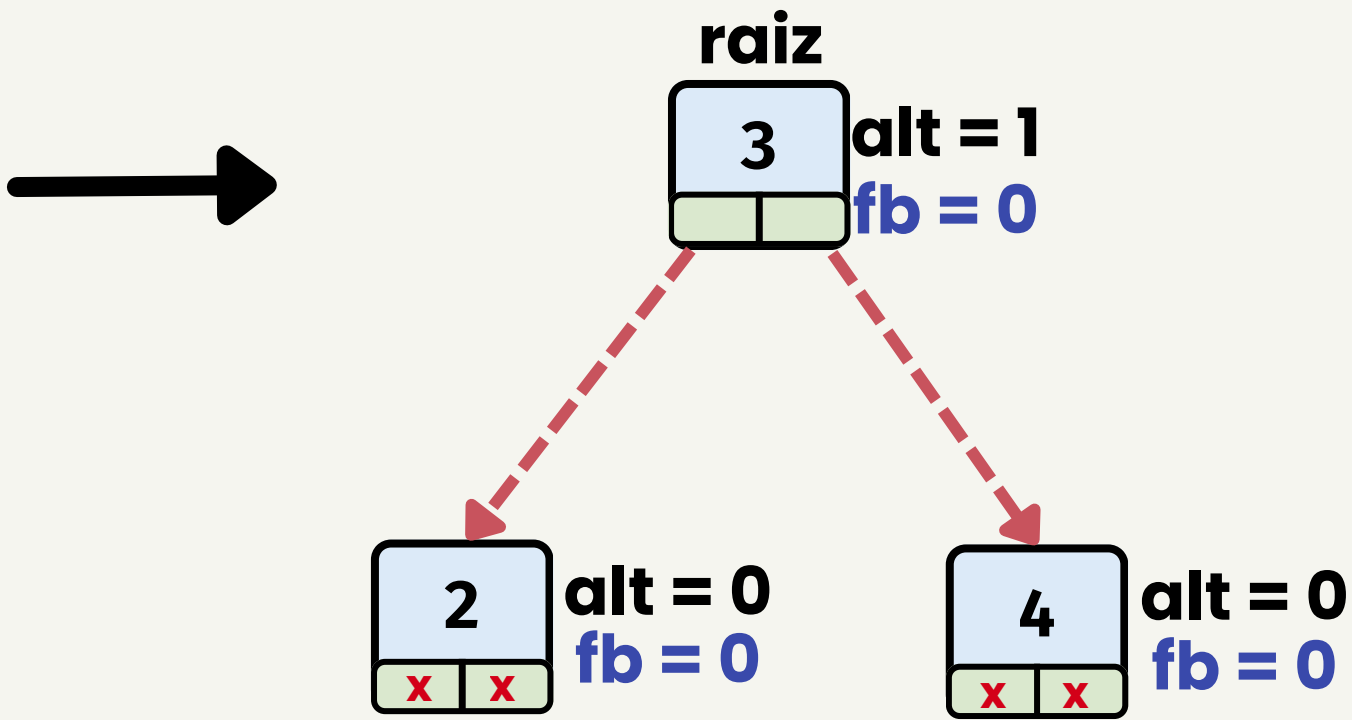
ROTAÇÃO ESQUERDA-DIREITA (RED)



ROTAÇÃO À ESQUERDA



ROTAÇÃO À DIREITA



ÁRVORE BALANCEADA

IMPLEMENTAÇÃO AVL

Estrutura da ABB

Modificações nas funções:

struct No → Altura

Criar Nó

Inserção

Remoção

Novas funções:

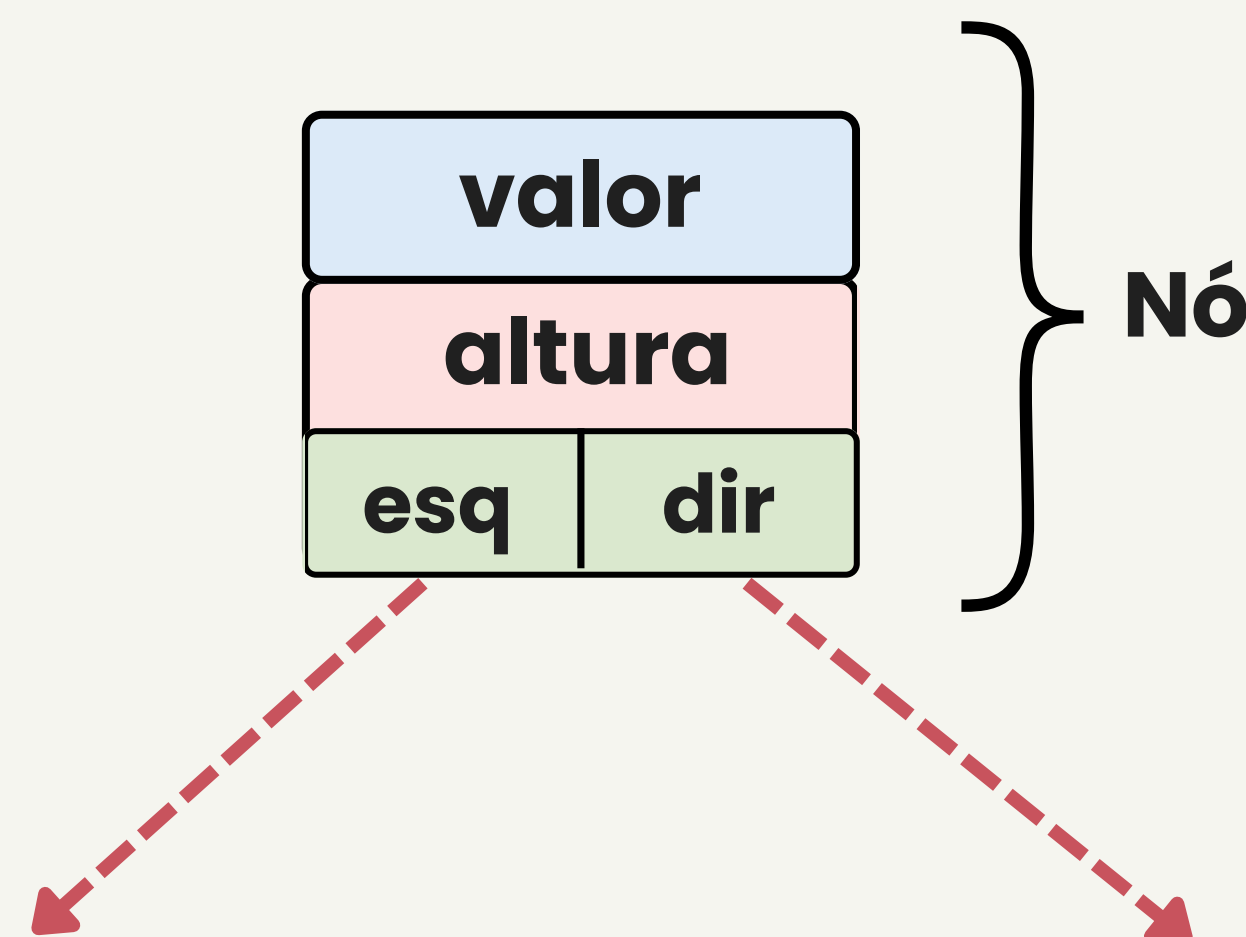
Cálculo do fb;

Balacear nó; → Rotações

IMPLEMENTAÇÃO AVL

Struct Nó:

- **valor**
- **2 ponteiros** (esquerda e direita)
- **altura**



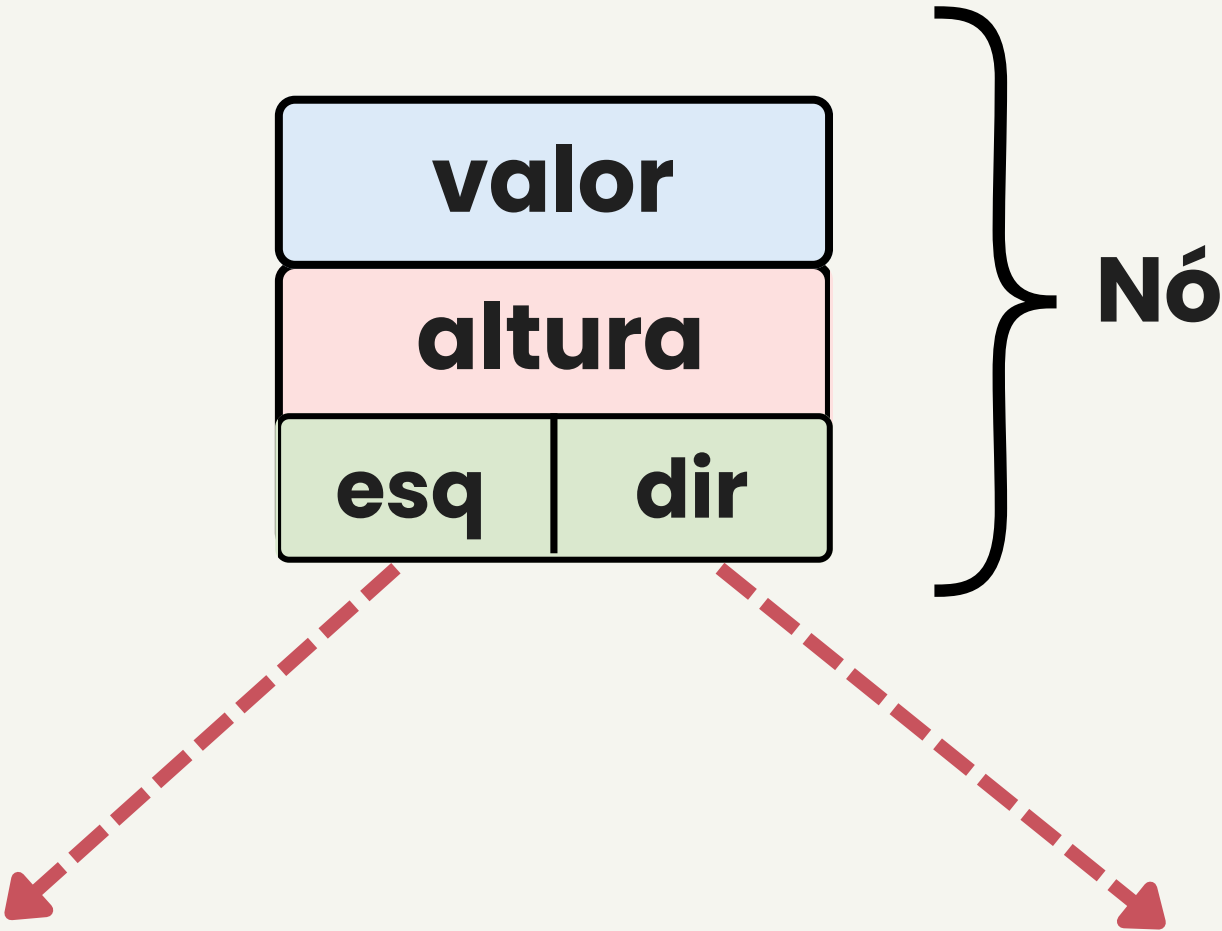
IMPLEMENTAÇÃO AVL

Struct Nó:

- **valor**
- **2 ponteiros** (esquerda e direita)
- **altura**

```

typedef struct NoArv {
    int valor;
    struct NoArv* esq;
    struct NoArv* dir;
    int altura;
} NoArv;
    
```



IMPLEMENTAÇÃO AVL

Modificação na função Criar Nó

```
static NoArv* criarNo(int valor) {  
    NoArv* novoNo = (NoArv*)malloc(sizeof(NoArv));  
    if(!novoNo) {  
        printf("Erro ao alocar memória para o nó.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->esq = NULL;  
    novoNo->dir = NULL;  
    novoNo->altura = 0;  
    return novoNo;  
}
```

IMPLEMENTAÇÃO AVL

Novas funções:

```
int alturaDoNo(NoArv* no) {  
    if (no == NULL) {  
        return -1;  
    }  
    return no->altura;  
}
```

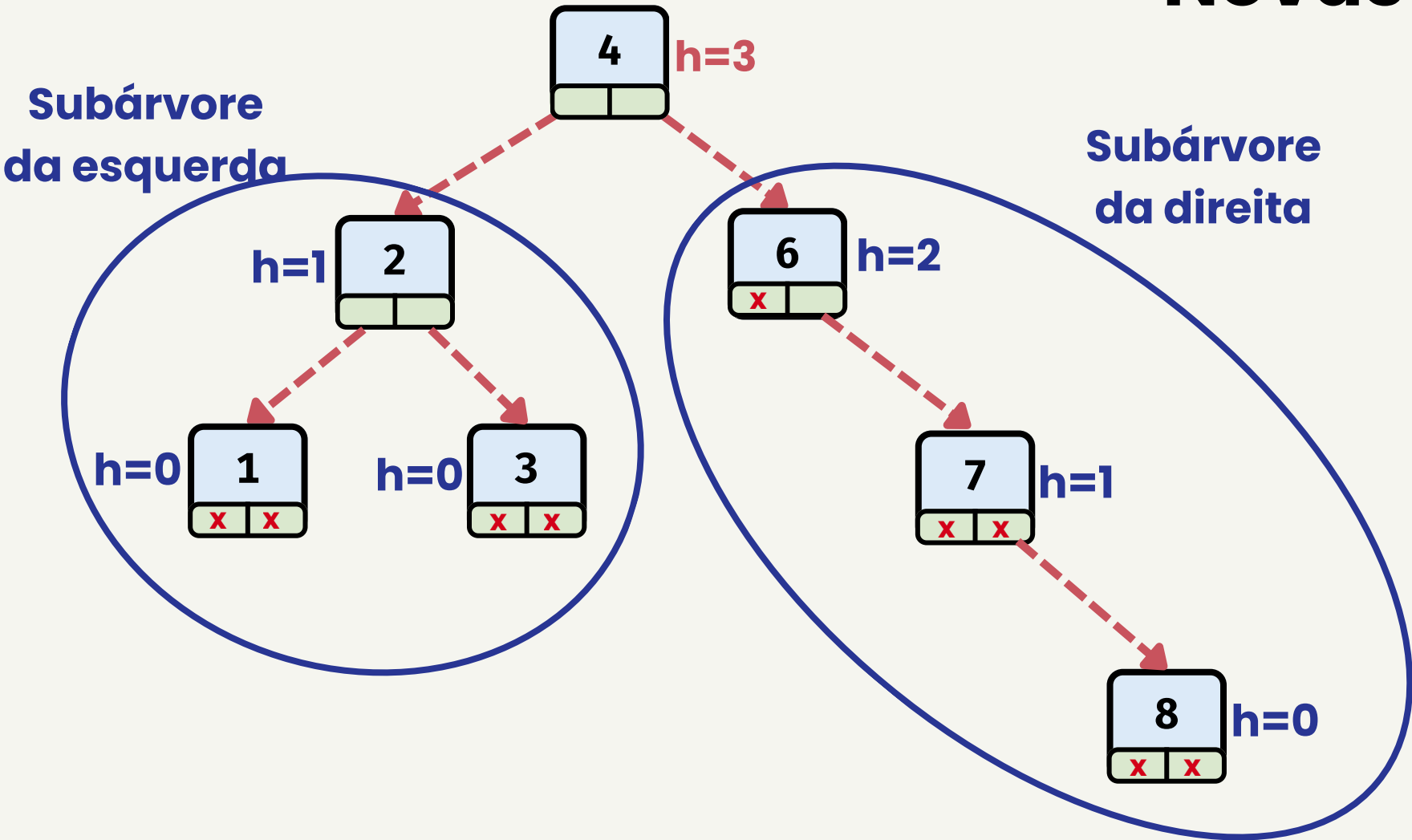
IMPLEMENTAÇÃO AVL

Novas funções:

Após alguma operação de remoção/inserção, a altura do nó vai **mudar**.

IMPLEMENTAÇÃO AVL

Novas funções:

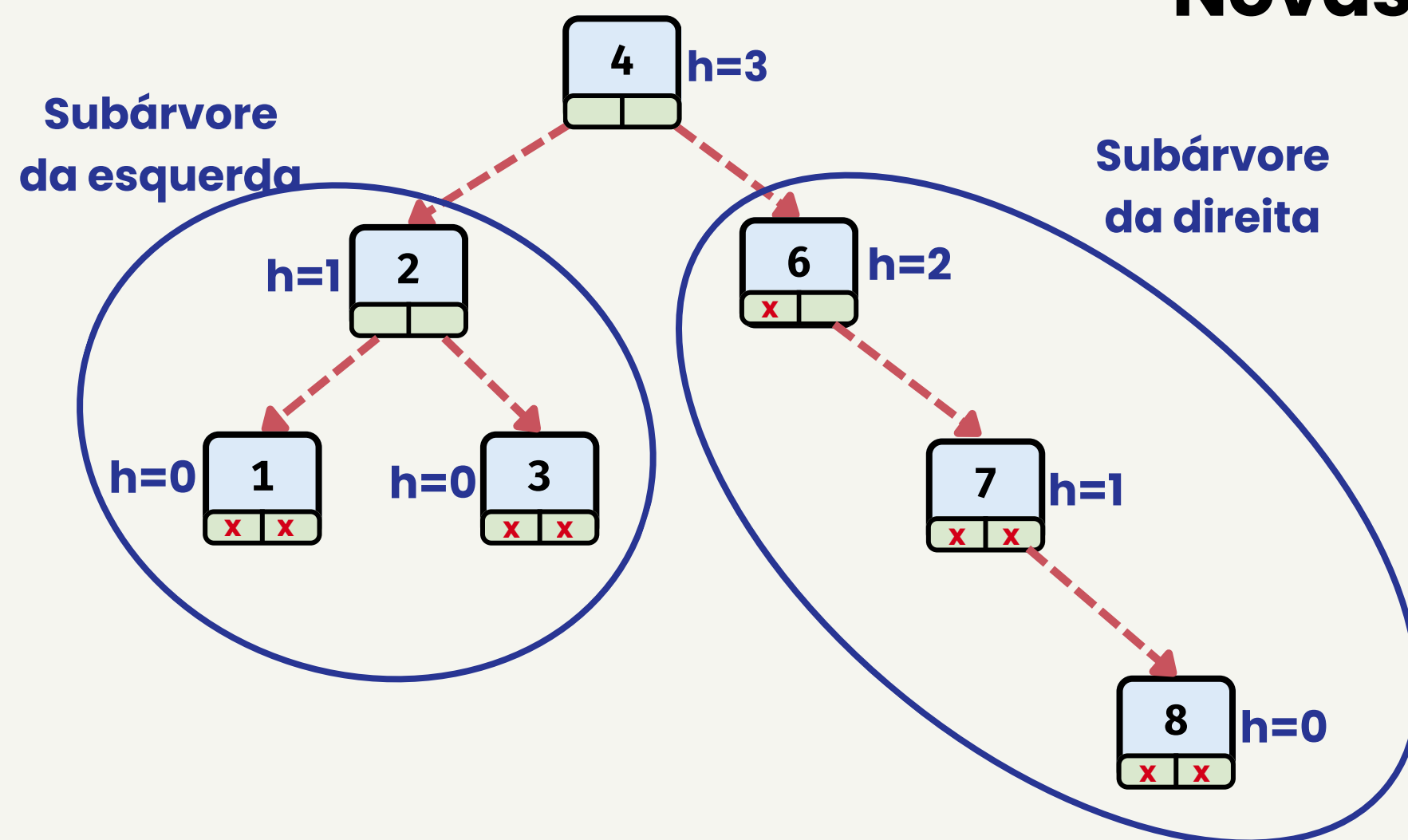


Após alguma operação de remoção/inserção, a altura do nó vai **mudar**.

Convenção Altura:
Maior altura entre a subárvore da direita e esquerda +1

IMPLEMENTAÇÃO AVL

Novas funções:



Após alguma operação de remoção/inserção, a altura do nó vai **mudar**.

Convenção Altura:

Maior altura entre a subárvore da direita e esquerda +1

Retorno: maior valor entre duas alturas de nós

```
int maiorValor(int alturaEsq, int alturaDir) {
    return (alturaEsq > alturaDir) ? alturaEsq : alturaDir;
}
```

IMPLEMENTAÇÃO AVL

Novas funções:

FATOR DE BALANCEAMENTO (fb)

$$\text{fb} = \text{altEsq} - \text{altDir}$$

```
int fatorBalanceamento(NoArv* no) {  
    if (no == NULL) {  
        return 0;  
    }  
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);  
}
```

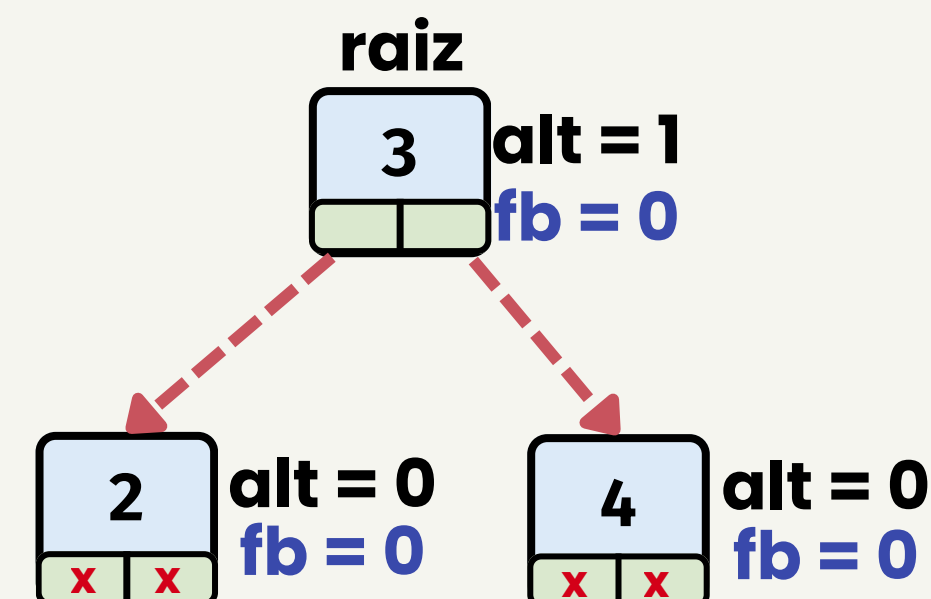
IMPLEMENTAÇÃO AVL

Novas funções:

FATOR DE BALANCEAMENTO (fb)

$$\text{fb} = \text{altEsq} - \text{altDir}$$

fb = 0, -1 ou 1 → Balanceada



```

int fatorBalanceamento(NoArv* no) {
    if (no == NULL) {
        return 0;
    }
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);
}
    
```

IMPLEMENTAÇÃO AVL

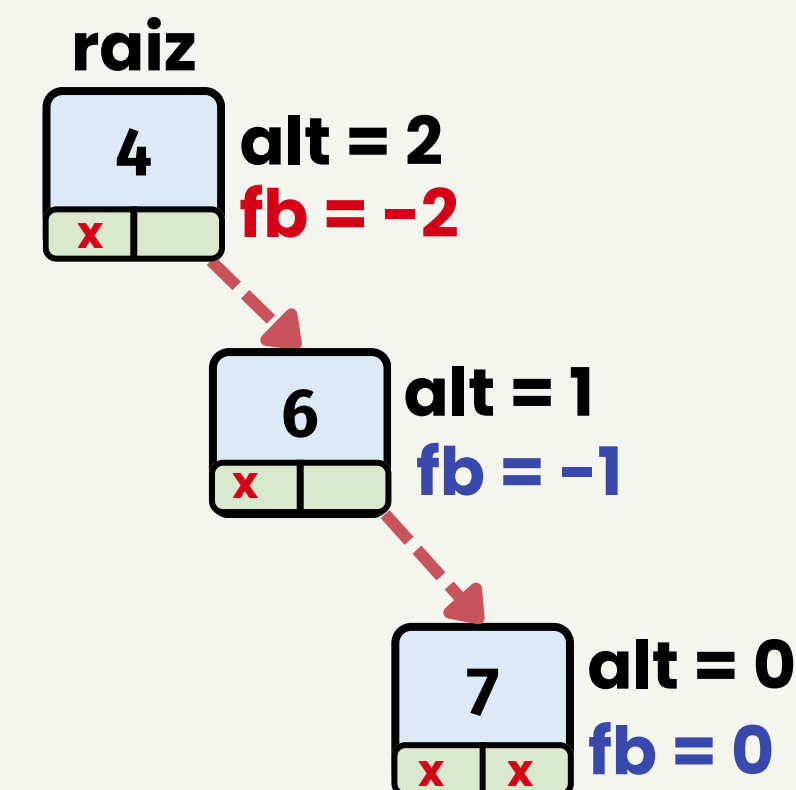
Novas funções:

FATOR DE BALANCEAMENTO (fb)

$$\text{fb} = \text{altEsq} - \text{altDir}$$

fb = 0, -1 ou 1 → Balanceada

fb < -1 → Desbalanceada para a direita



```

int fatorBalanceamento(NoArv* no) {
    if (no == NULL) {
        return 0;
    }
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);
}
    
```

IMPLEMENTAÇÃO AVL

Novas funções:

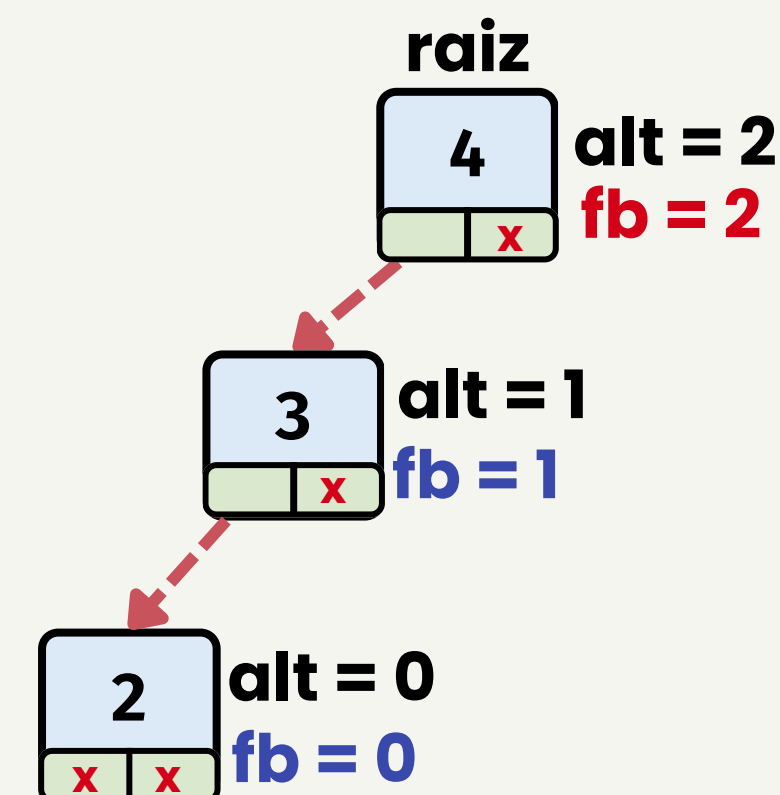
FATOR DE BALANCEAMENTO (fb)

$$\text{fb} = \text{altEsq} - \text{altDir}$$

fb = 0, -1 ou 1 → Balanceada

fb < -1 → Desbalanceada para a **direita**

fb > 1 → Desbalanceada para a **esquerda**

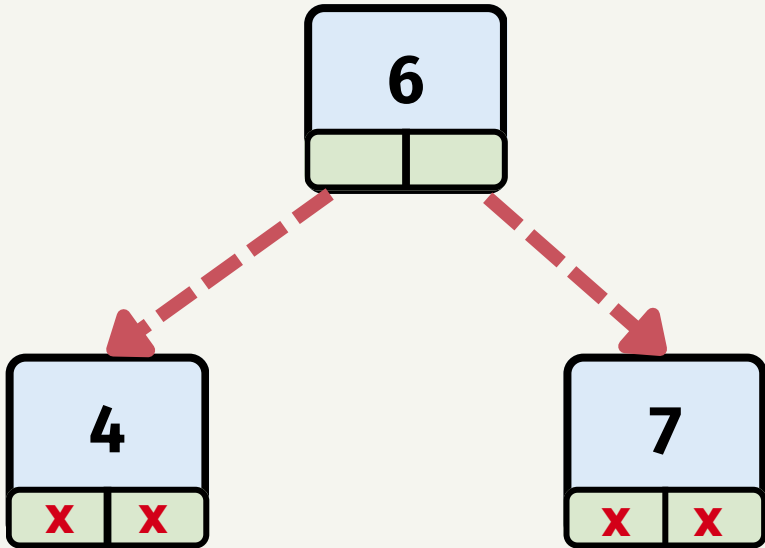
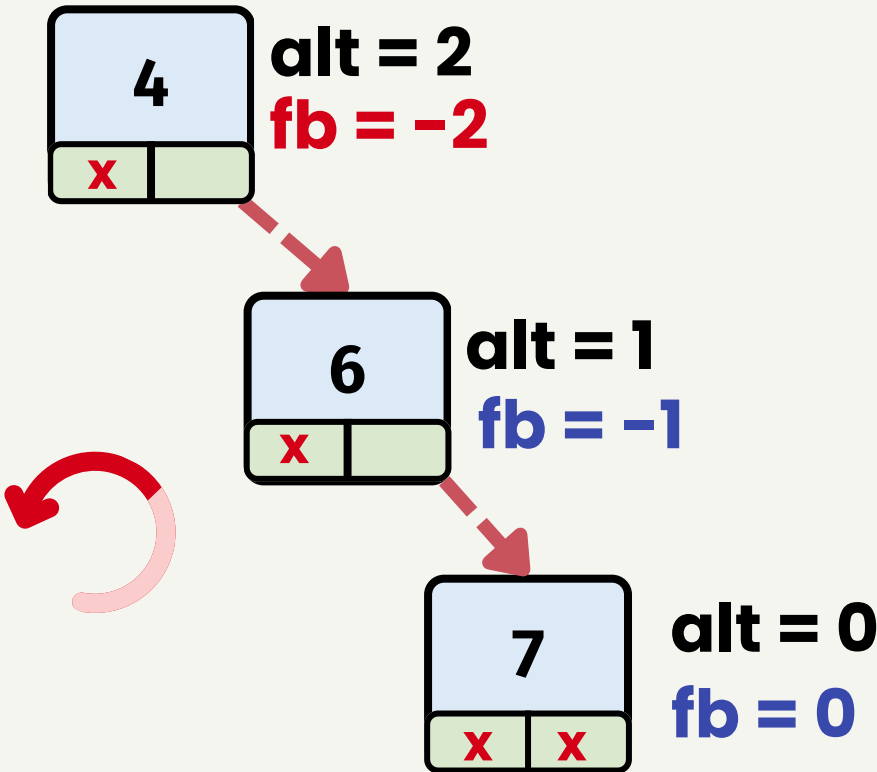


```

int fatorBalanceamento(NoArv* no) {
    if (no == NULL) {
        return 0;
    }
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);
}
    
```

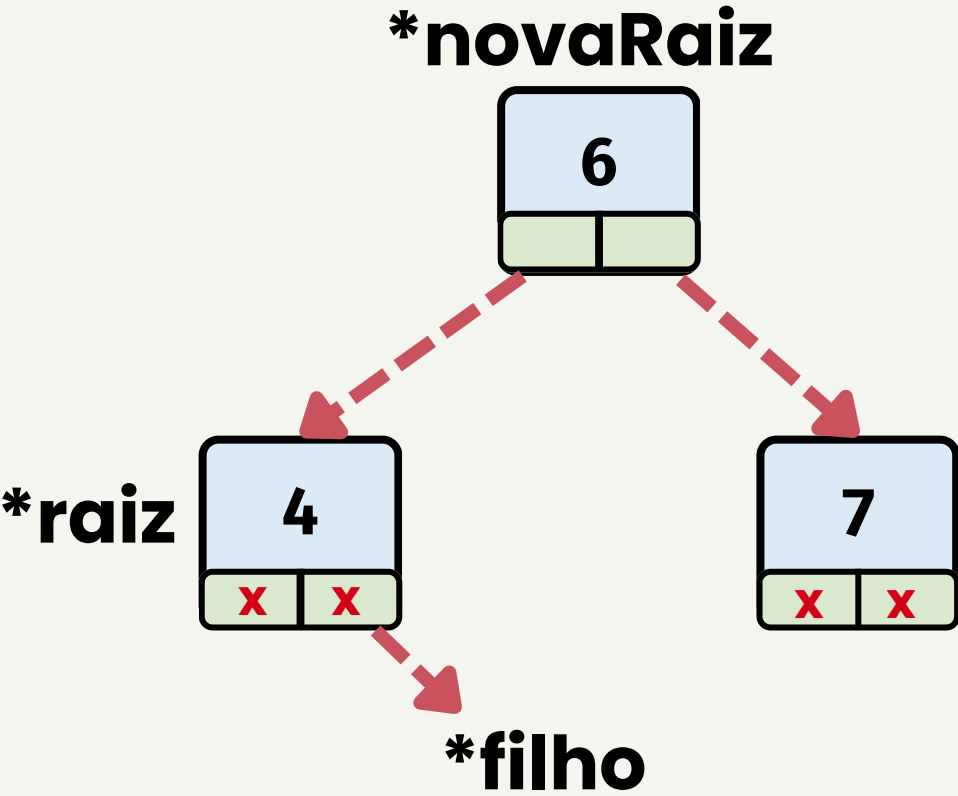
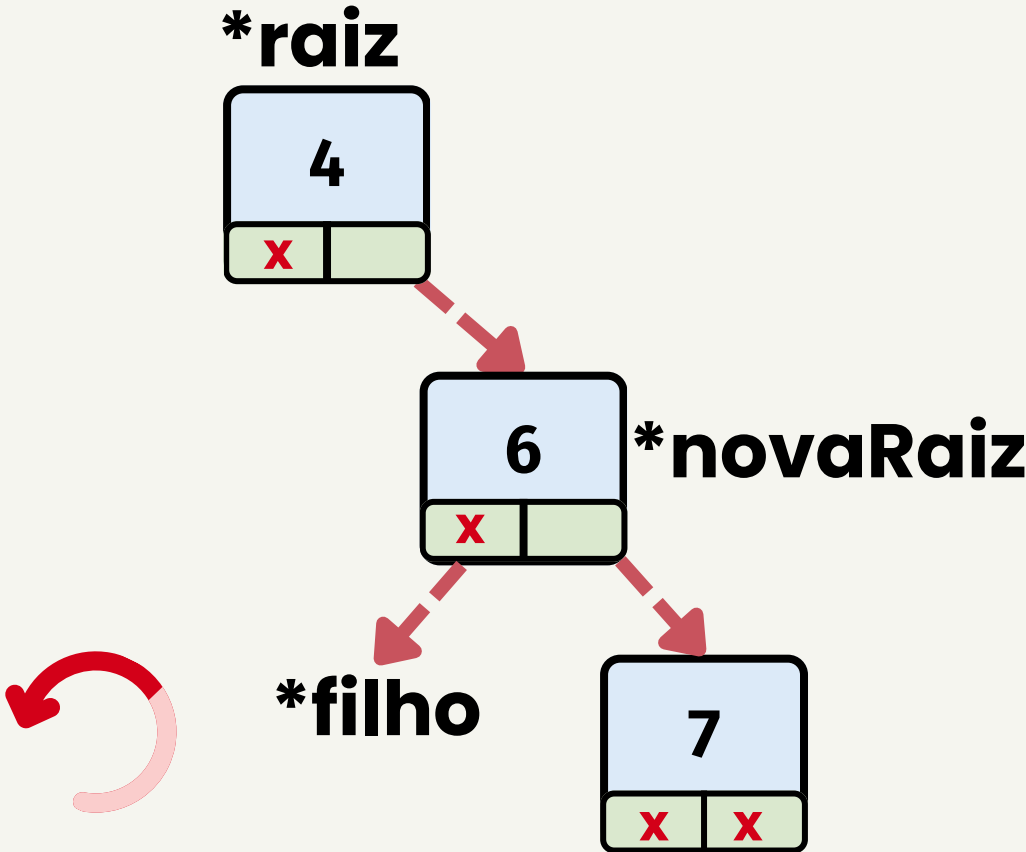
ROTAÇÃO À ESQUERDA

fb < -1



ROTAÇÃO À ESQUERDA

fb < -1



ROTAÇÃO À ESQUERDA

fb < -1

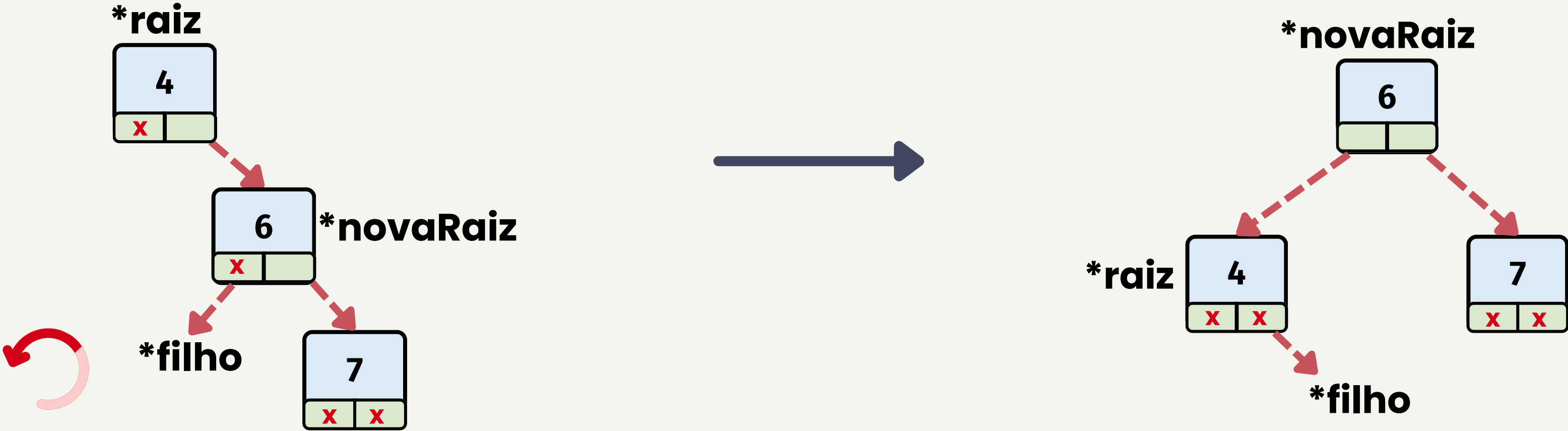
```

NoArv* rotacaoEsquerda(NoArv* raiz) {
    NoArv* novaRaiz = raiz->dir;
    NoArv* filho = novaRaiz->esq;

    novaRaiz->esq = raiz;
    raiz->dir = filho;

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));

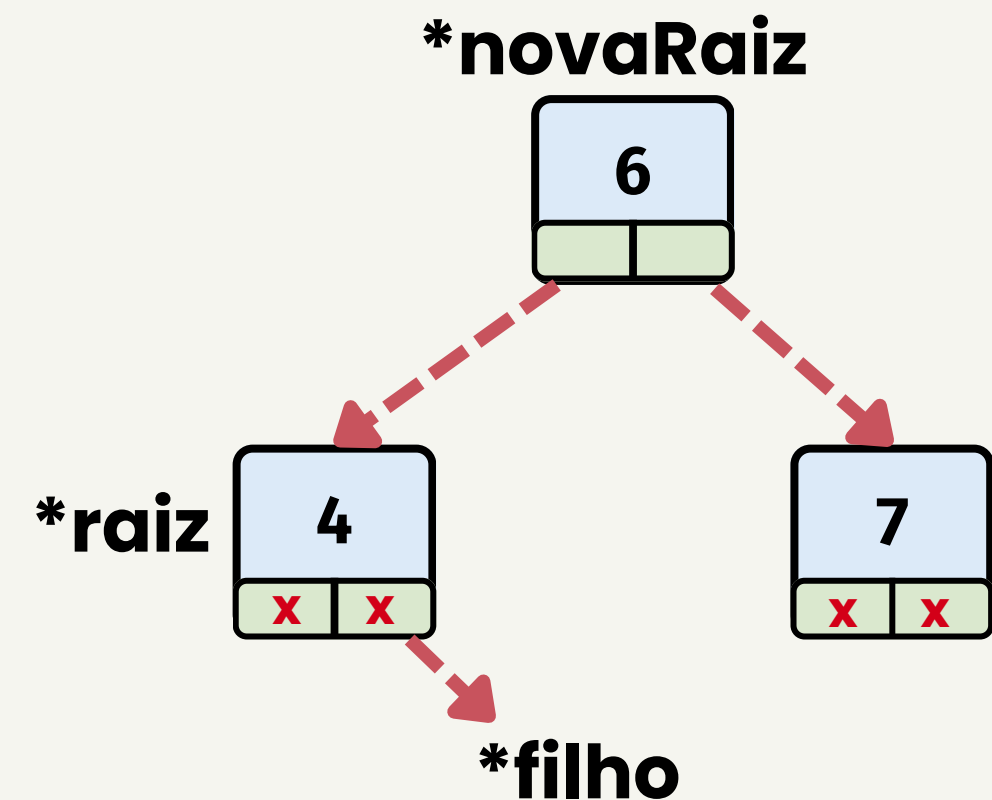
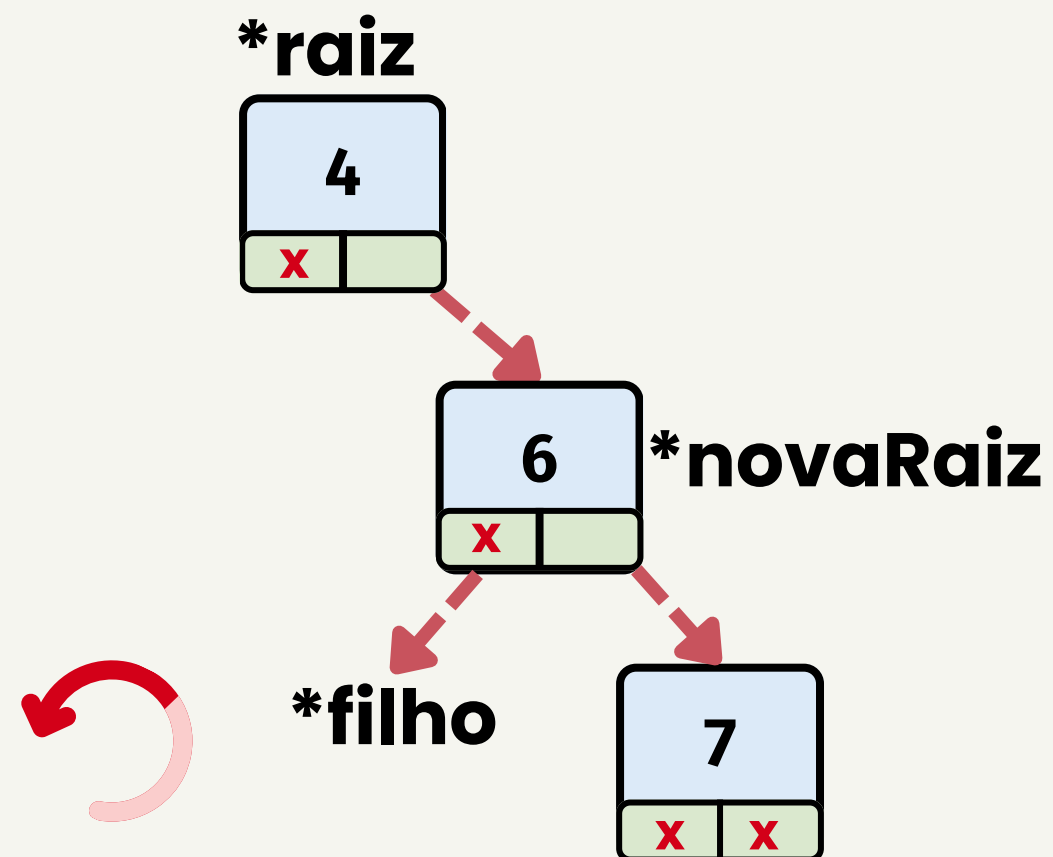
    return novaRaiz;
}
    
```



ROTAÇÃO À ESQUERDA

fb < -1

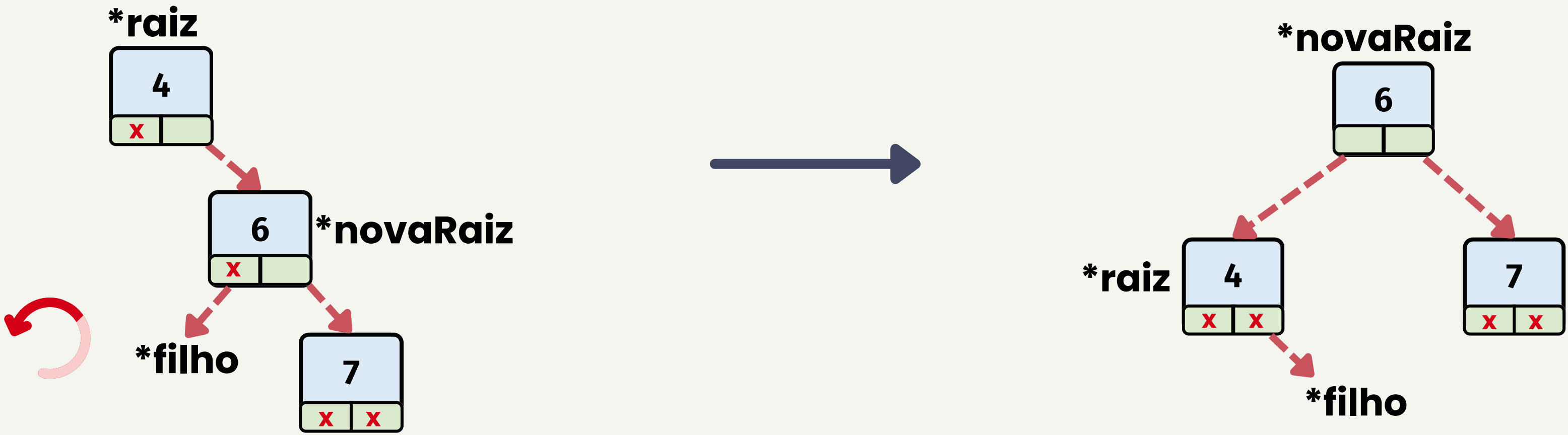
```
NoArv* rotacaoEsquerda(NoArv* raiz) {  
    NoArv* novaRaiz = raiz->dir;  
    NoArv* filho = novaRaiz->esq;  
  
    novaRaiz->esq = raiz;  
    raiz->dir = filho;  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));  
  
    return novaRaiz;  
}
```



ROTAÇÃO À ESQUERDA

fb < -1

```
NoArv* rotacaoEsquerda(NoArv* raiz) {  
    NoArv* novaRaiz = raiz->dir;  
    NoArv* filho = novaRaiz->esq;  
  
    novaRaiz->esq = raiz;  
    raiz->dir = filho;  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));  
  
    return novaRaiz;  
}
```



ROTAÇÃO À ESQUERDA

fb < -1

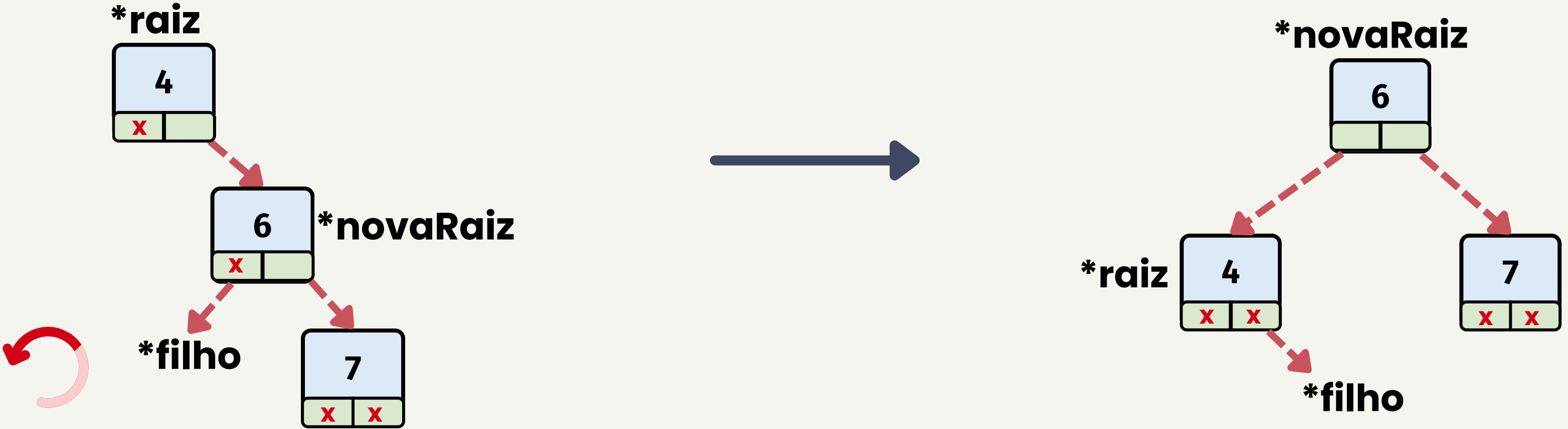
```

NoArv* rotacaoEsquerda(NoArv* raiz) {
    NoArv* novaRaiz = raiz->dir;
    NoArv* filho = novaRaiz->esq;

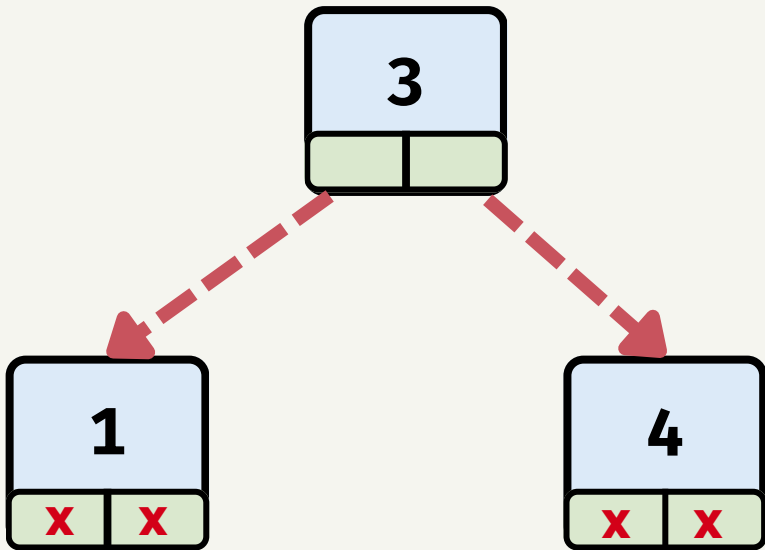
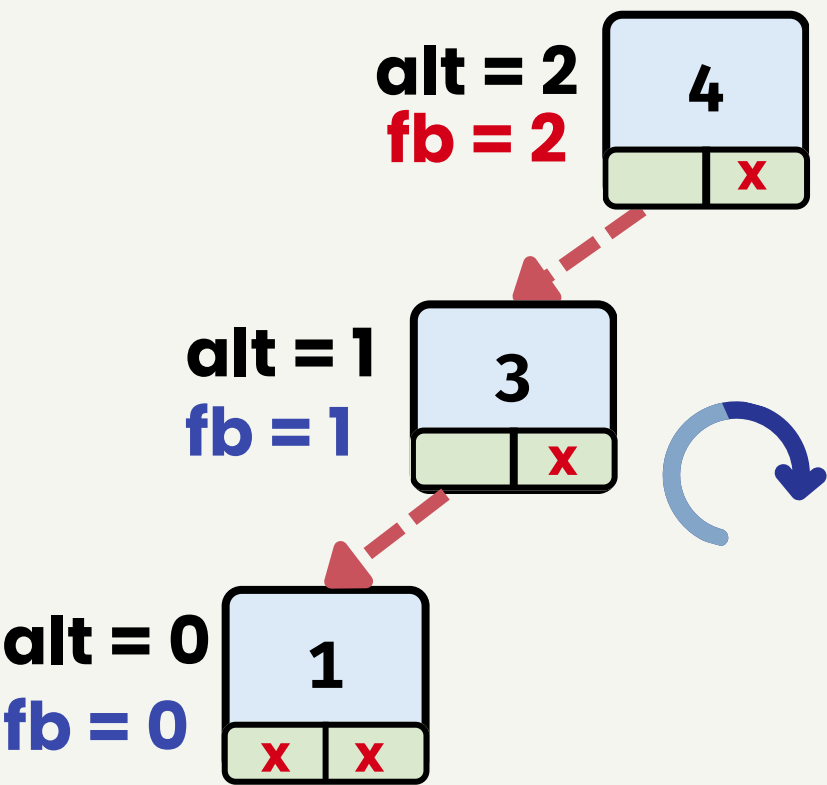
    novaRaiz->esq = raiz;
    raiz->dir = filho;

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));

    return novaRaiz;
}
    
```

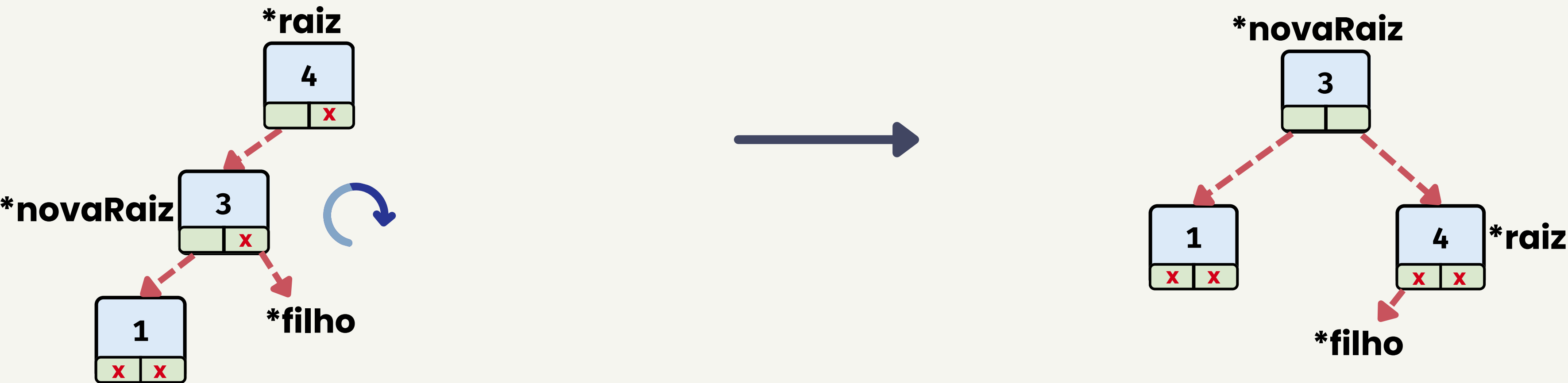


ROTAÇÃO À DIREITA fb > 1



ROTAÇÃO À DIREITA

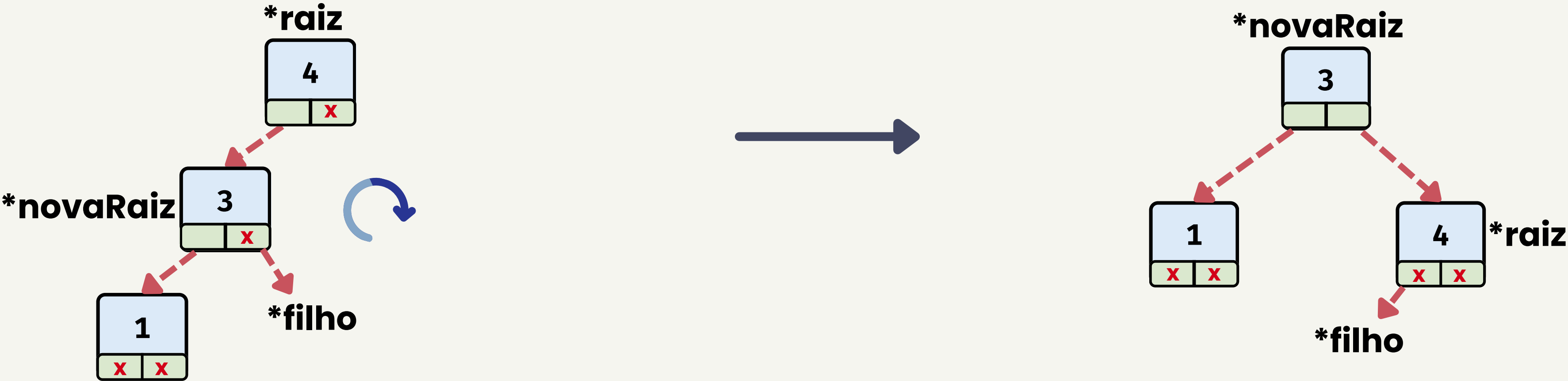
fb > 1



ROTAÇÃO À DIREITA

fb > 1

```
NoArv* rotacaoDireita(NoArv* raiz) {  
    NoArv* novaRaiz = raiz->esq;  
    NoArv* filho = novaRaiz->dir;  
  
    novaRaiz->dir = raiz;  
    raiz->esq = filho;  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));  
  
    return novaRaiz;  
}
```



ROTAÇÃO À DIREITA

fb > 1

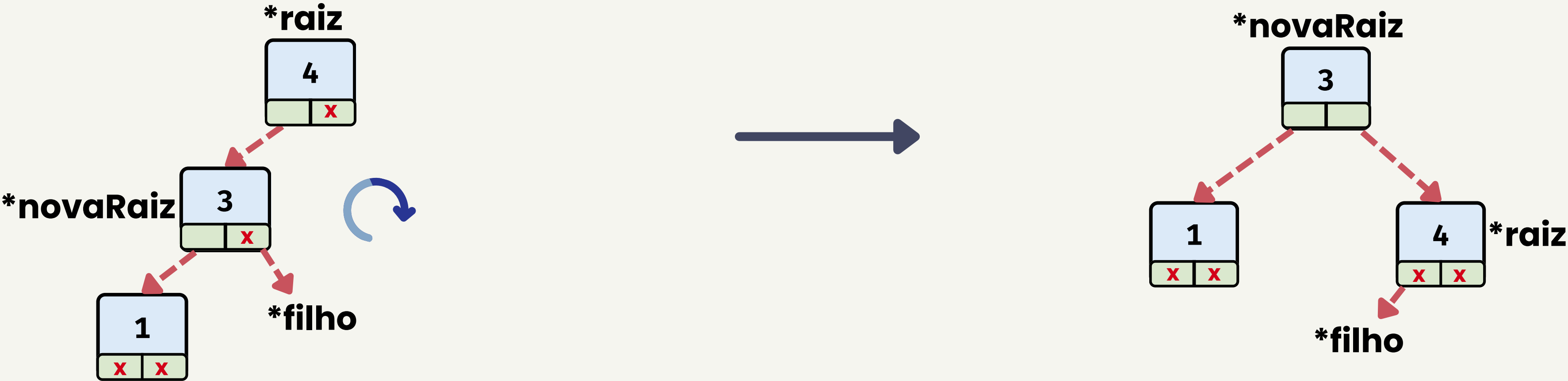
```

NoArv* rotacaoDireita(NoArv* raiz) {
    NoArv* novaRaiz = raiz->esq;
    NoArv* filho = novaRaiz->dir;

    novaRaiz->dir = raiz;
    raiz->esq = filho;

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));

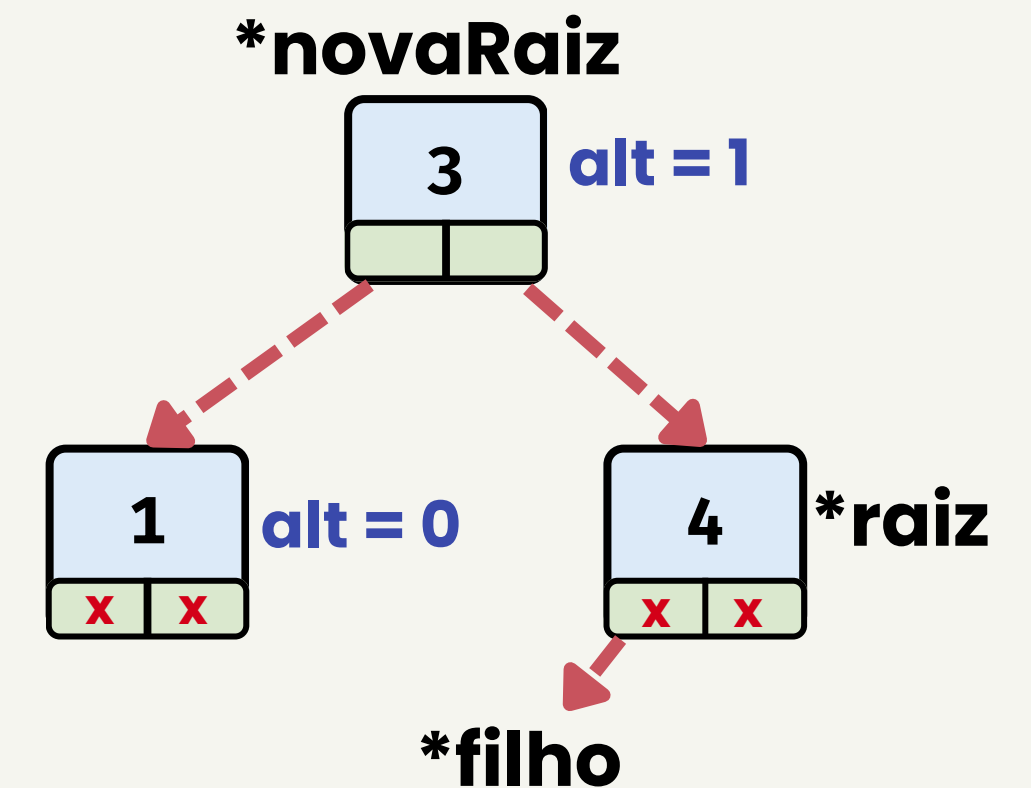
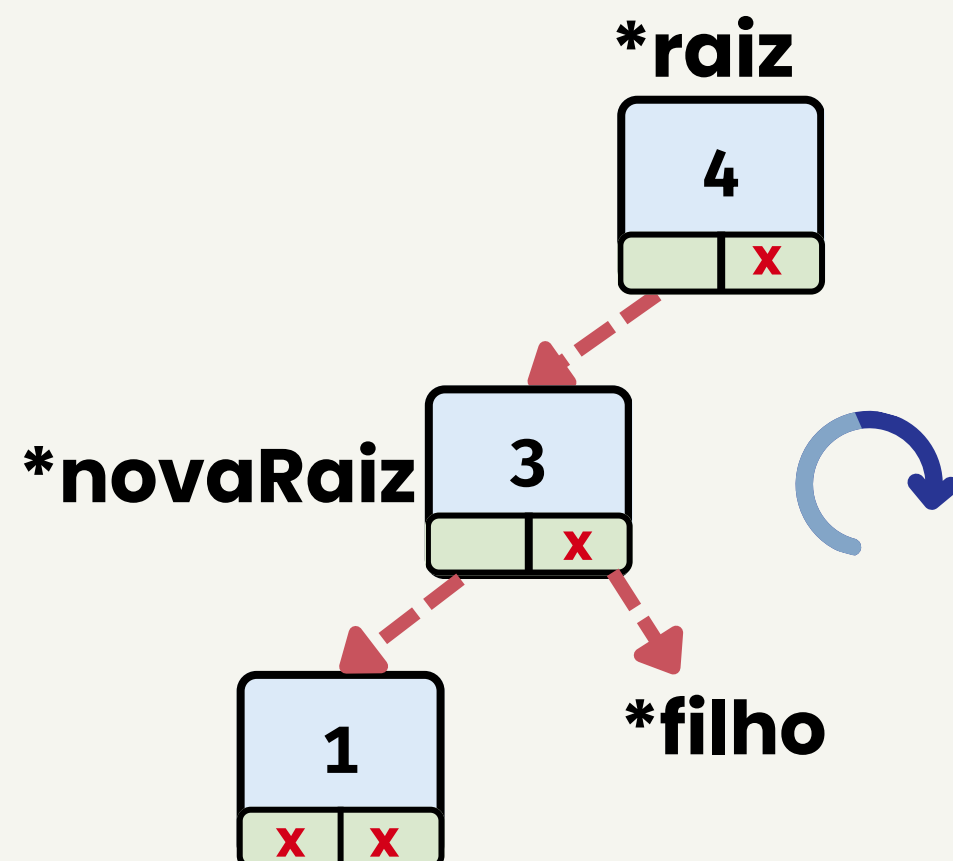
    return novaRaiz;
}
    
```



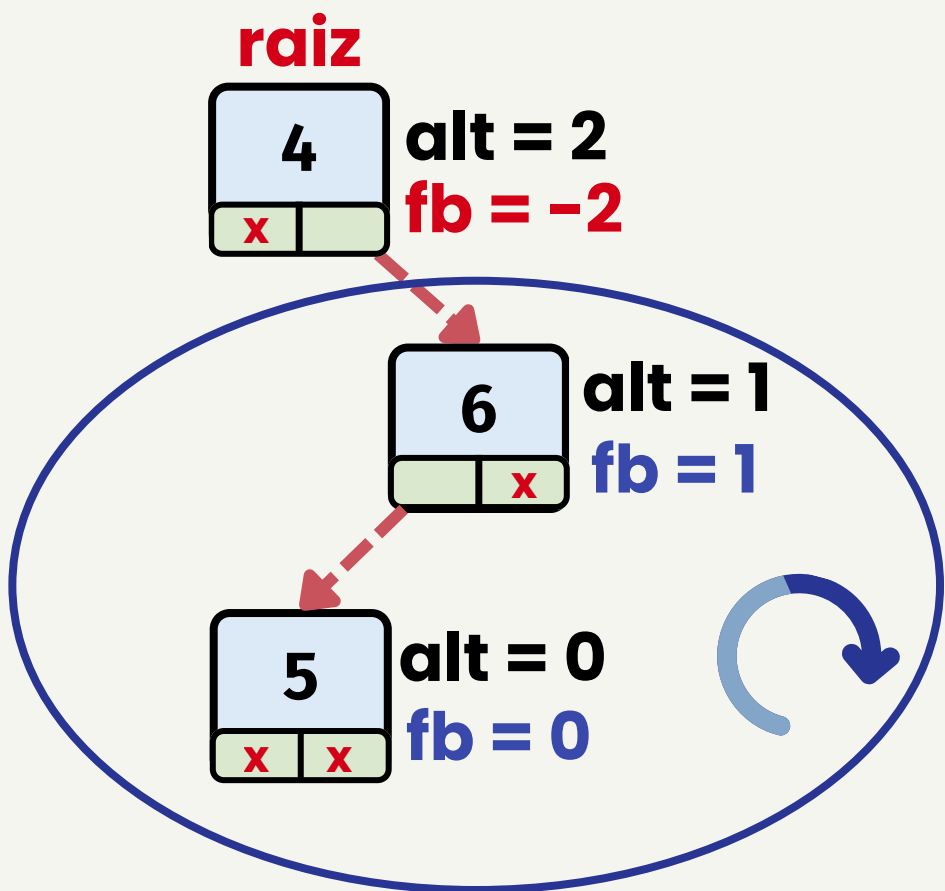
ROTAÇÃO À DIREITA

fb > 1

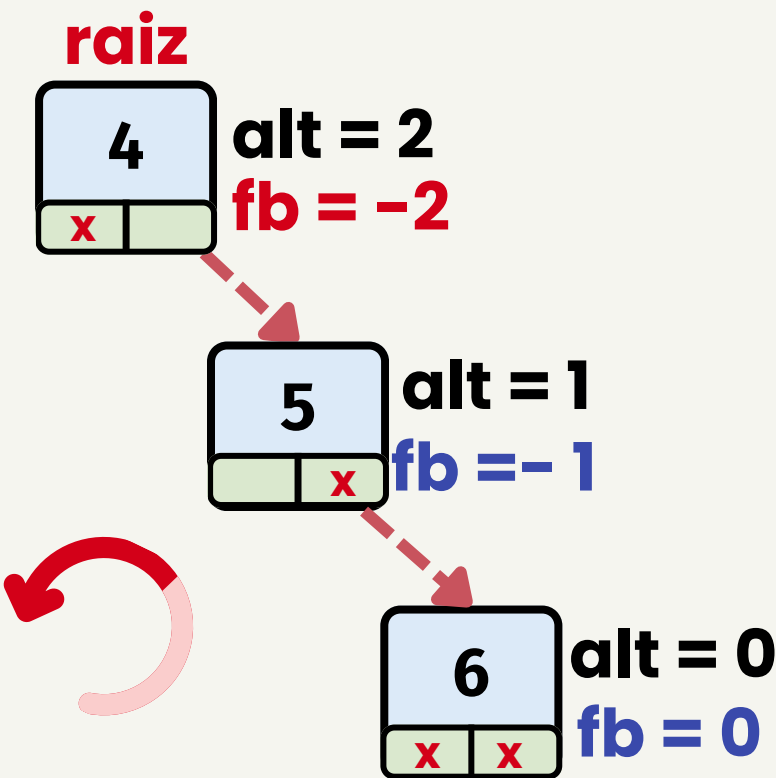
```
NoArv* rotacaoDireita(NoArv* raiz) {  
    NoArv* novaRaiz = raiz->esq;  
    NoArv* filho = novaRaiz->dir;  
  
    novaRaiz->dir = raiz;  
    raiz->esq = filho;  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    novaRaiz->altura = 1 + maiorValor(alturaDoNo(novaRaiz->esq), alturaDoNo(novaRaiz->dir));  
  
    return novaRaiz;  
}
```



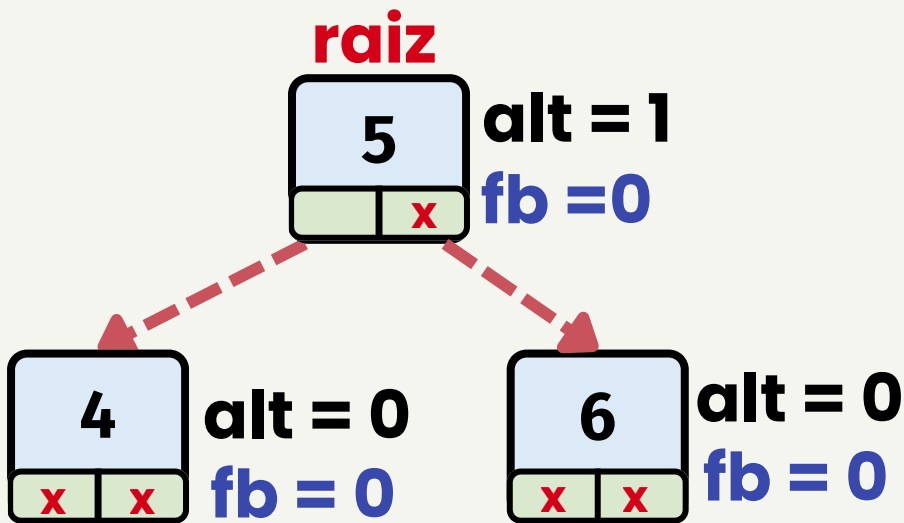
ROTAÇÕES DIREITA-ESQUERDA



ROTAÇÃO À DIREITA

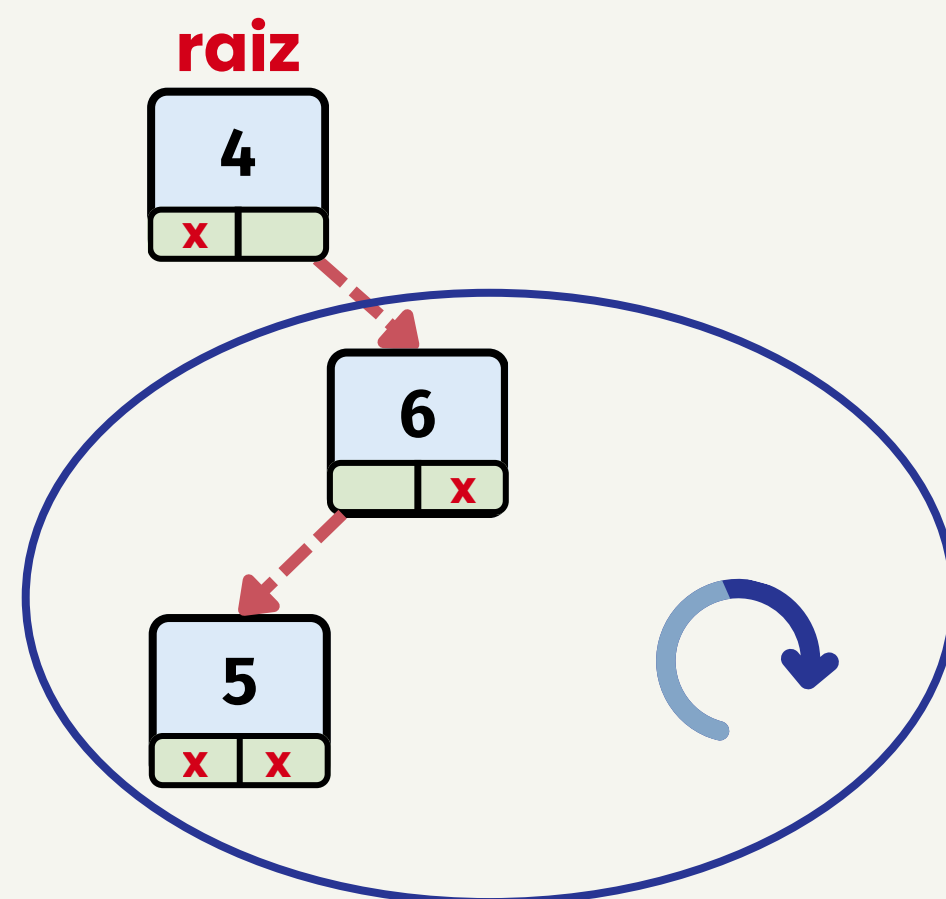


ROTAÇÃO À ESQUERDA



ÁRVORE BALANCEADA

ROTAÇÕES DIREITA-ESQUERDA

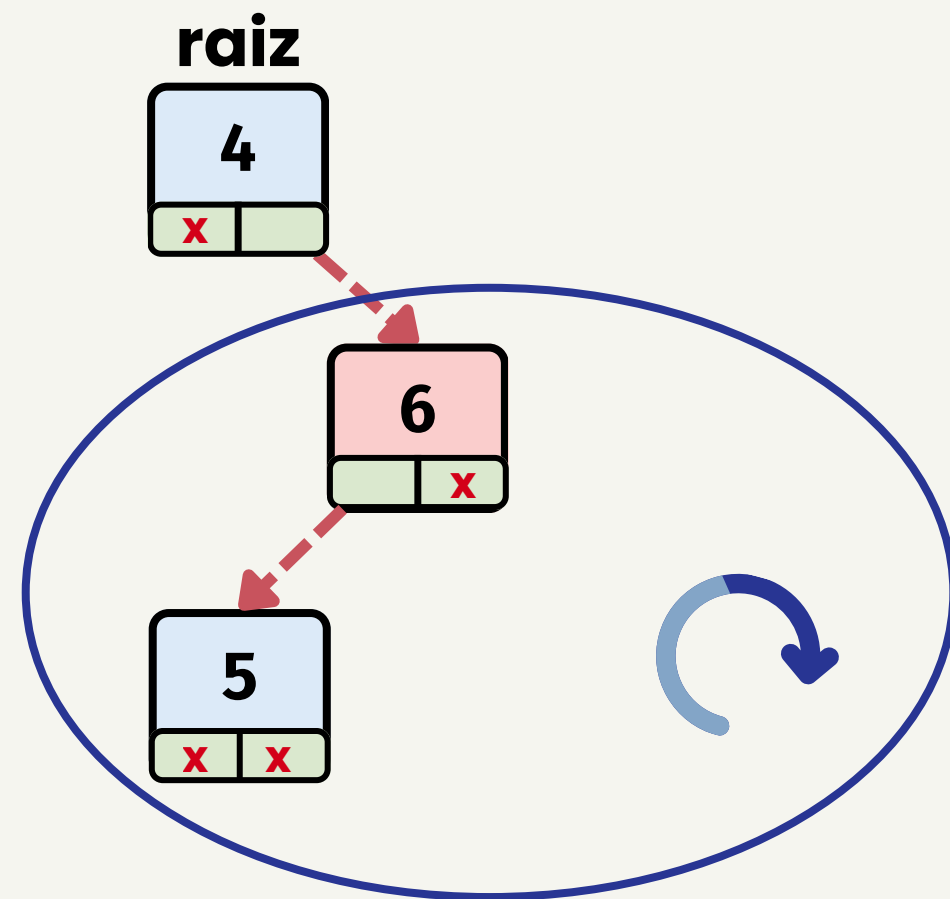


ROTAÇÃO À DIREITA

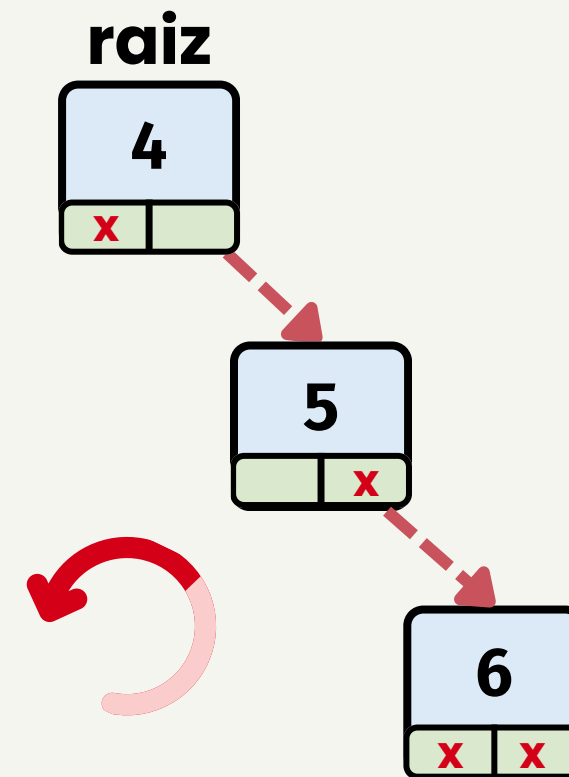
```

NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {
    raiz->dir = rotacaoDireita(raiz->dir);
    return rotacaoEsquerda(raiz);
}
  
```

ROTAÇÕES DIREITA-ESQUERDA



ROTAÇÃO À DIREITA

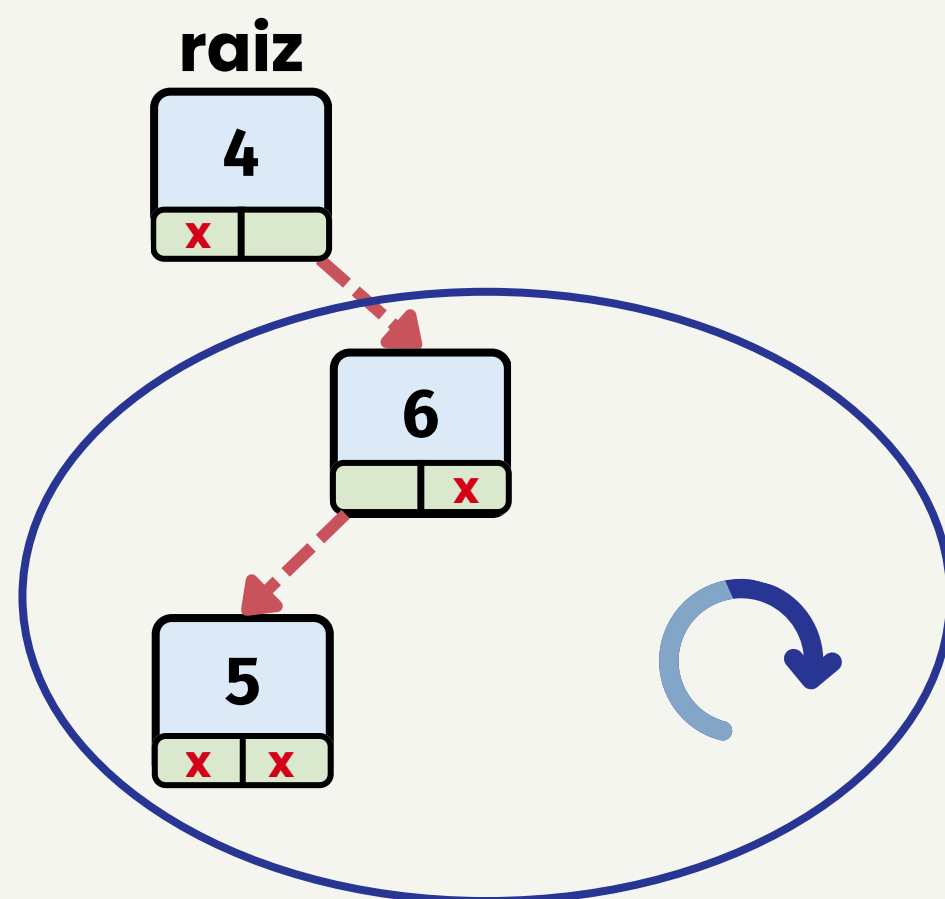


ROTAÇÃO À ESQUERDA

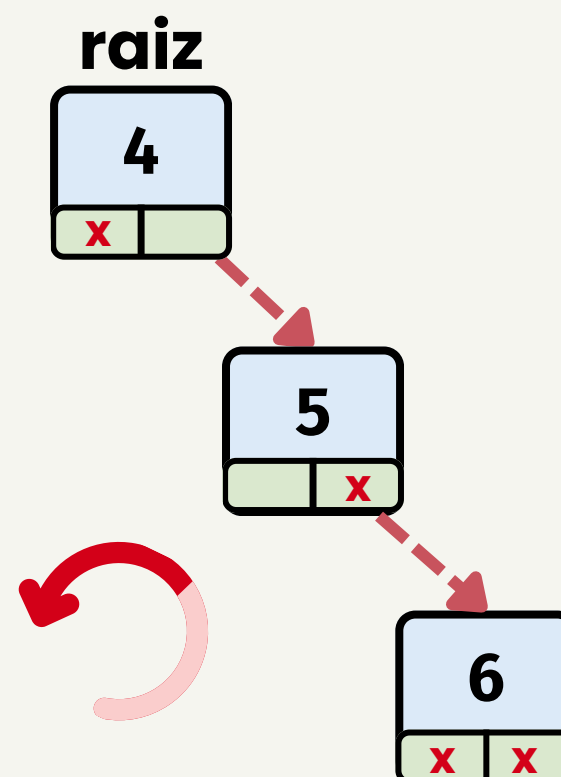
```

NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {
    raiz->dir = rotacaoDireita(raiz->dir);
    return rotacaoEsquerda(raiz);
}
  
```

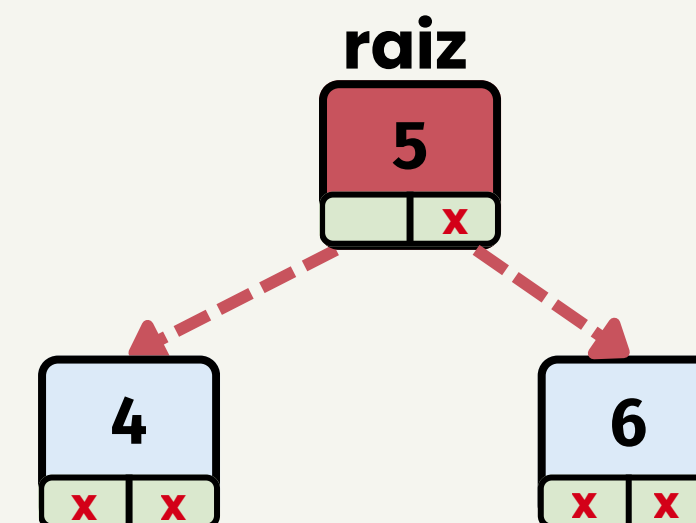
ROTAÇÕES DIREITA-ESQUERDA



ROTAÇÃO À DIREITA



ROTAÇÃO À ESQUERDA

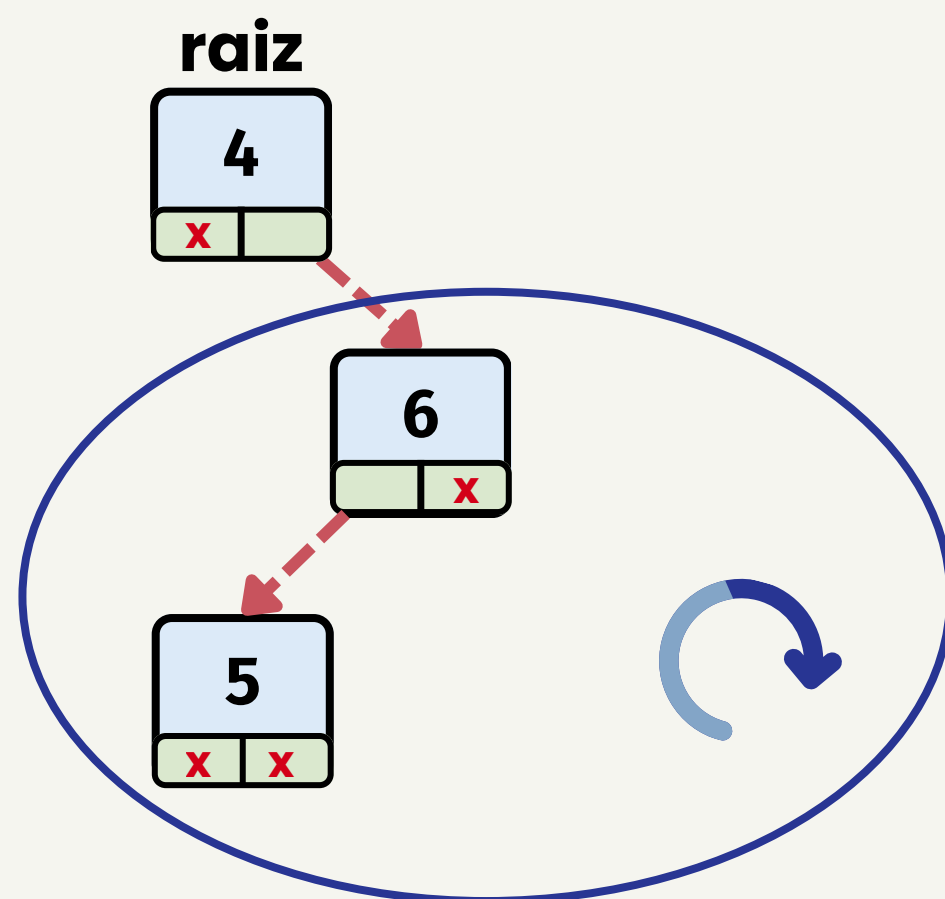


ÁRVORE BALANCEADA

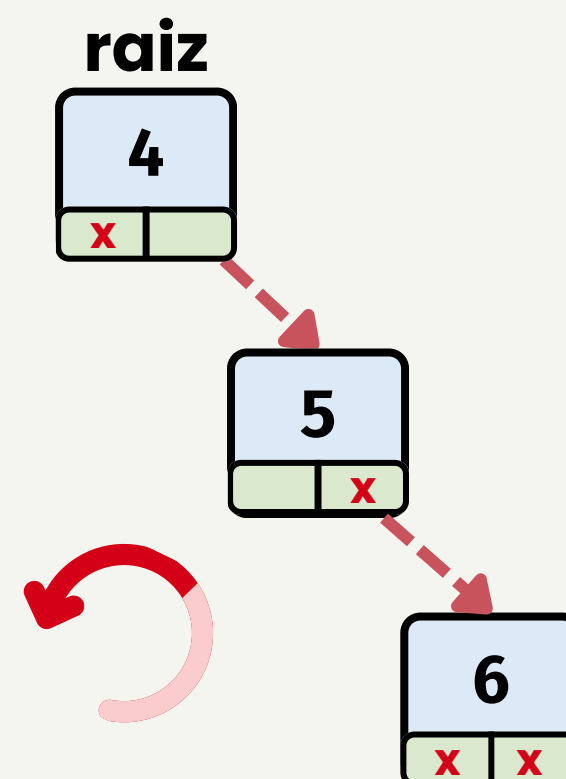
```

NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {
    raiz->dir = rotacaoDireita(raiz->dir);
    return rotacaoEsquerda(raiz);
}
  
```

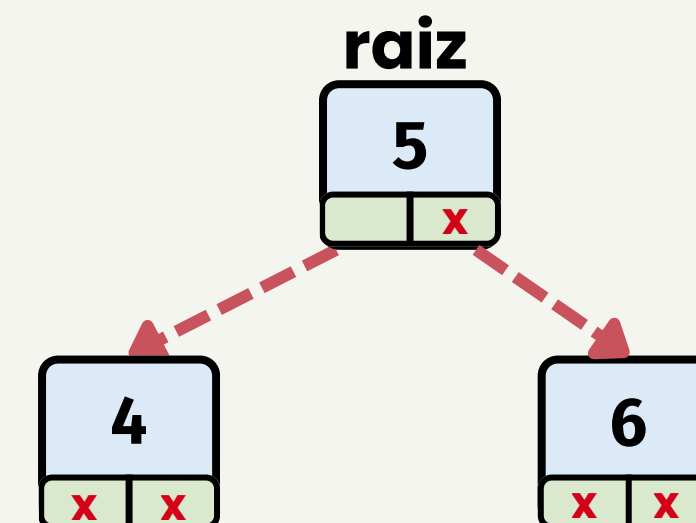
ROTAÇÕES DIREITA-ESQUERDA



ROTAÇÃO À DIREITA



ROTAÇÃO À ESQUERDA



ÁRVORE BALANCEADA

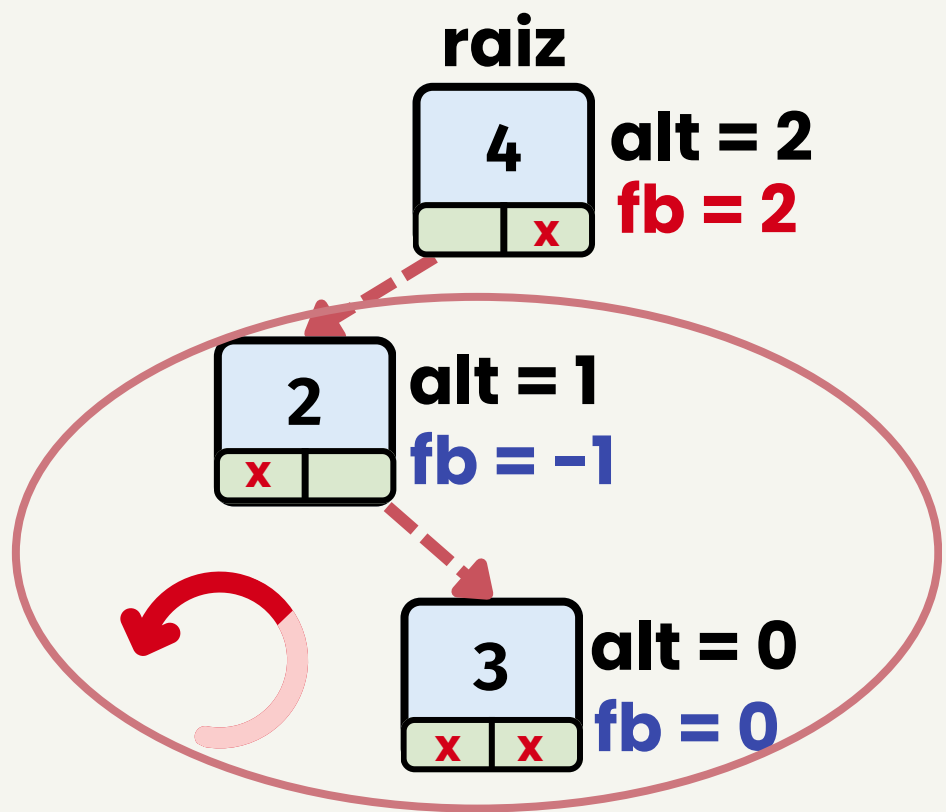
```

NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {
    raiz->dir = rotacaoDireita(raiz->dir);
    return rotacaoEsquerda(raiz);
}
  
```

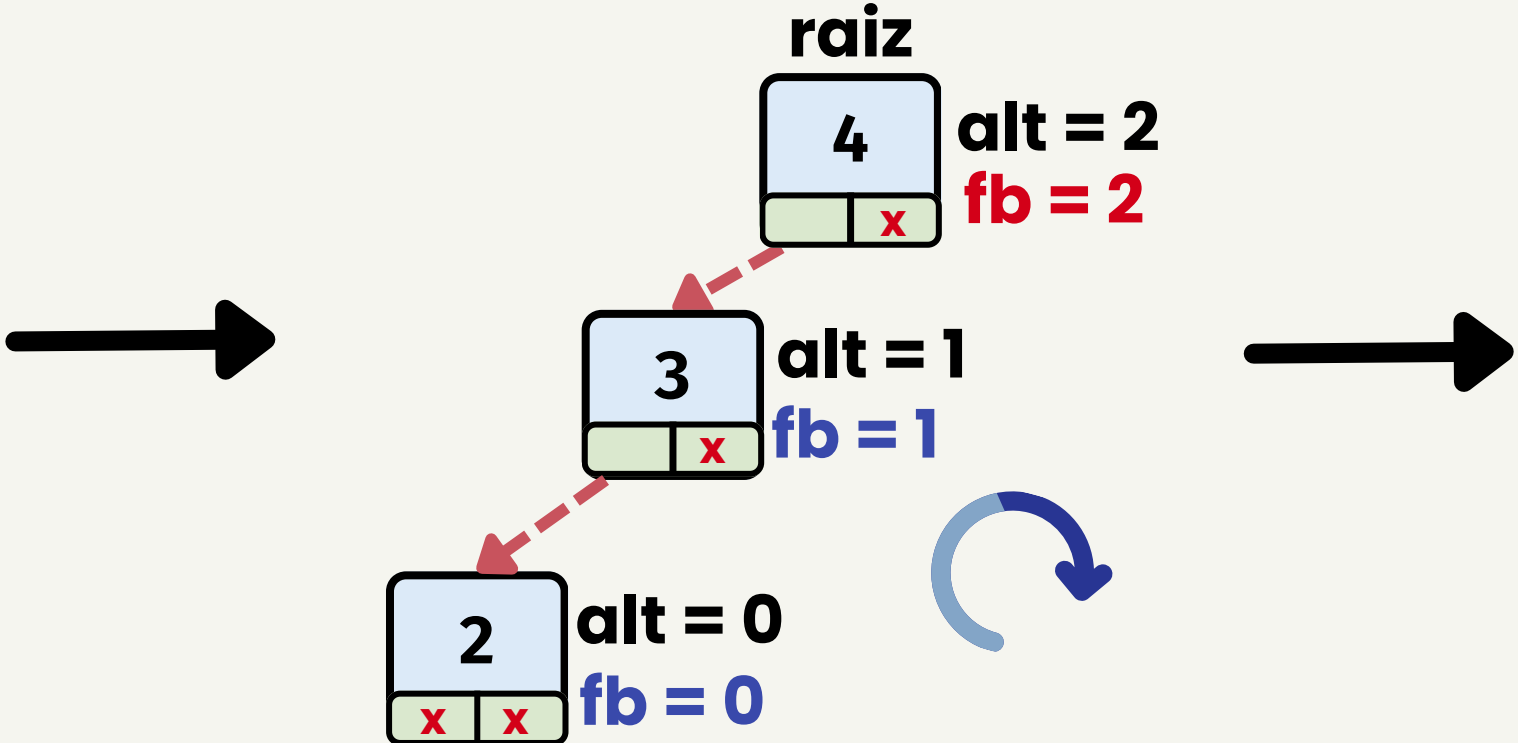


Não é necessário alterar as alturas, pois nas outras funções isso já é feito!!!

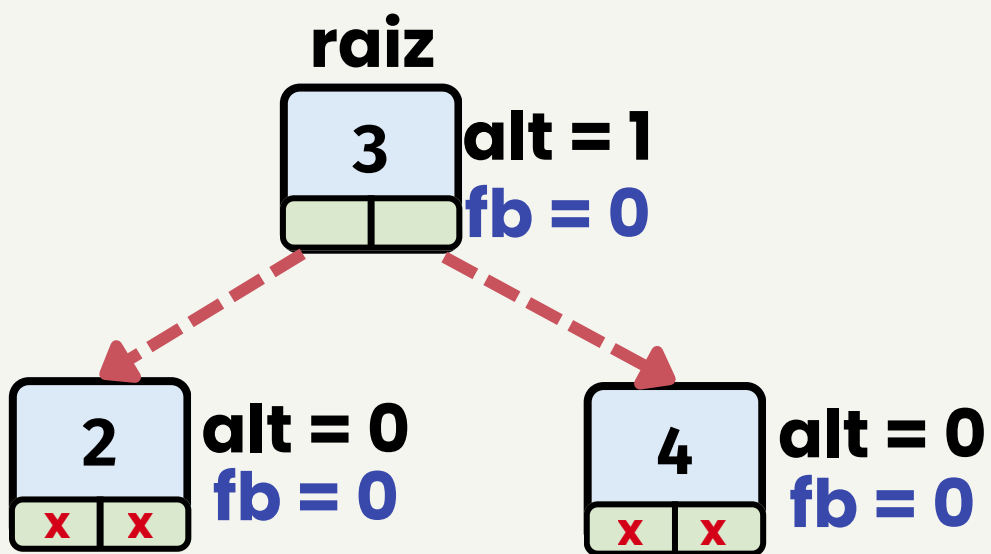
ROTAÇÕES ESQUERDA-DIREITA



ROTAÇÃO À ESQUERDA

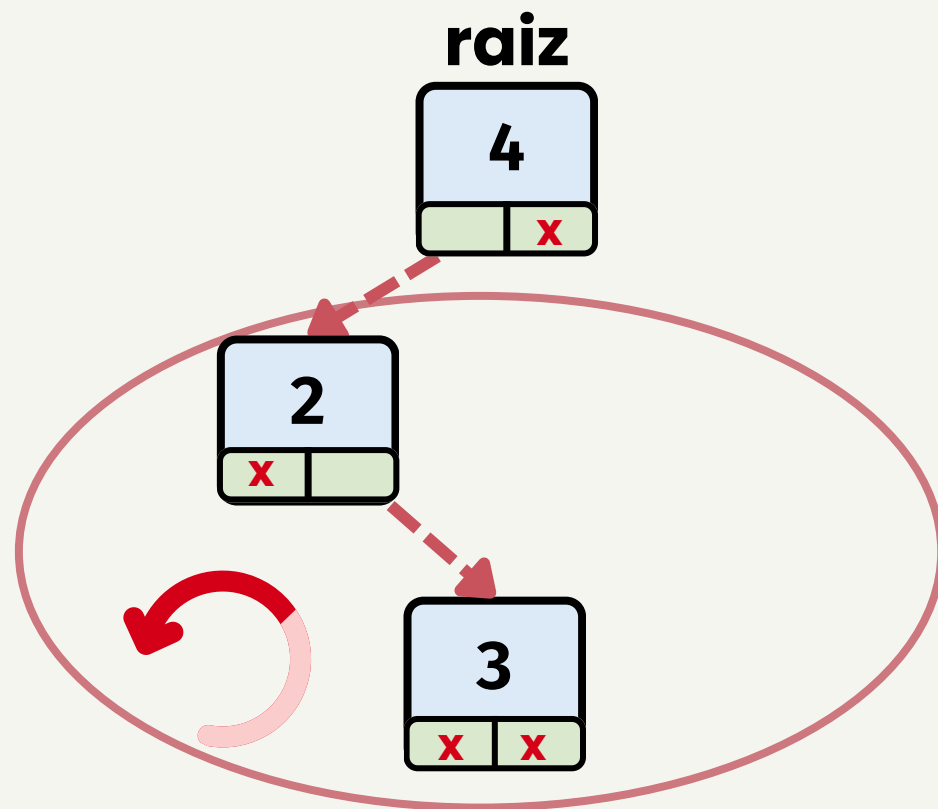


ROTAÇÃO À DIREITA



ÁRVORE BALANCEADA

ROTAÇÕES ESQUERDA-DIREITA

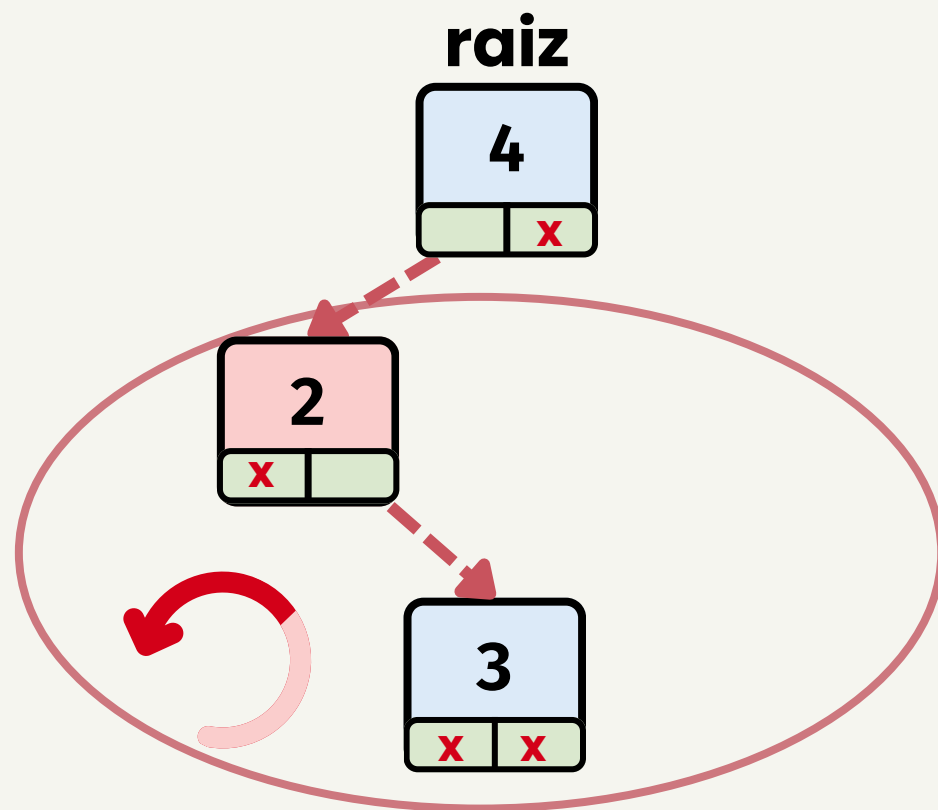


ROTAÇÃO À ESQUERDA

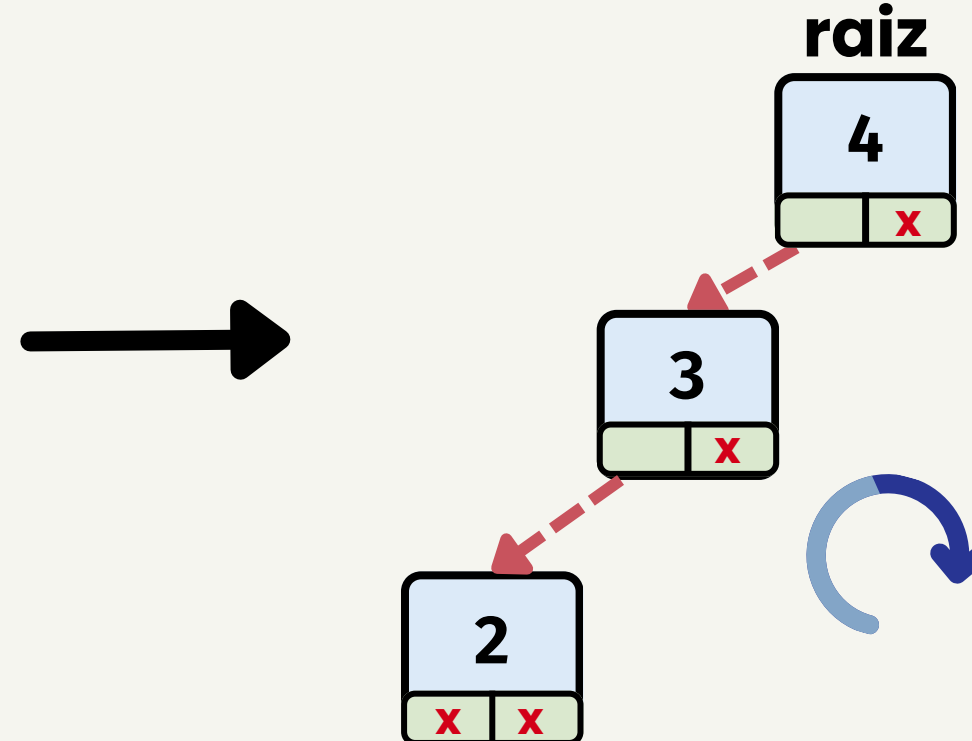
```

NoArv* rotacaoEsquerdaDireita(NoArv* raiz) {
    raiz->esq = rotacaoEsquerda(raiz->esq);
    return rotacaoDireita(raiz);
}
    
```

ROTAÇÕES ESQUERDA-DIREITA



ROTAÇÃO À ESQUERDA

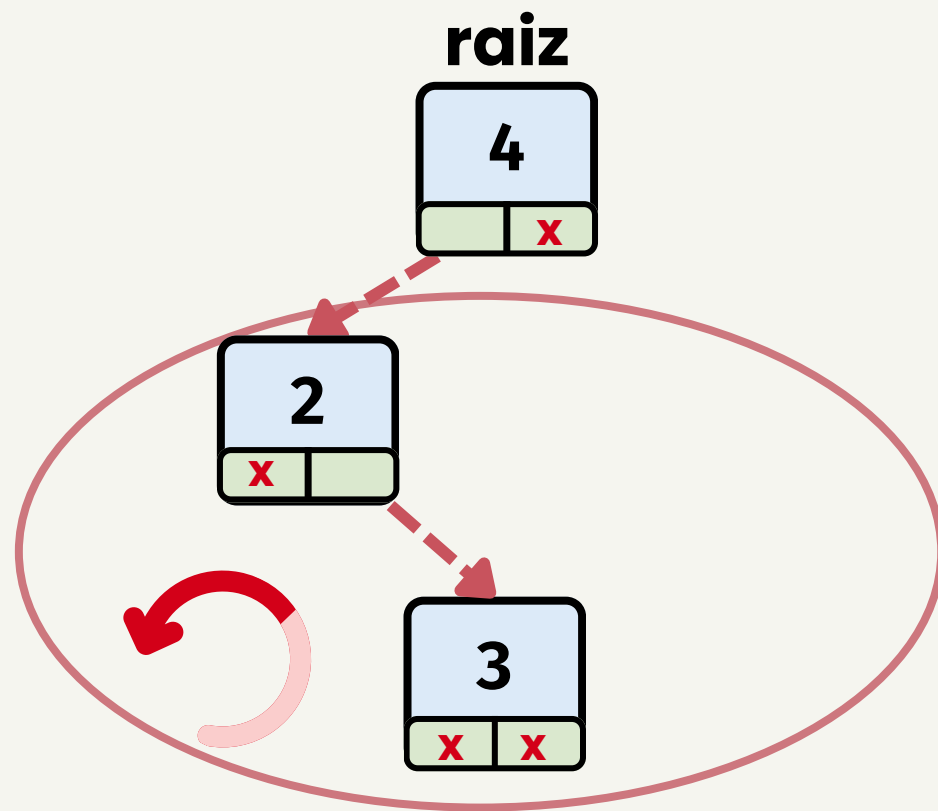


ROTAÇÃO À DIREITA

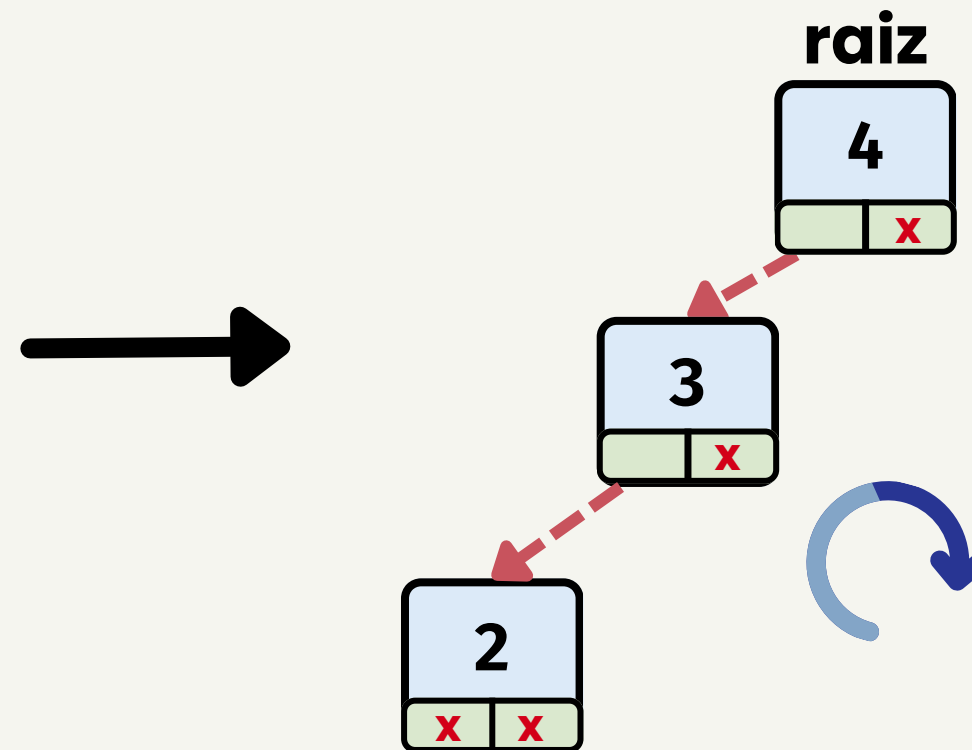
```

NoArv* rotacaoEsquerdaDireita(NoArv* raiz) {
    raiz->esq = rotacaoEsquerda(raiz->esq);
    return rotacaoDireita(raiz);
}
  
```

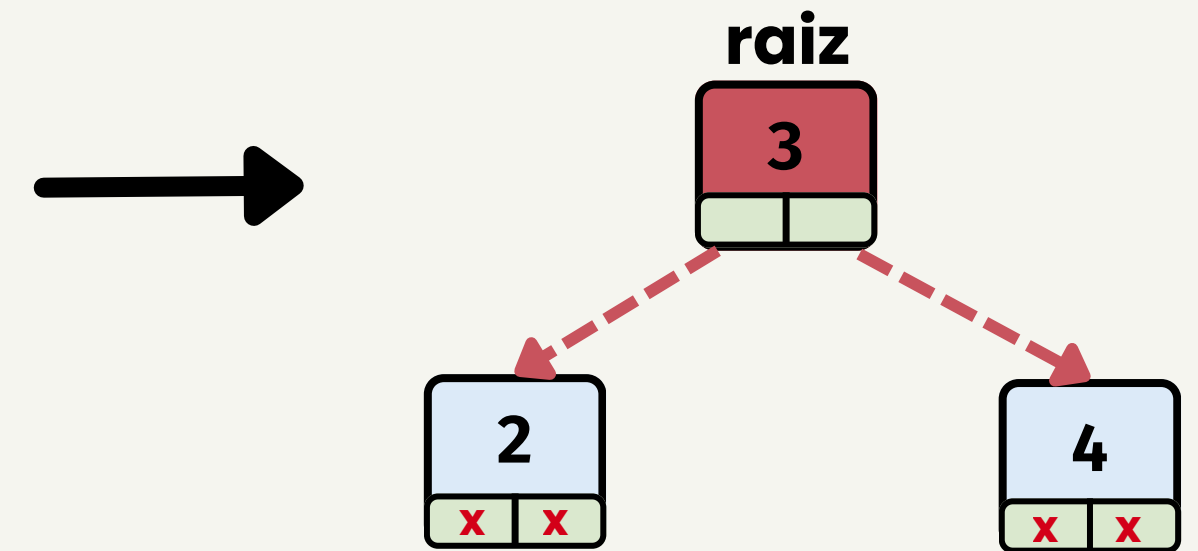
ROTAÇÕES ESQUERDA-DIREITA



ROTAÇÃO À ESQUERDA



ROTAÇÃO À DIREITA



ÁRVORE BALANCEADA

```

NoArv* rotacaoEsquerdaDireita(NoArv* raiz) {
    raiz->esq = rotacaoEsquerda(raiz->esq);
    return rotacaoDireita(raiz);
}
  
```

ROTAÇÕES ESQUERDA-DIREITA

```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }
    return raiz;
}
```

ROTAÇÕES ESQUERDA-DIREITA

```
NoArv* balancear(NoArv* raiz) {  
    int fb = fatorBalanceamento(raiz);  
  
    if (fb > 1) {  
        if (fatorBalanceamento(raiz->esq) >= 0) {  
            return rotacaoDireita(raiz);  
        } else {  
            return rotacaoEsquerdaDireita(raiz);  
        }  
    } else if (fb < -1) {  
        if (fatorBalanceamento(raiz->dir) <= 0) {  
            return rotacaoEsquerda(raiz);  
        } else {  
            return rotacaoDireitaEsquerda(raiz);  
        }  
    }  
    return raiz;  
}
```

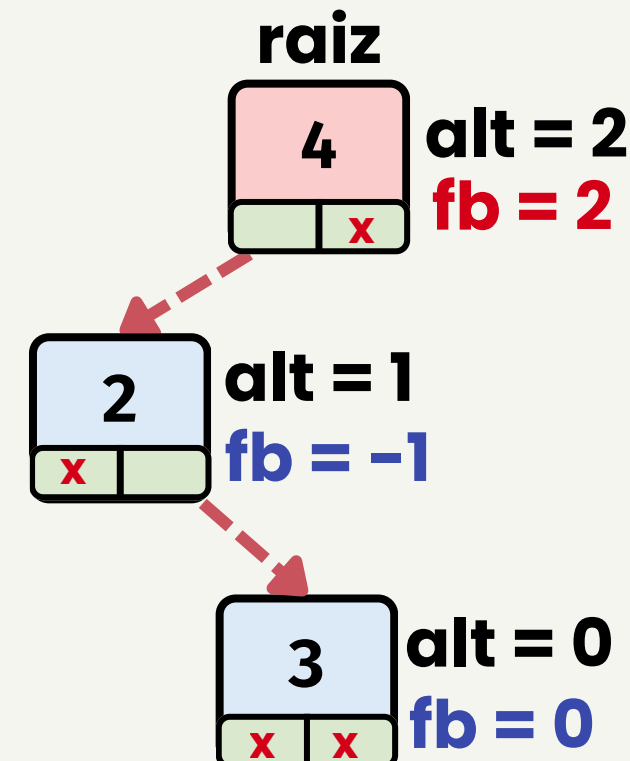
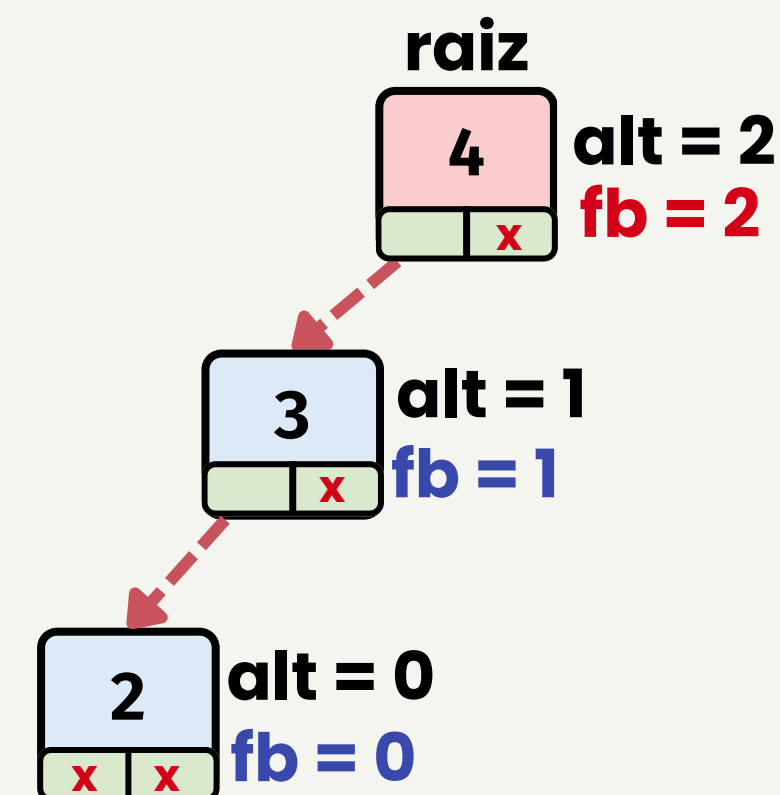
ROTAÇÕES ESQUERDA-DIREITA

```

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
    
```

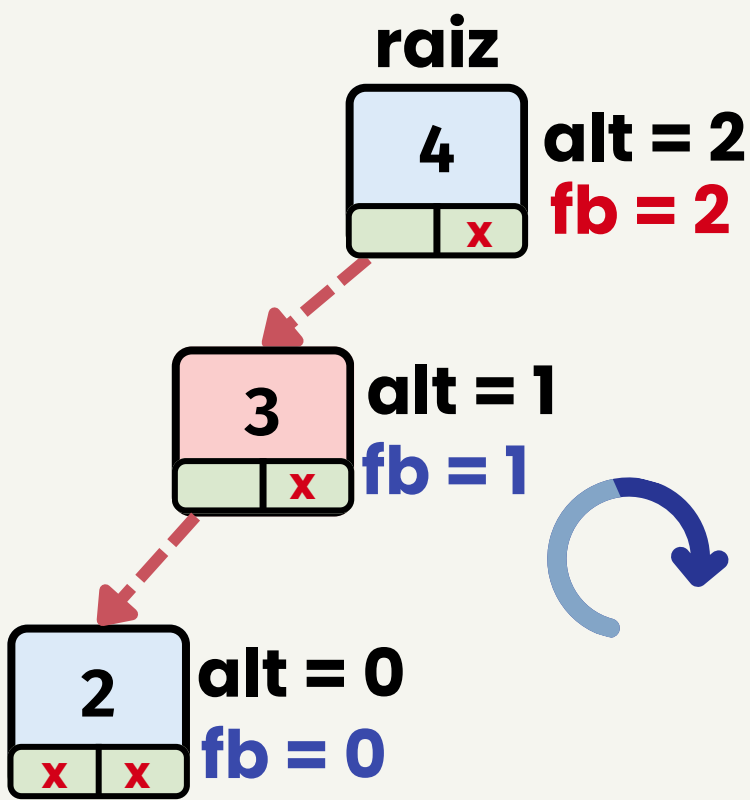


ROTAÇÕES ESQUERDA-DIREITA

```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```



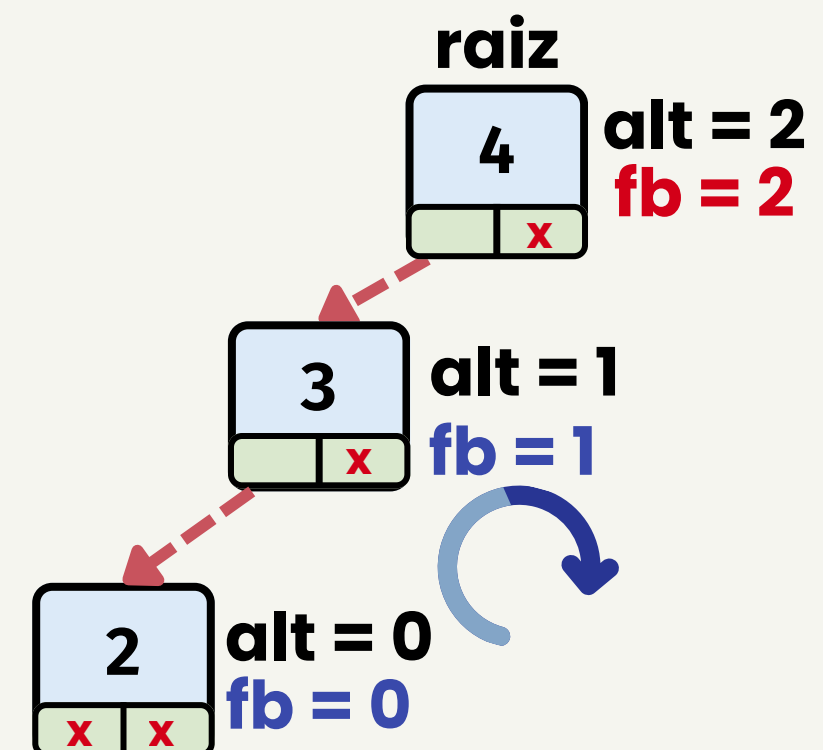
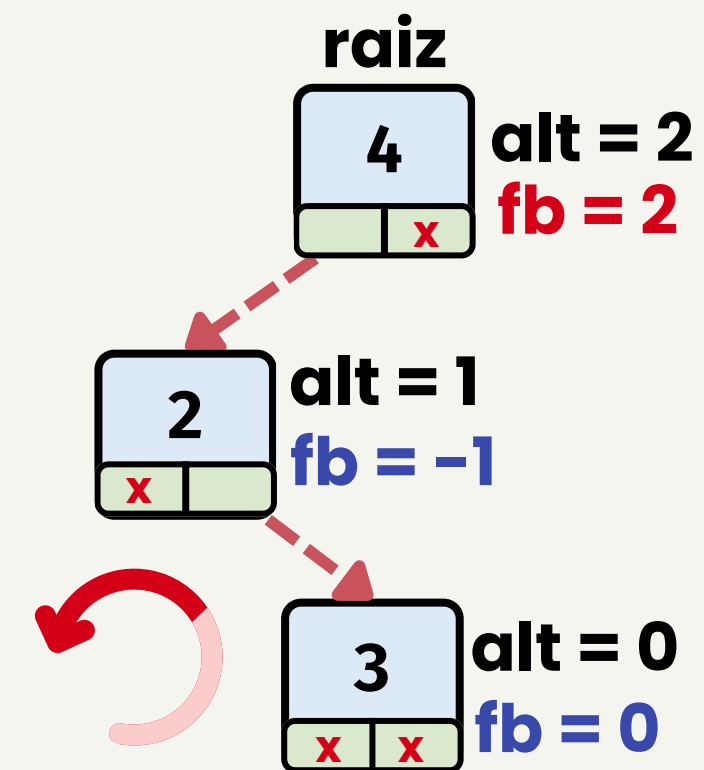
ROTAÇÕES ESQUERDA-DIREITA

```

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
    
```



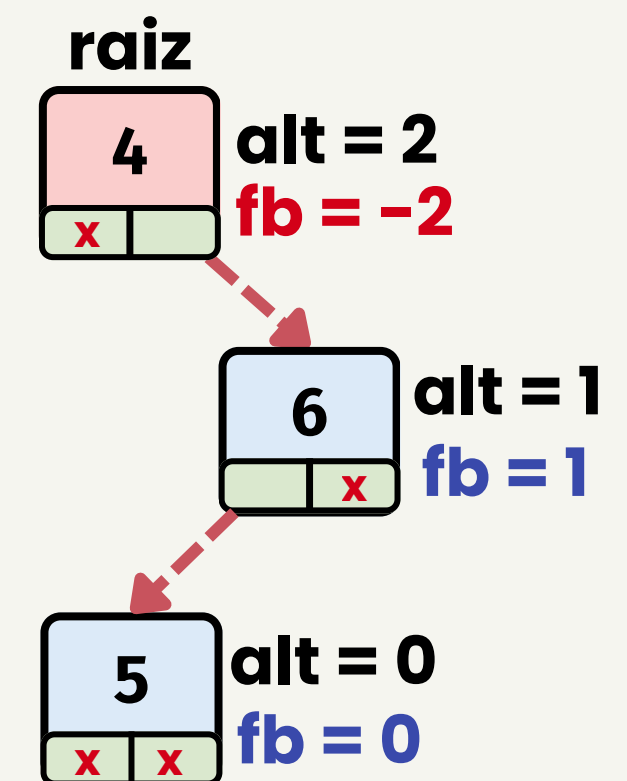
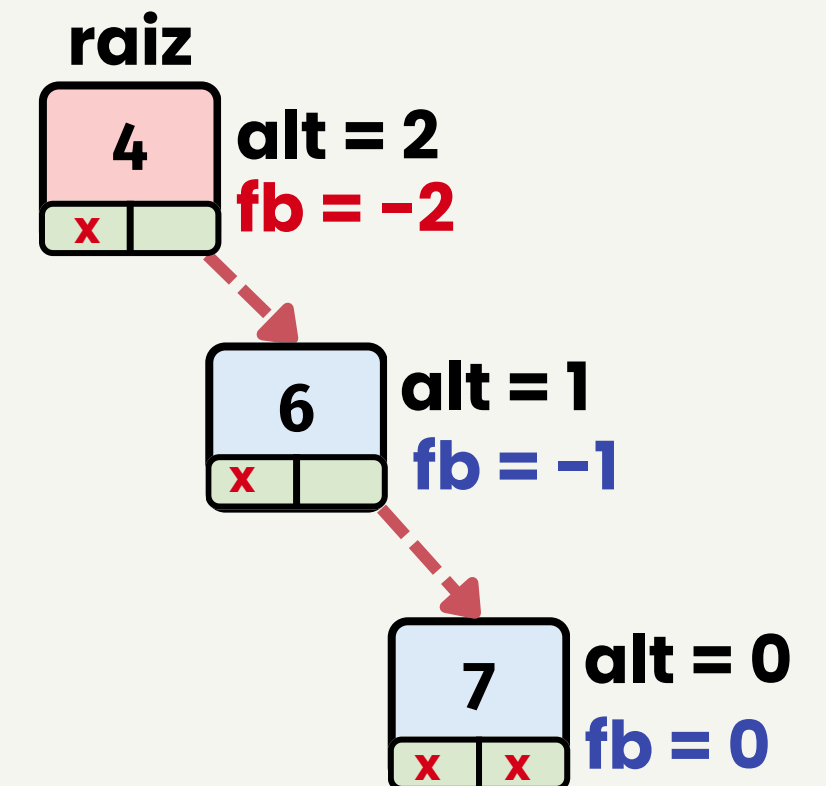
ROTAÇÕES ESQUERDA-DIREITA

```

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
    
```



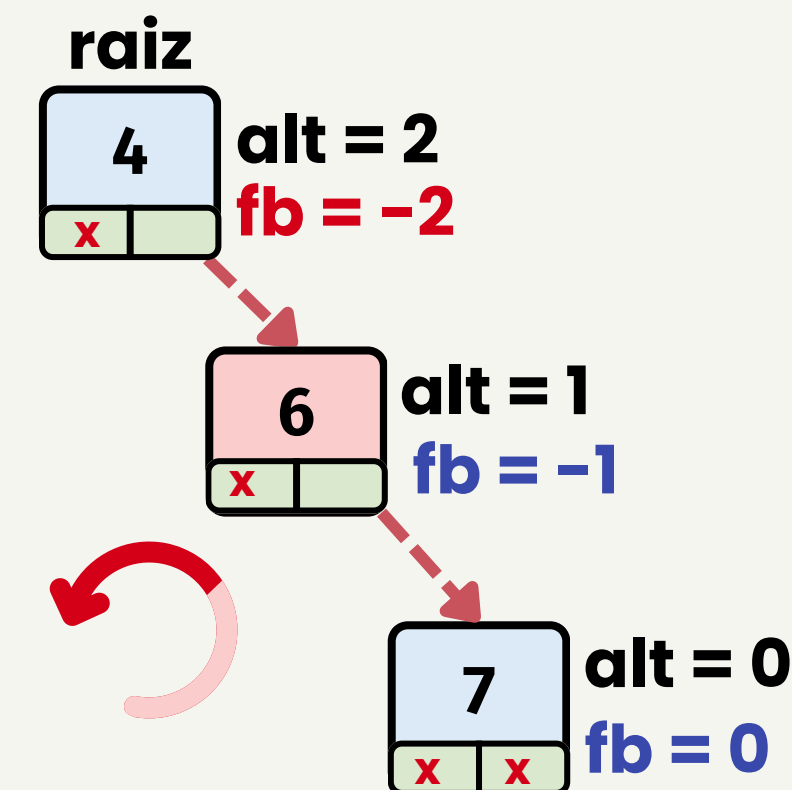
ROTAÇÕES ESQUERDA-DIREITA

```

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
    
```

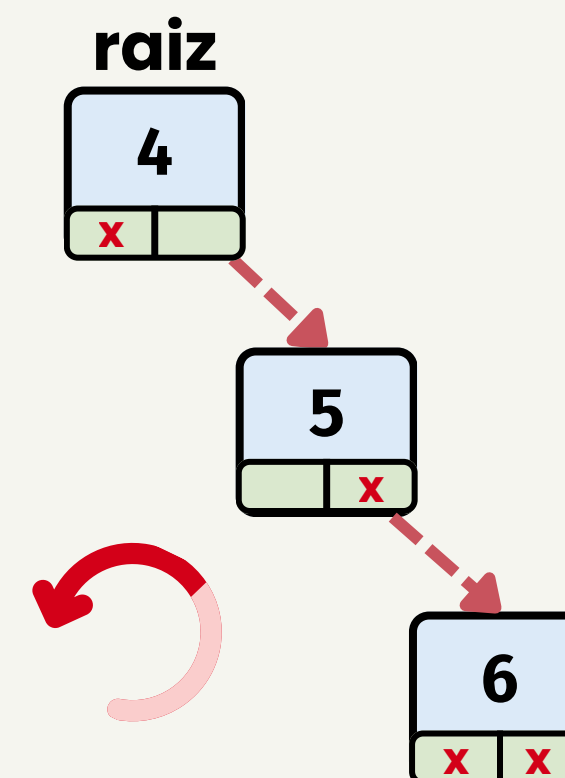
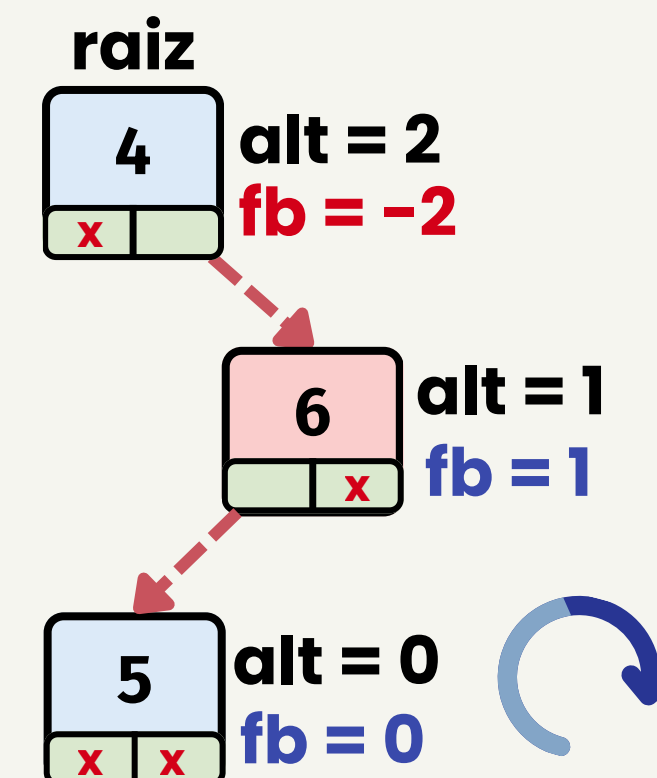


ROTAÇÕES ESQUERDA-DIREITA

```

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }
    return raiz;
}
    
```



Próxima aula:

- **Modificar as funções inserir e remover da ABB para se tornar AVL**
- **Exercícios**